HashiCorp Certified Terraform Associate 2026

PPT Version

PPT Release Date = 4th January 2026

We regularly release new version of PPT when we update this course.

Please check regularly that you are using the latest version.

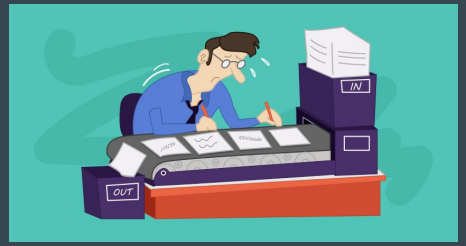
The Latest Version Details are mentioned in the PPT Lecture in Section 1.

Understanding the Need

My personal journey with Terraform began with implementing the “AWS Hardening” guidelines.

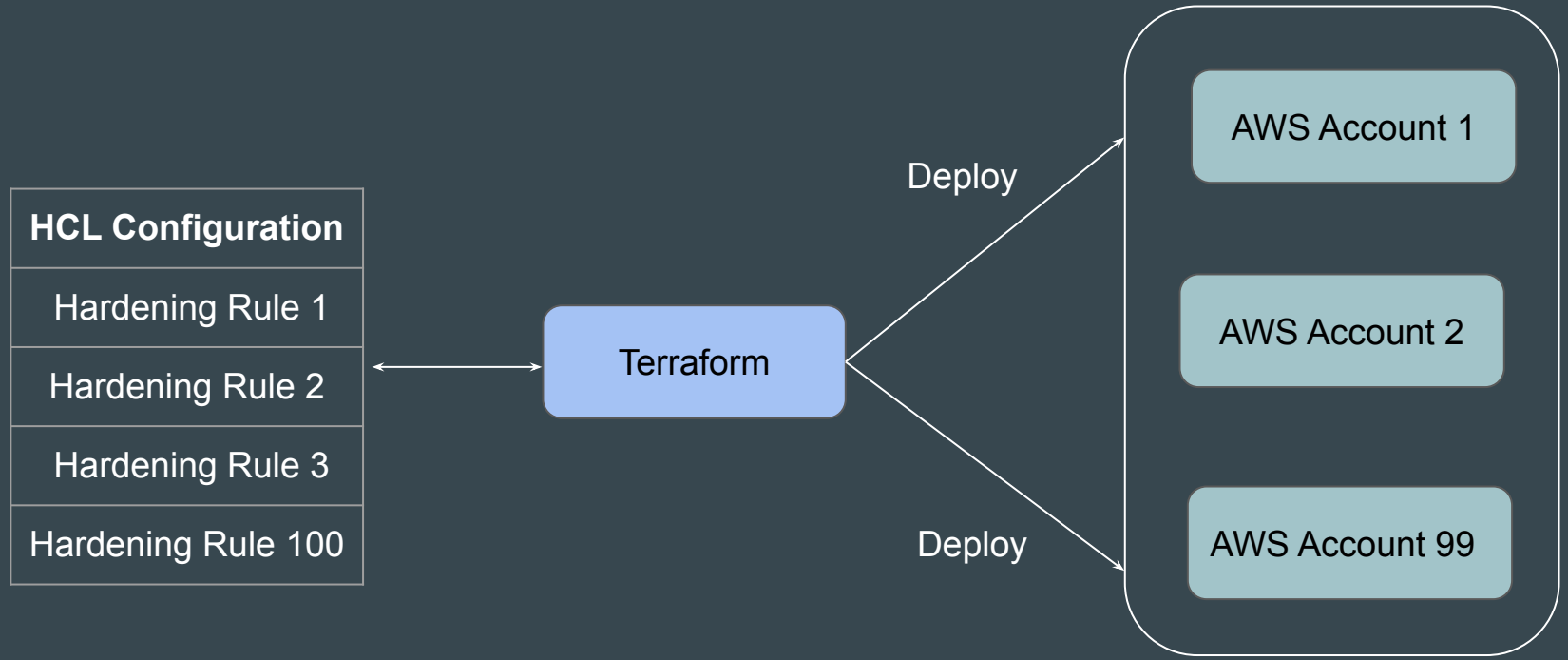
Back then, the guidelines spanned over 100 pages and required 2-3 days to apply to a single account.

Today, these guidelines have grown to more than 250 pages.



Challenge that Terraform Solves

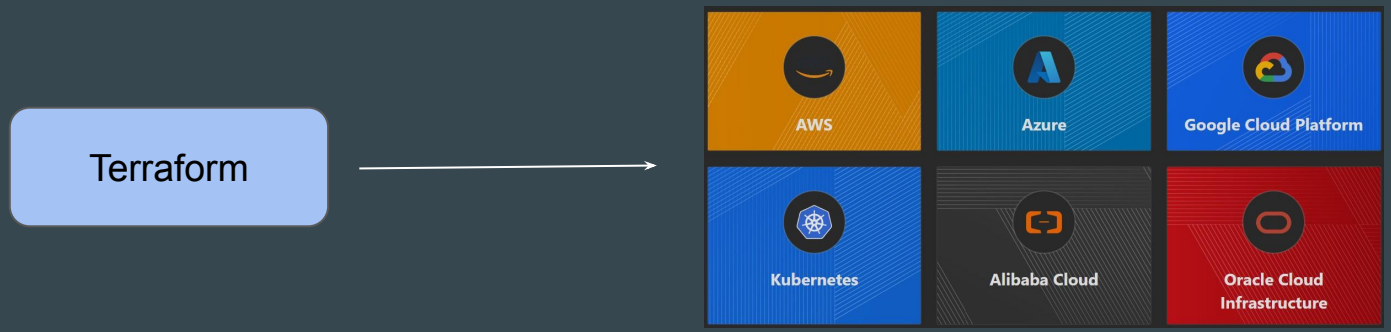
Terraform allows us to create reusable code that can deploy an identical set of infrastructure resources in a repeatable fashion.



Amazing Terraform

One of the great benefits of Terraform is that it supports thousands of providers.

Once you learn Terraform Core concepts, you can write code to create and manage infrastructure across all the providers.



Overview of Terraform Certification

Terraform has become one of the most popular and widely used tools for creating and managing infrastructure, and a de facto IaC tool for DevOps.

HashiCorp has introduced an official Terraform certification designed to validate students' knowledge and skills in core Terraform concepts.



What Does this Course Cover?

We start this Terraform course from absolute scratch. Once core concepts are covered, we move on to advanced topics.

We cover ALL the topics of the official exams.

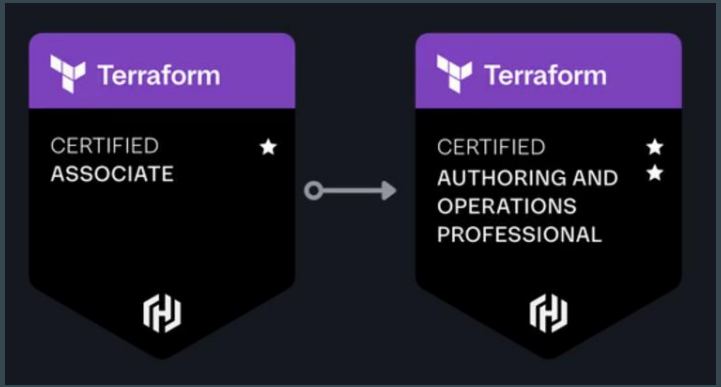


Long Term Vision

Due to immense popularity, HashiCorp has also released a Terraform Professional certification.

A long, difficult, practical-based exam of 4 hours

We launched the first Terraform Professional certification course in the world.



About Me

- DevSecOps Engineer - Defensive Security.
- Teaching is one of my passions.
- I have total of 17 courses, and around 440,000+ students now.

Something about me :-



- HashiCorp Certified Terraform [Associate and Professional]
- Certified Ethical Hacker
- AWS Certified [DevOps Pro, SA Pro, Advanced Networking, Security Specialty …]
- RedHat Certified Architect (RHCA) + 13 more Certifications
- Part time Security Consultant

Join us in our Adventure

Be Awesome



kplabs.in/chat



kplabs.in/linkedin

Exploring Exam BluePrint

Understanding the Basics

This is a certification specific course and we cover all the pointers that are part of the official exam blueprint.

Exam Objectives

1	**Infrastructure as Code (IaC) with Terraform**
1a	Explain what IaC is
1b	Describe the advantages of IaC patterns
1c	Explain how Terraform manages multi-cloud, hybrid cloud, and service-agnostic workflows
2	**Terraform fundamentals**
2a	Install and version Terraform providers
2b	Describe how Terraform uses providers
2c	Write Terraform configuration using multiple providers
2d	Explain how Terraform uses and manages state

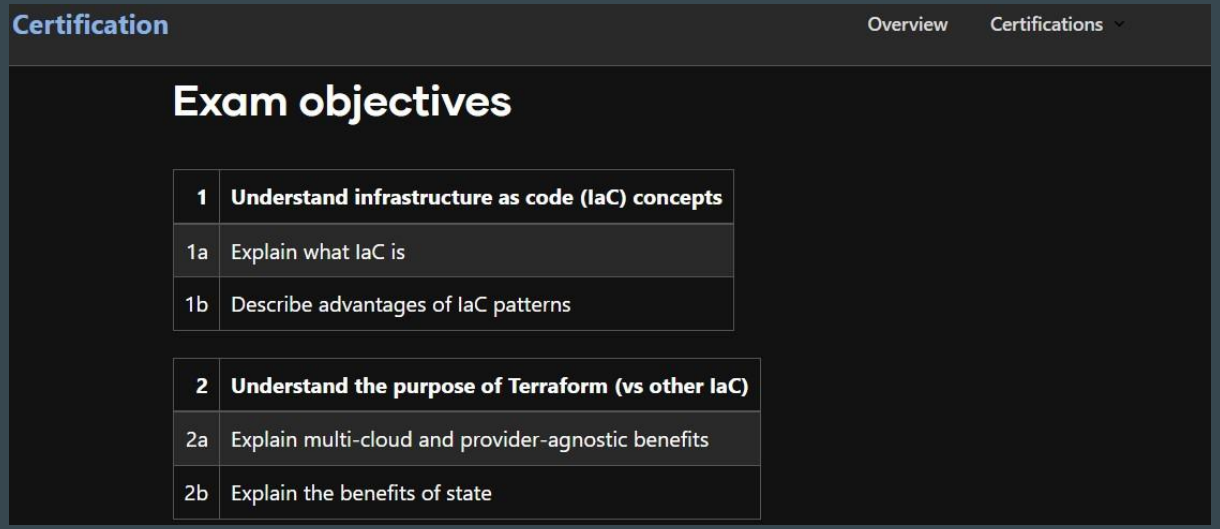
Point to Note

The arrangement of topics in this course is a little different from the exam blueprint to ensure this course remains beginner friendly and topics are covered in a step by step manner.

About the Course and Important Resources

Understanding the Basics

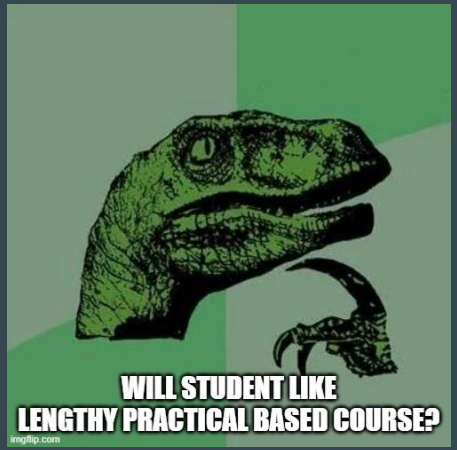
This is a certification specific course and we cover all the pointers that are part of the official exam blueprint.



2 Choice for Instructor

1. Cover Theory and Basic Demos -- Course Length Maintained

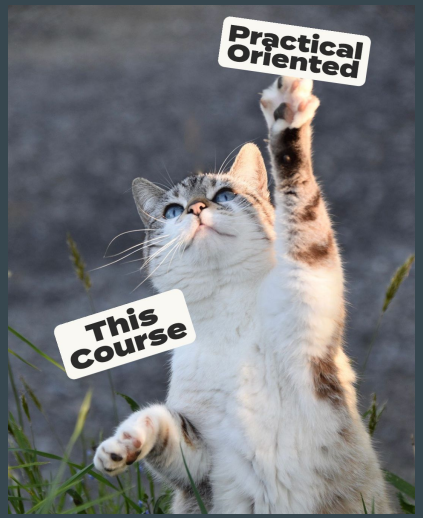
2. Cover Practically -- Course Length will be Longer



Our Choice

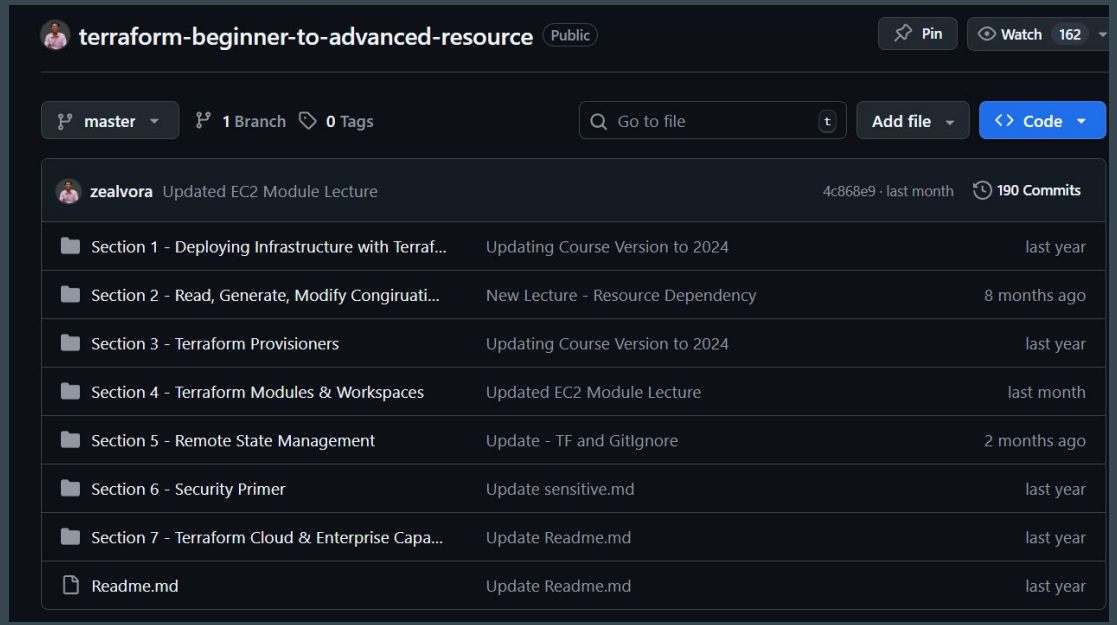
This course follows practical based approach.

The course will be lengthy but worth it.



Course Resource - GitHub

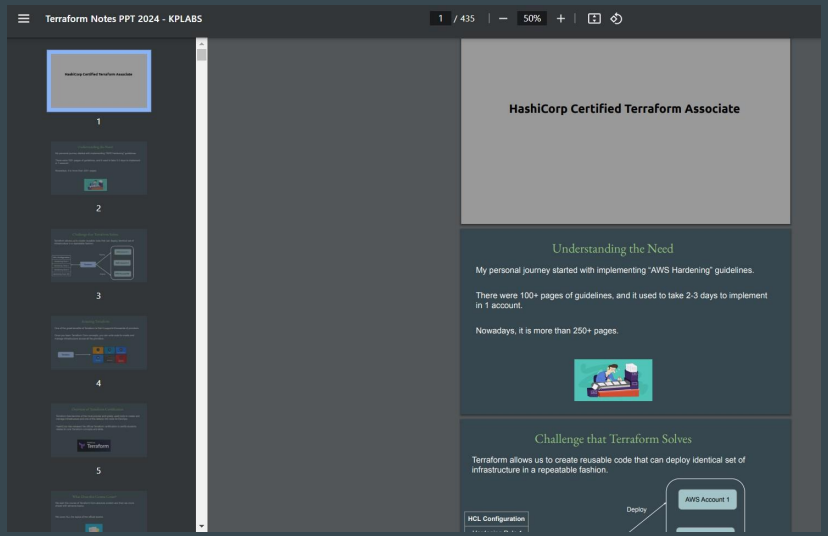
All the code that we use during practicals have been added to our GitHub page.



Course Resource - PPT Slides

All the slides we use in this course are available to download as PDFs.

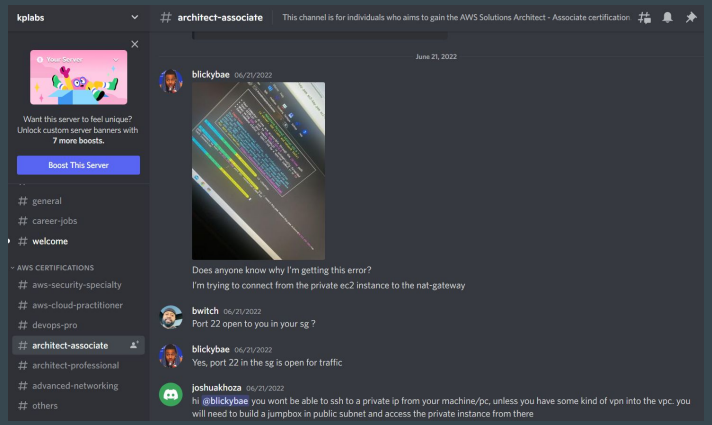
The PDF is attached as part of the lecture titled “Central PPT Notes”.



Our Community (Optional)

We also have a Discord community where individuals preparing for the same certification can connect, discuss, and seek technical support.

https://kplabs.in/chat

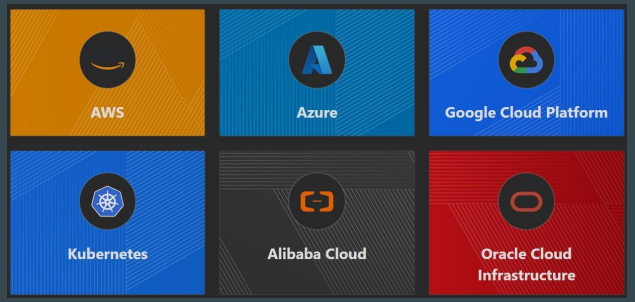


Important Note - Platform for This Course

Terraform supports hundreds of platforms like AWS, Azure, GCP, etc.

To learn Terraform concepts, we have to choose 1 platform for our testing.

For this course, we have chosen AWS.



Clarification Regarding AWS Platform

Aim of this course is to learn Core Concepts of Terraform and not AWS.

We use very basic AWS services like Virtual Machine, AWS users to demonstrate and Learn the Core Terraform concepts.

The Terraform structure and concepts remain SAME irrespective of platform.

We have hundreds of students from diverse platform backgrounds, such as Azure and GCP, who have completed this course and are actively implementing Terraform across platforms.

Infrastructure as Code (IAC)

Understanding the Basics

There are two ways in which you can create and manage your infrastructure:

- Manually approach.
- Through Automation



Work Requirement: Database Backup

I was assigned a task to take database backup every day at 10 PM and the backup had to be stored in Amazon S3 Storage with appropriate timestamp.

- db-backup-01-01-2024.sql
- db-backup-02-01-2024.sql

Initially due to lack of time, I used to manually take DB backup at 10 PM and upload it to S3.



Learning from this Work Requirement

If a particular task has to be done in an repeatable manner, it MUST be automated.

Points to Note:

1. Depending on the type of task, the tools for automation will change.

2. There are wide variety of Tools & Technologies used for Automation like Ansible, CloudFormation, Terraform, Python etc.

Example of a Single Service

Set of resources (Virtual Machine, Database, S3, AWS Users) must be created with exact similar configuration in Dev, Stage and Production environment.

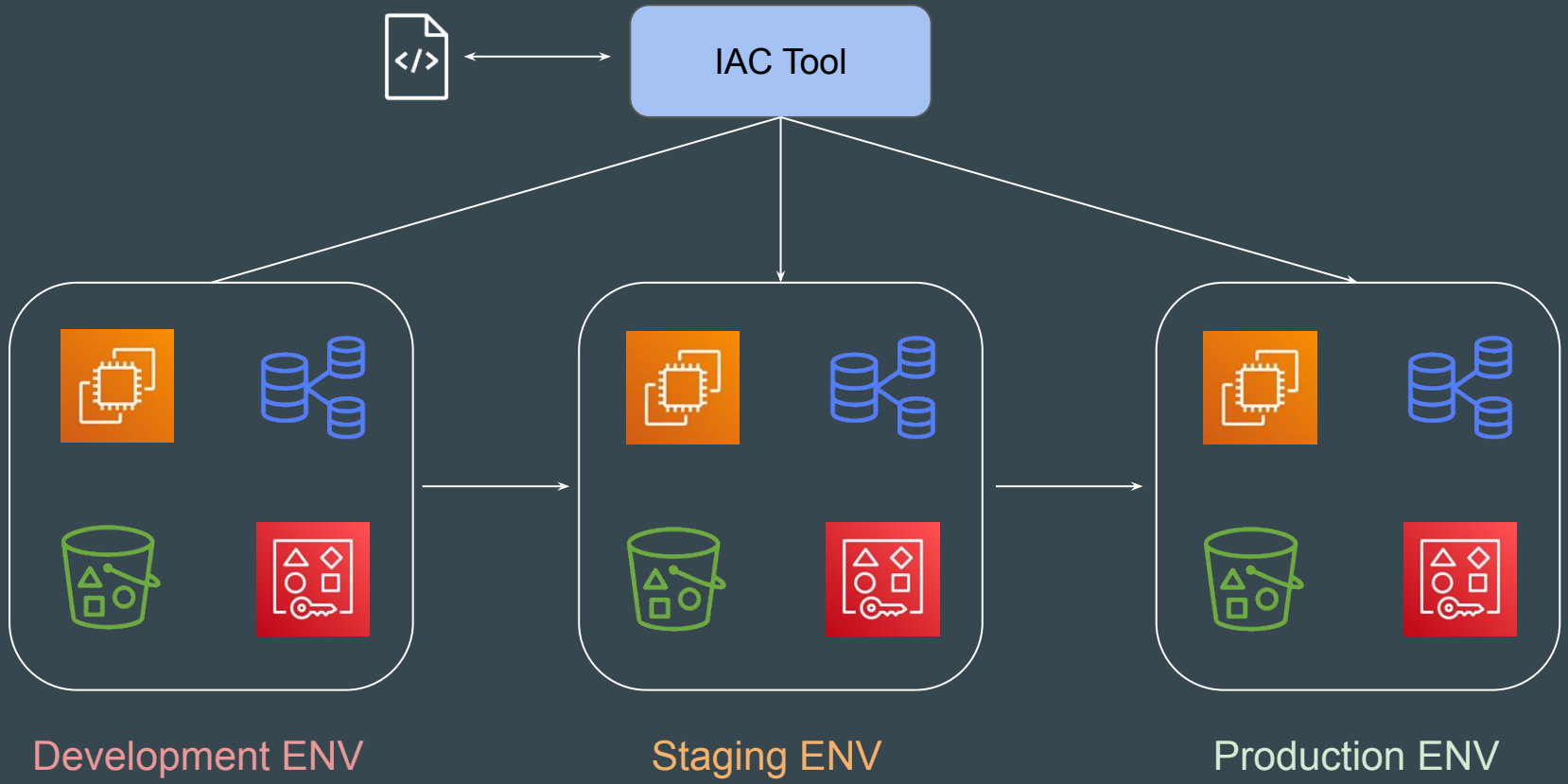


Development ENV

Staging ENV

Production ENV

Example of a Single Service - Automated Way



Basics of Infrastructure as Code

Infrastructure as Code (IaC) is the managing and provisioning of infrastructure through code instead of through manual processes.

test.yaml

```yaml
AWSTemplateFormatVersion: 2010-09-09
Description: Simple EC2

Resources:
  WebAppInstance:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: ami-079db87dc4c10ac91
      InstanceType: t2.micro
```

vm.tf > ...

```
resource "aws_instance" "myec2" {
    ami = "ami-079db87dc4c10ac91"
    instance_type = "t2.micro"
}
```

Benefits of Infrastructure As Code

There are several benefits of designing your infrastructure as code:

- Speed of Infrastructure Management.
- Low Risk of Human Errors.
- Version Control.
- Easy collaboration between Teams.

Choosing Right IAC Tool

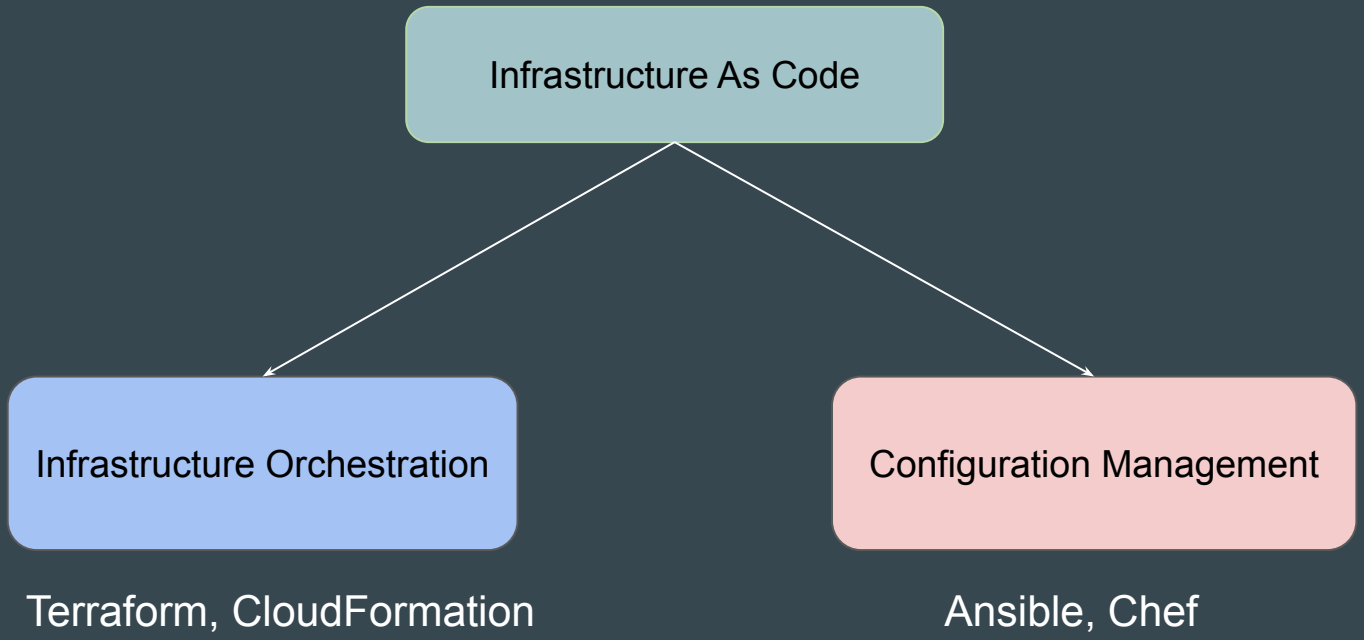
Available Tools

There are various types of tools that can allow you to deploy infrastructure as code :

- Terraform
- CloudFormation
- Heat
- Ansible
- SaltStack
- Chef, Puppet and others

Categories of Tools

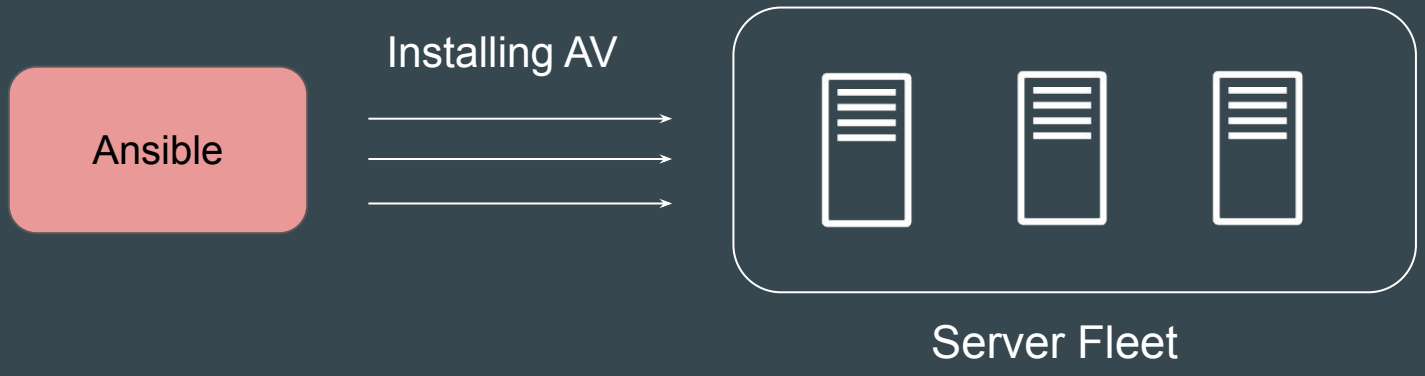
The tools are widely divided into two major categories



Configuration Management

Configuration Management tools are primarily used to maintain desired configuration of systems (inside servers)

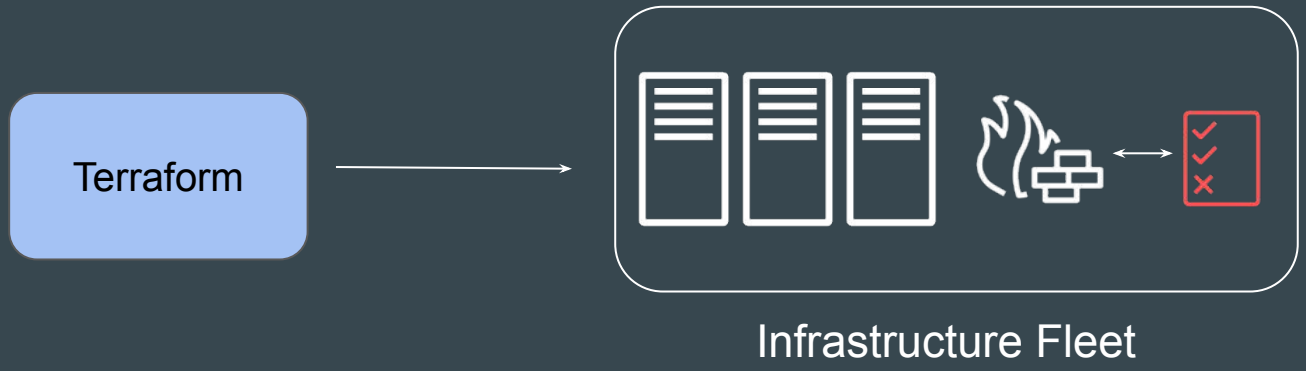
Example: ALL servers should have Antivirus installed with version 10.0.2



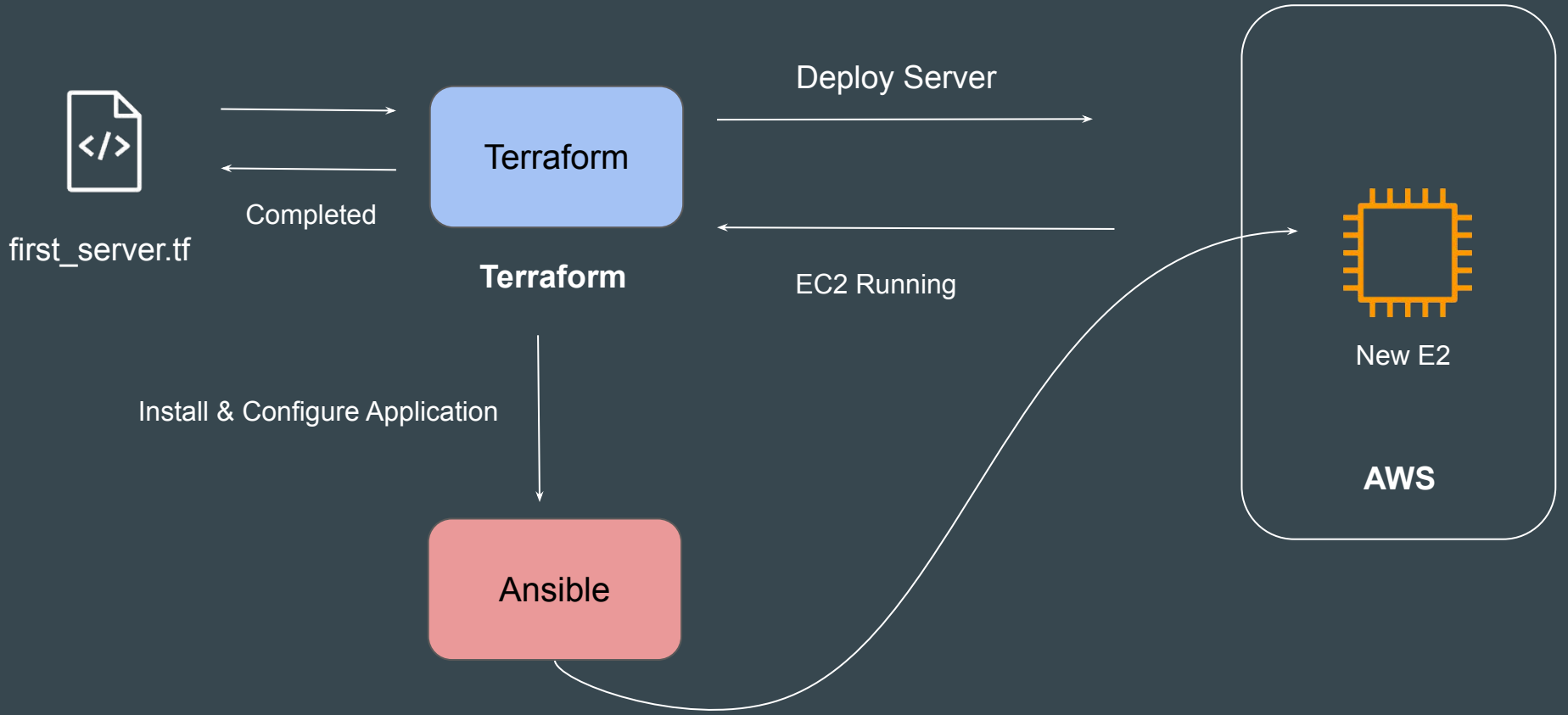
Infrastructure Orchestration

Infrastructure Orchestration is primarily used to create and manage infrastructure environments.

Example: Create 3 Servers with 4 GB RAM, 2 vCPUs. Each server should have firewall rule to allow SSH connection from Office IPs.



IAC & Configuration Management = Friends



How to choose IAC Tool?

i) Is your infrastructure going to be vendor specific in longer term ? Example AWS.

ii) Are you planning to have multi-cloud / hybrid cloud based infrastructure ?

iii) How well does it integrate with configuration management tools ?

iv) Price and Support

Use-Case 1 - Requirement of Organization 1

1. Organization is going to be based on AWS for next 25 years.

2. Official support is required in-case if team face any issue related to IAC tool or code itself.

3. They want some kind of GUI interface that supports automatic code generation.

Use-Case 2 - Requirement of Organization 2

1. Organization is based on Hybrid Solution. They use VMware for on-premise setup; AWS, Azure and GCP for Cloud.

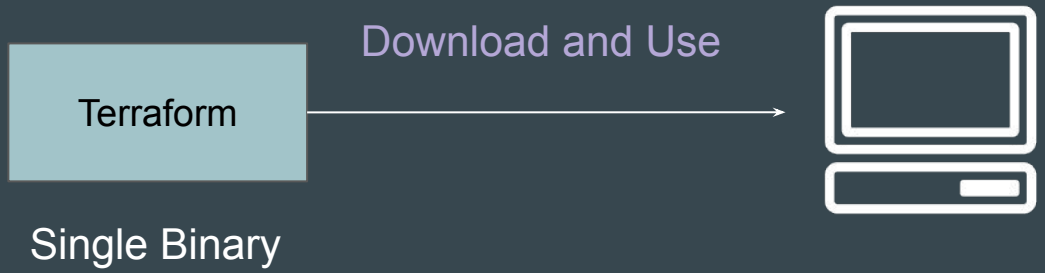
2. Official support is required in-case if IAC tool has any issues.

Installing Terraform

Overview of Installation Process

Terraform installation process is very simple.

You have a single binary file, download and use it.



Supported Operating Systems

Terraform supports major operating systems including:

- Windows
- macOS
- Linux
- FreeBSD
- OpenBSD
- Solaris

Point to Note

If you are manually downloading the Terraform binary rather than installing it via a package manager, you will need to ensure that the directory containing the Terraform executable is included in your system's PATH environment variable.

Choosing IDE For Terraform

Terraform in detail

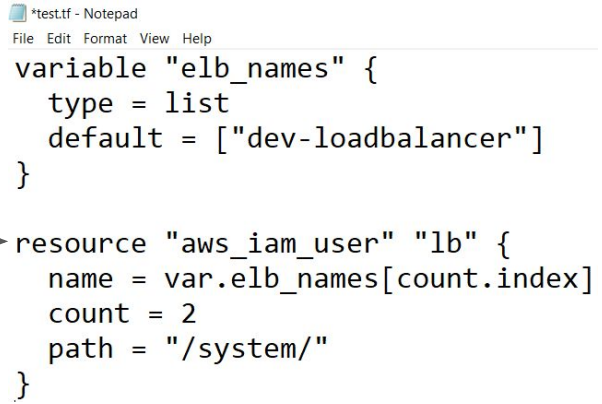
Terraform Code in NotePad!

You can write Terraform code in Notepad and it will not have any impact.

Downsides:

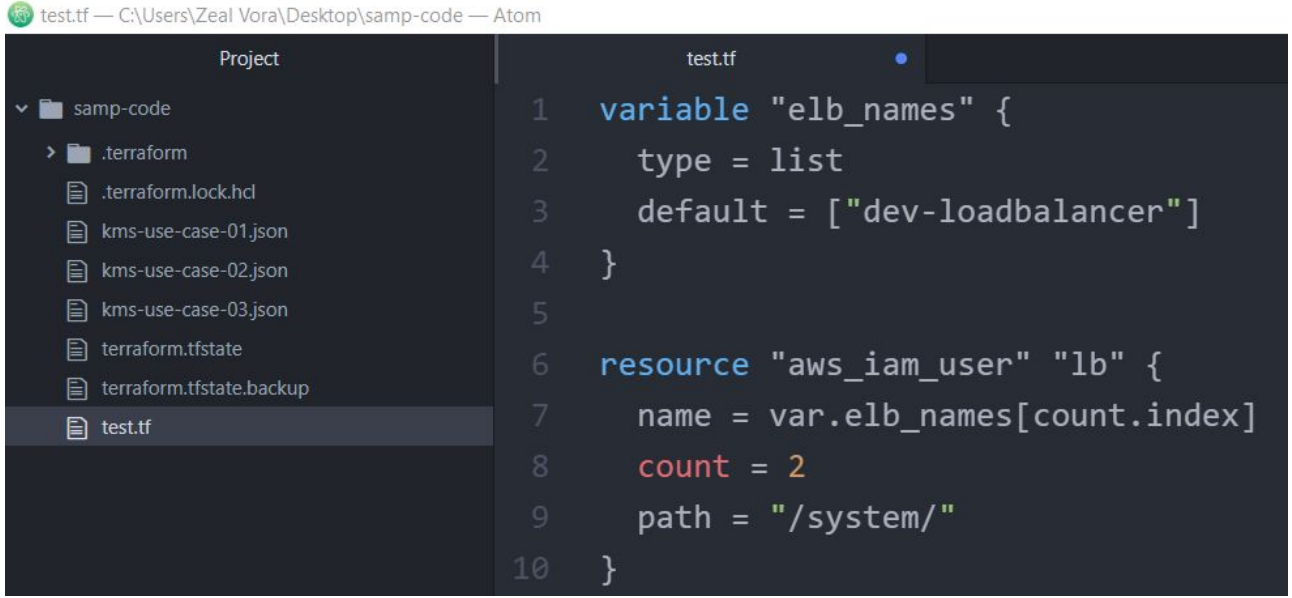
- Slower Development
- Limited Features





Need of a Better Software

There is a need of a better application that allows us to develop code faster.

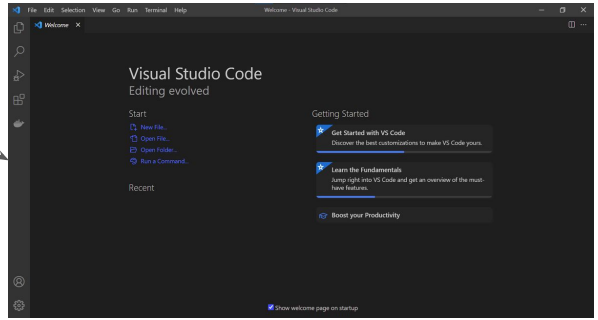


What are the Options!

There are many popular source code editors available in the market.

Source Code Editors



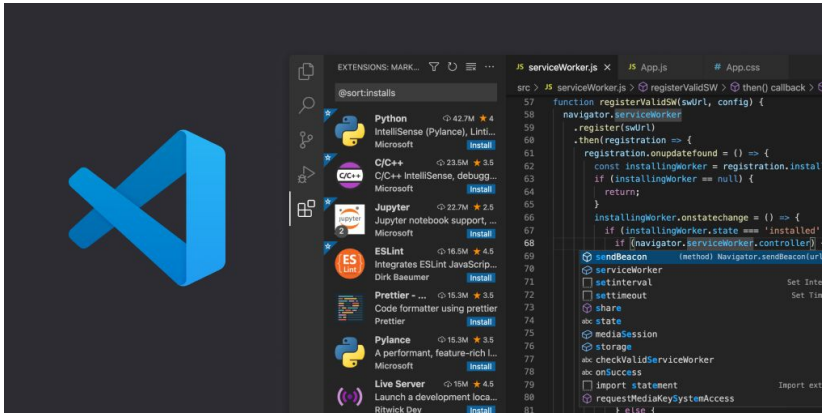


Editor for This Course

We are going to make use of Visual Studio Code as primary editor in this course.

Advantages:

1. Supports Windows, Mac, Linux
2. Supports Wide variety of programming languages.
3. Many Extensions.



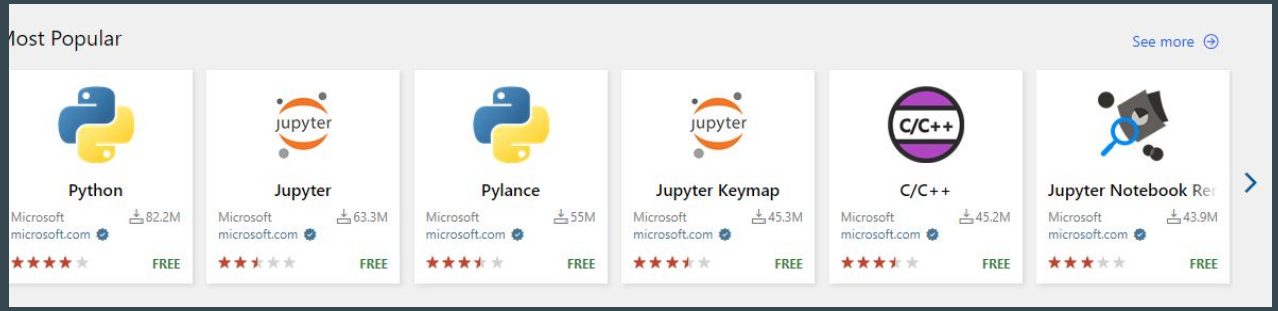

Using
Notepad
Using Visual
Studio Code
imgflip.com

Visual Studio Code Extensions

Understanding the Basics

Extensions are add-ons that allow you to customize and enhance your experience in Visual Studio by adding new features or integrating existing tools

They offer wide range of functionality related to colors, auto-complete, report spelling errors etc.

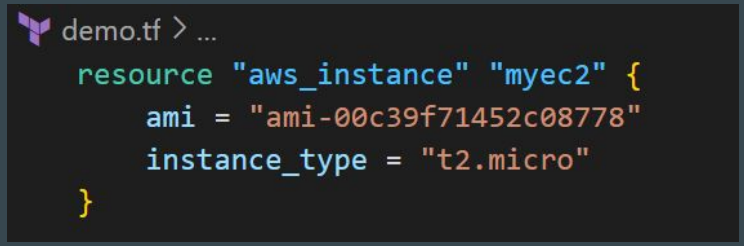


Terraform Extension

HashiCorp also provides extension for Terraform for Visual Studio Code.

demo.tf

```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
}
```



AWS Account Sign-Up Process

Signing Up for AWS Account

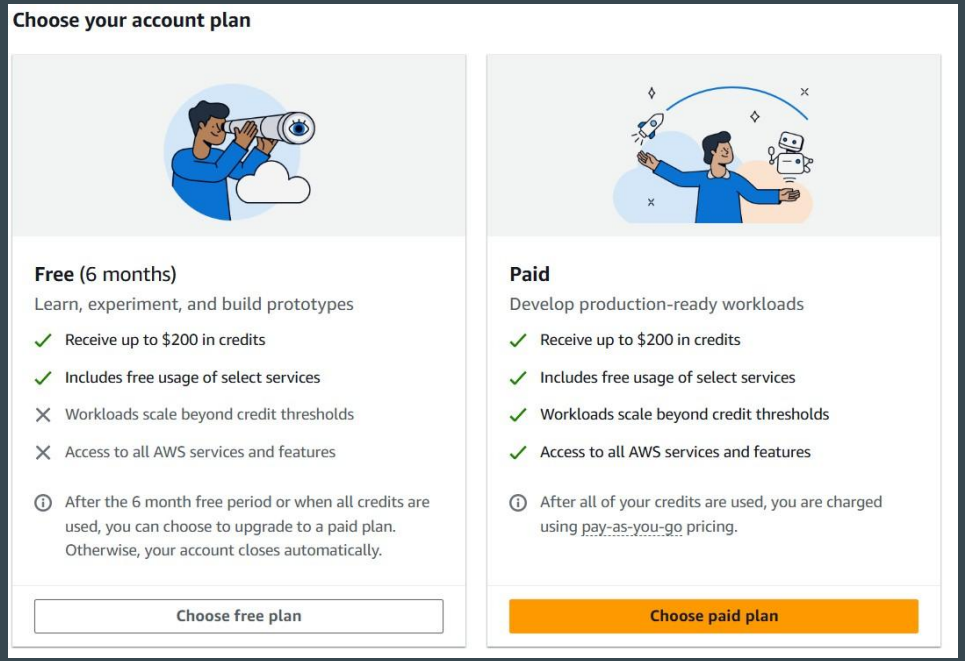
Creating a new AWS account is a simple and straightforward process.

High Level Steps
Username + Password
Choosing between Free Plan / Paid Plan
Phone Number Verification + Address
Payment Information



Setting the Base

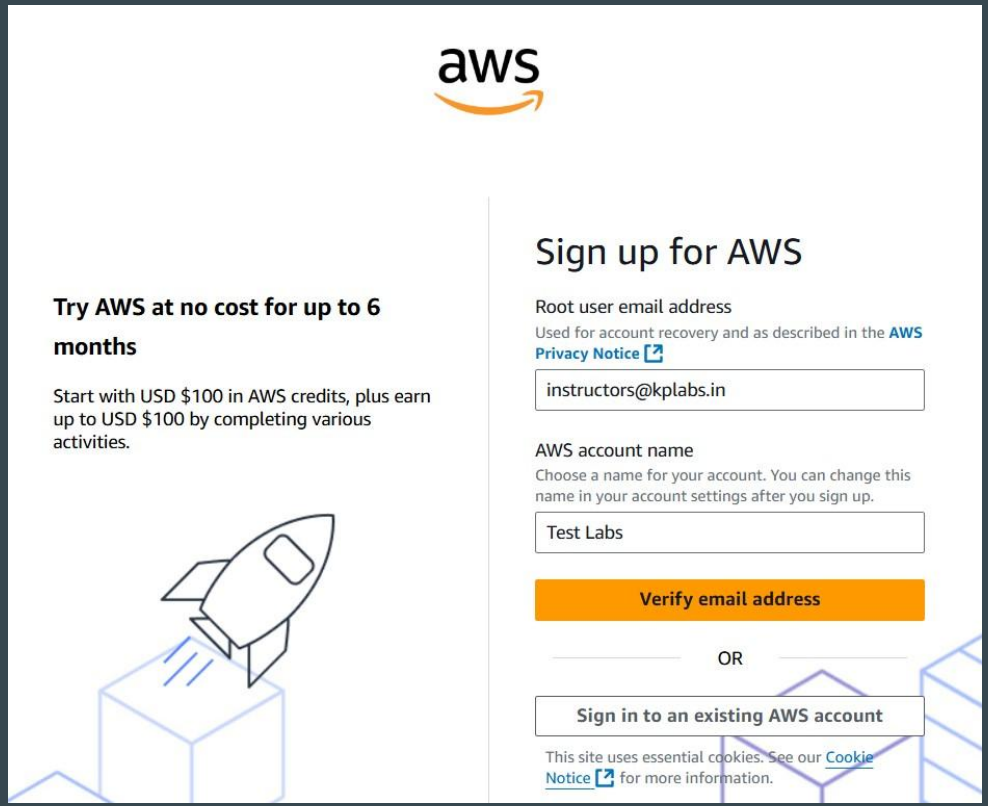
When you sign up for your AWS account, you can choose between Free plan or Paid account plan.



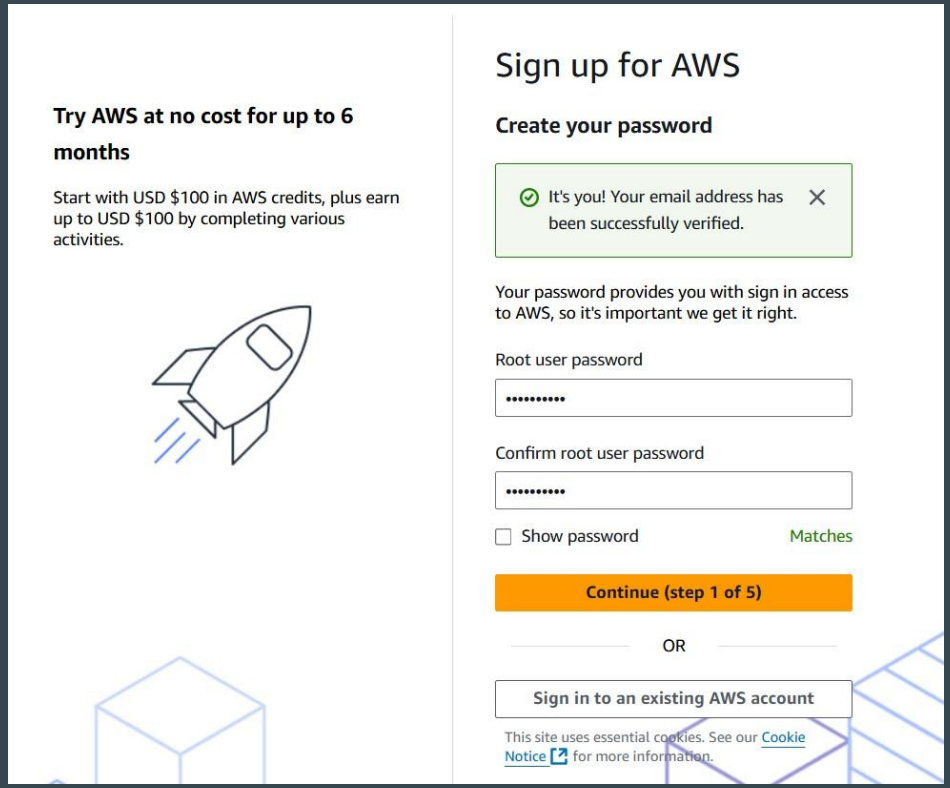
Free Plan vs Paid Plan

Free Plan	Paid Plan
Receive USD $100 sign up credit and earn up to $100 in additional credits	Receive USD $100 sign up credit and earn up to $100 in additional credits
Access to Always free services	Access to Always free services and short-term trial offers
Access to select AWS services and features	Access to all AWS services and features
No charges incur during usage	Pay for charges that exceed the credit balance
Account closes when credits are depleted or when the plan duration ends	Account doesn't close when credits are depleted
Not eligible for other promotional credits and discounts	Eligible for other promotional credits and discounts

Reference Screenshot - Email Address



Reference Screenshot - Password



Reference Screenshot - Plans

Choose your account plan



Free (6 months)

Learn, experiment, and build prototypes

- ✓ Receive up to $200 in credits
- ✓ Includes free usage of select services
- ✗ Workloads scale beyond credit thresholds
- ✗ Access to all AWS services and features

ⓘ After the 6 month free period or when all credits are used, you can choose to upgrade to a paid plan. Otherwise, your account closes automatically.

Choose free plan



Paid

Develop production-ready workloads

- ✓ Receive up to $200 in credits
- ✓ Includes free usage of select services
- ✓ Workloads scale beyond credit thresholds
- ✓ Access to all AWS services and features

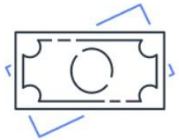
ⓘ After all of your credits are used, you are charged using pay-as-you-go pricing.

Choose paid plan

Reference Screenshot - Contact Information

Earn additional AWS credits

Complete various activities to earn up to an additional USD $100 in credits, such as creating your first AWS budget to monitor cloud costs.



Sign up for AWS

Contact Information

How do you plan to use AWS?

- ○ Business - for your work, school, or organization
- ○ Personal - for your own projects

Who should we contact about this account?

Full Name

Country Code | Phone Number

+1 | 222-333-4444

Country or Region

United States

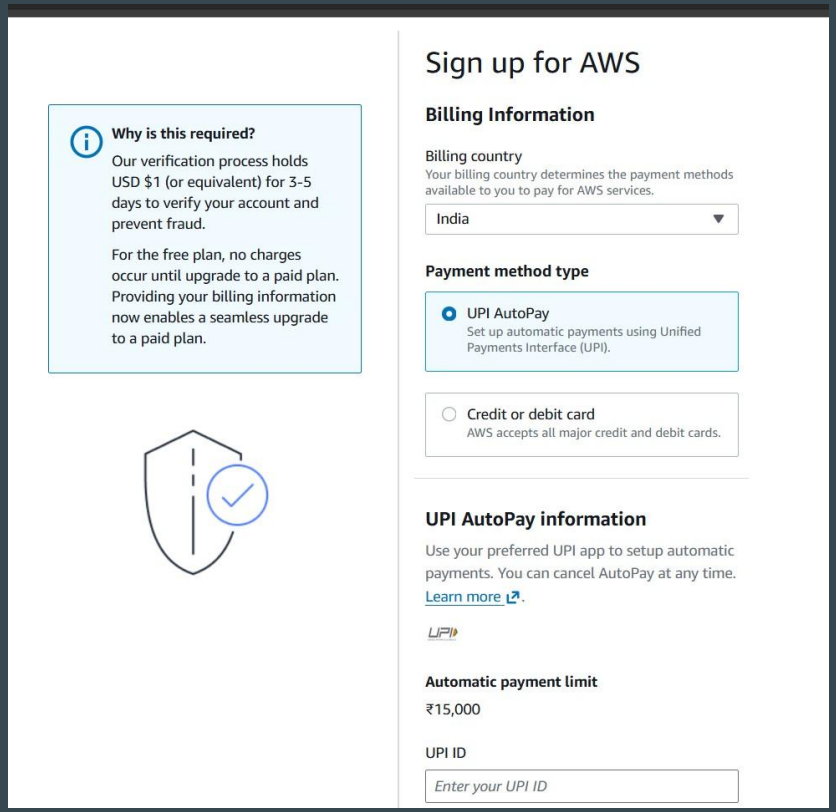
Address line 1

Address line 2

Apartment, suite, unit, building, floor, etc.

City

Reference Screenshot - Payment

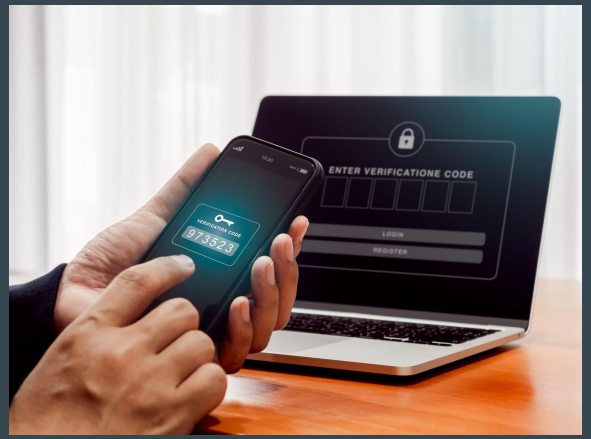


MFA for AWS Account

Multi-Factor Authentication is Important

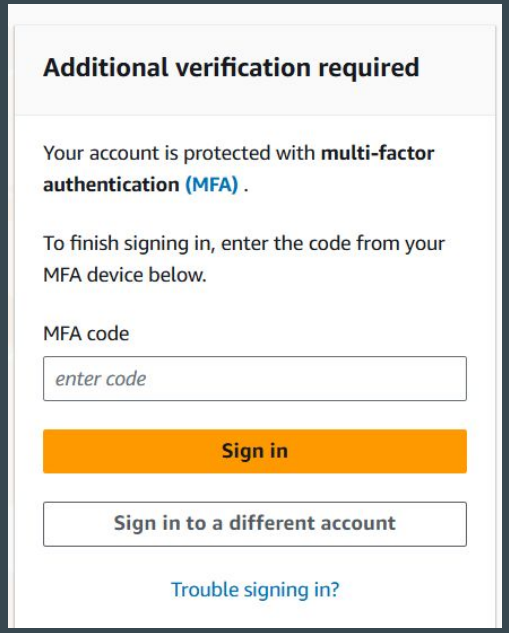
Your AWS account is connected to your payment methods (such as debit or credit cards).

If your login credentials are compromised, an attacker could deploy hundreds of servers for activities like Bitcoin mining, resulting in significant charges to your account.



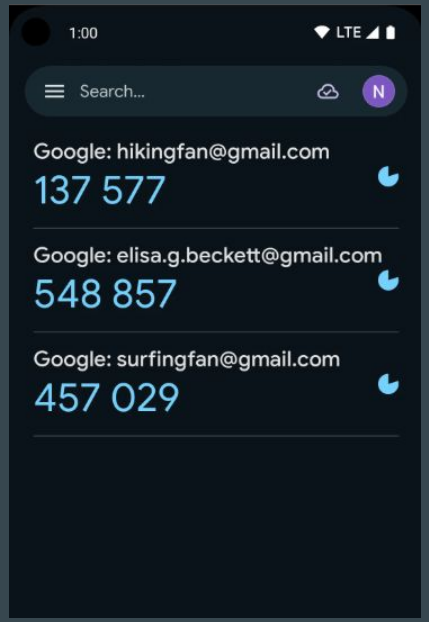
Setting the Base

With MFA enabled, when a user signs in to the AWS Management Console, they are prompted for their username and password— something they know—and an authentication code from their MFA device— something they have



Multiple Ways to Set MFA - Authenticator Apps

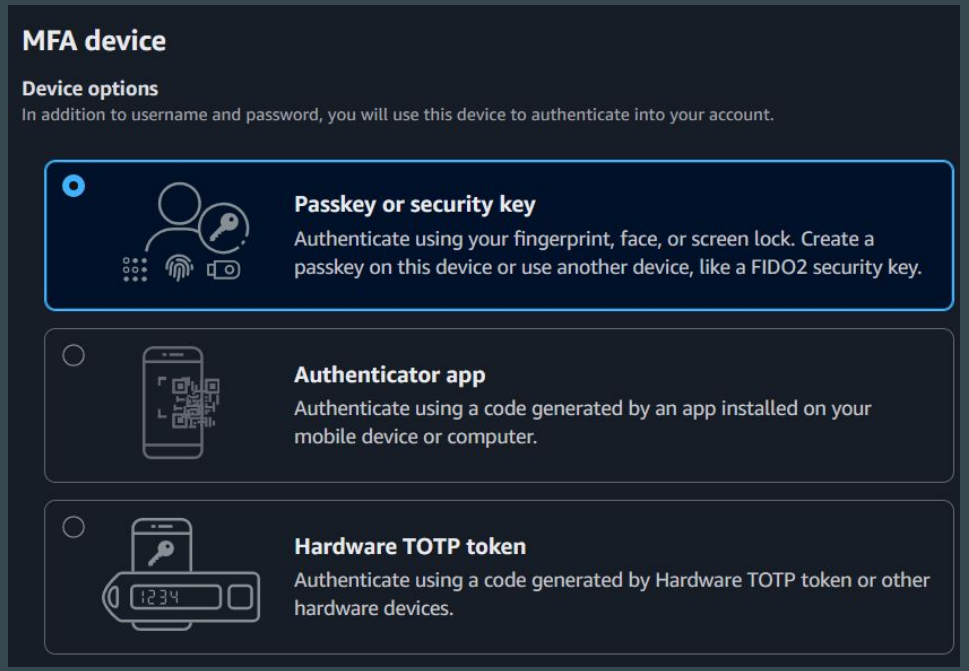
There are several options available for setting up multi-factor authentication (MFA), such as Google Authenticator and Authy.



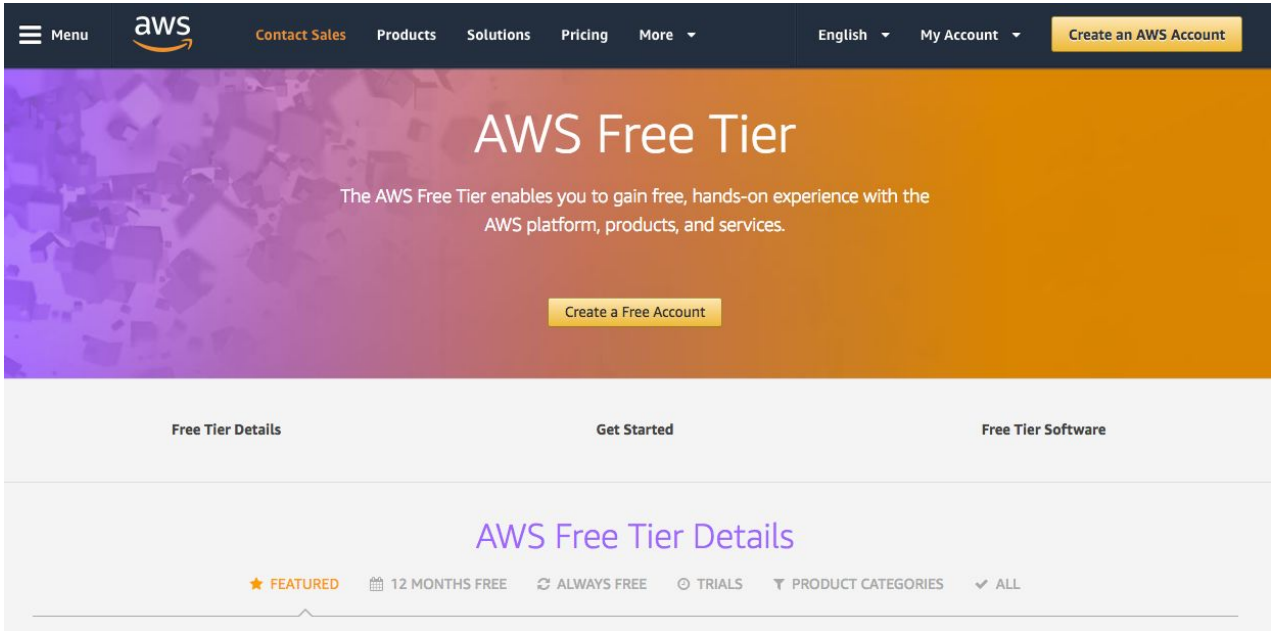
AUTHY

Device Options for MFA

You can use Passkey, Authenticator app, and Hardware TOTP token for setting up MFA



Registering an AWS Account

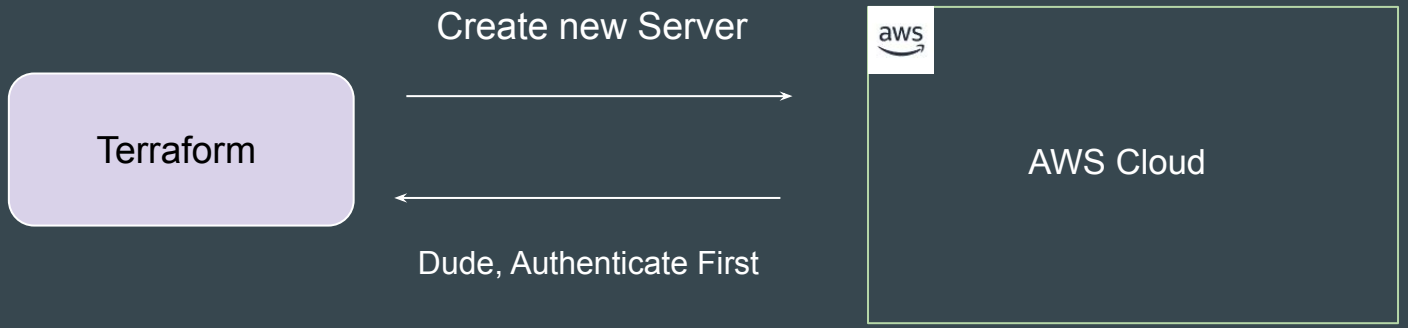


Authentication and Authorization



Understanding the Basics

Before we start working on managing environments through Terraform, the first important step is related to Authentication and Authorization.



Basics of Authentication and Authorization

Authentication is the process of verifying who a user is.

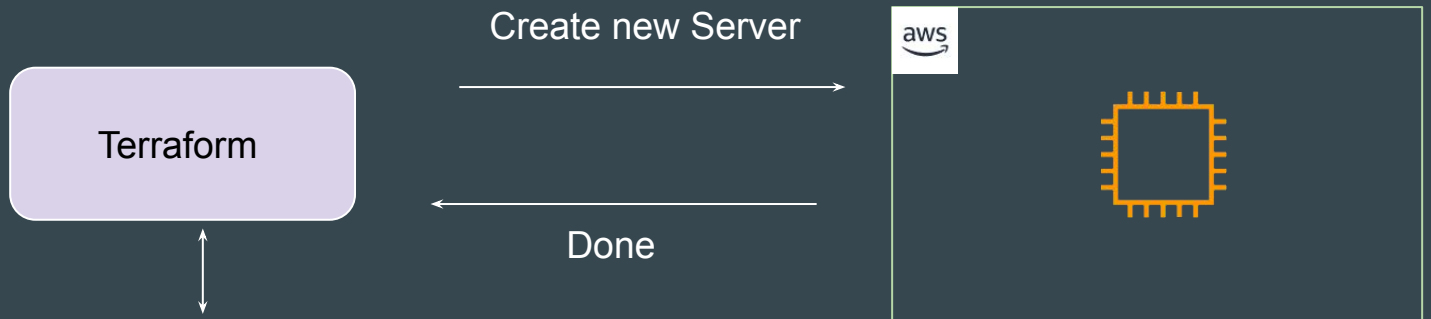
Authorization is the process of verifying what they have access to

Example:

Alice is a user in AWS with no access to any service.

Learning for Todays' Video

Terraform needs access credentials with relevant permissions to create and manage the environments.



username	password
Bob	pwd928#

Access Credentials

Depending on the provider, the type of access credentials would change.

Provider	Access Credentials
AWS	Access Keys and Secret Keys
GitHub	Tokens
Kubernetes	Kubeconfig file, Credentials Config
Digital Ocean	Tokens

First Virtual Machine Through Terraform



Revising the Basics of EC2

EC2 stands for Elastic Compute Cloud.

In-short, it's a name for a virtual server that you launch in AWS.



Available Regions

Cloud providers offers multiple regions in which we can create our resource.

You need to decide the region in which Terraform would create the resource.



Virtual Machine Configuration

A Virtual Machine would have it’s own set of configurations.

- CPU
- Memory
- Storage
- Operating System

While creating VM through Terraform, you will need to define these.

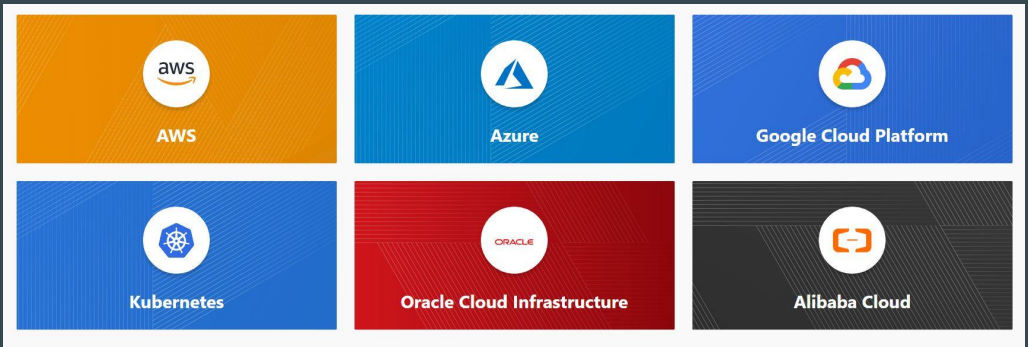
Providers and Resources



Basics of Providers

Terraform supports multiple providers.

Depending on what type of infrastructure we want to launch, we have to use appropriate providers accordingly.



Learning 1 - Provider Plugins

A provider is a plugin that lets Terraform manage an external API.

When we run terraform init, plugins required for the provider are automatically downloaded and saved locally to a .terraform directory.

```
C:\Users\zealv\Desktop\kplabs-terraform>terraform init

Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v4.60.0...
- Installed hashicorp/aws v4.60.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Learning 2 - Resource

Resource block describes one or more infrastructure objects

Example:

- resource aws_instance
- resource aws_alb
- resource iam_user
- resource digitalocean_droplet

```
resource "aws_instance" "myec2" {
   ami = "ami-082b5a644766e0e6f"
   instance_type = "t2.micro"
}
```

Learning 3 - Resource Blocks

A resource block declares a resource of a given type ("aws_instance") with a given local name ("myec2").

Resource type and Name together serve as an identifier for a given resource and so must be unique.

```
resource "aws_instance" "myec2" {
   ami = "ami-082b5a644766e0e6f"
   instance_type = "t2.micro"
}
```

EC2 Instance Number 1

```
resource "aws_instance" "web" {
  ami           = ami-123
  instance_type = "t2.micro"
}
```

EC2 Instance Number 2

Point to Note

You can only use the resource that are supported by a specific provider.

In the below example, provider of Azure is used with resource of aws_instance

```
provider "azurerm" {}

resource "aws_instance" "web" {
  ami           = ami-123
  instance_type = "t2.micro"

}
```

Important Question

The core concepts, standard syntax remains similar across all providers.

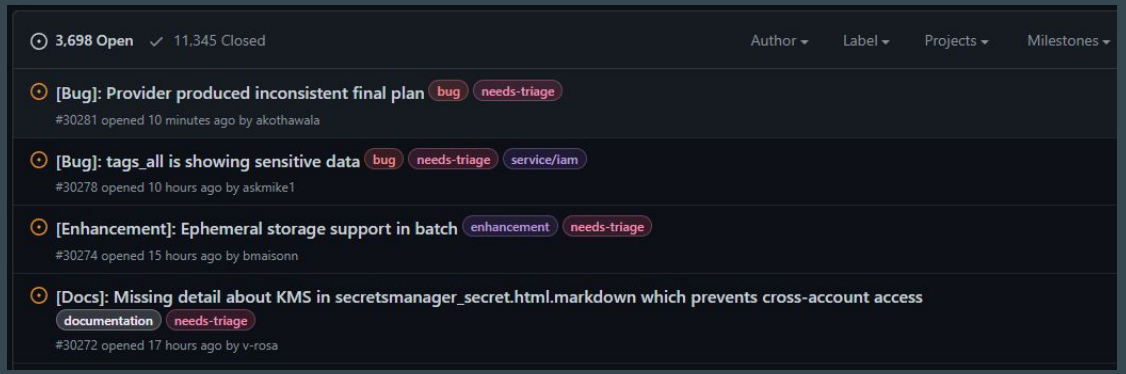
If you learn the basics, you should be able to work with all providers easily.



Issues and Bugs with Providers

A provider that is maintained by HashiCorp does not mean it has no bugs.

It can happen that there are inconsistencies from your output and things mentioned in documentation. You can raise issue at Provider page.



Relax and Have a Meme Before Proceeding



Provider Tiers



Provider Maintainers

There are 3 primary type of provider tiers in Terraform.

Provider Tiers	Description
Official	Owned and Maintained by HashiCorp.
Partner	Owned and Maintained by Technology Company that maintains direct partnership with HashiCorp.
Community	Owned and Maintained by Individual Contributors.

Provider Namespace

Namespaces are used to help users identify the organization or publisher responsible for the integration

Tier	Description
Official	hashicorp
Partner	Third-party organization e.g. mongodb/mongodbatlas
Community	Maintainer's individual or organization account, e.g. DeviaVir/gsuite

Important Learning

Terraform requires explicit source information for any providers that are not HashiCorp-maintained, using a new syntax in the required_providers nested block inside the terraform configuration block

```
provider "aws" {
  region     = "us-west-2"
  access_key = "PUT-YOUR-ACCESS-KEY-HERE"
  secret_key = "PUT-YOUR-SECRET-KEY-HERE"
}
```

HashiCorp Maintained

```
terraform {
  required_providers {
    digitalocean = {
      source = "digitalocean/digitalocean"
    }
  }
}

provider "digitalocean" {
  token = "PUT-YOUR-TOKEN-HERE"
}
```

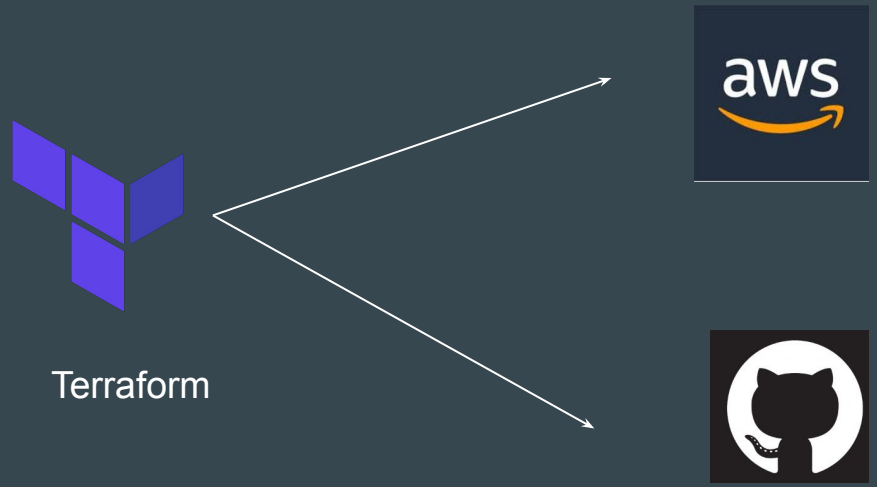
Non-HashiCorp Maintained

Terraform Destroy

Learning to Destroy Resources

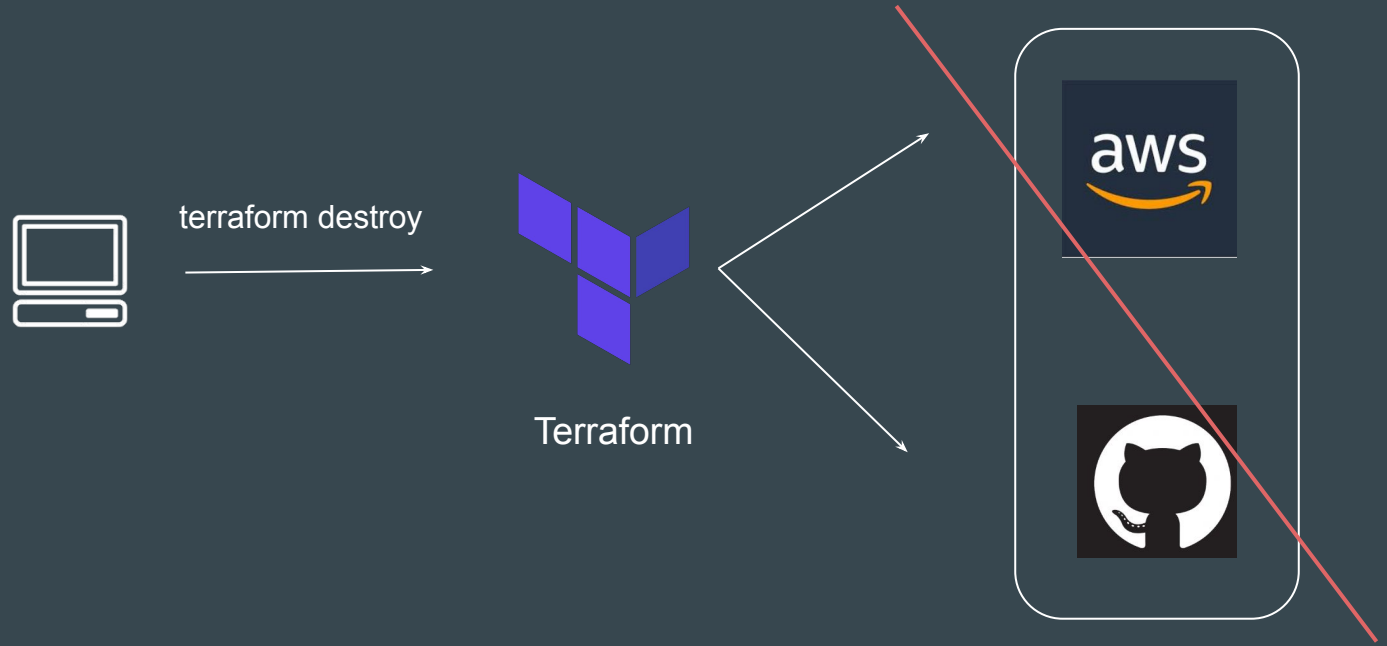
If you keep the infrastructure running, you will get charged for it.

Hence it is important for us to also know on how we can delete the infrastructure resources created via terraform.



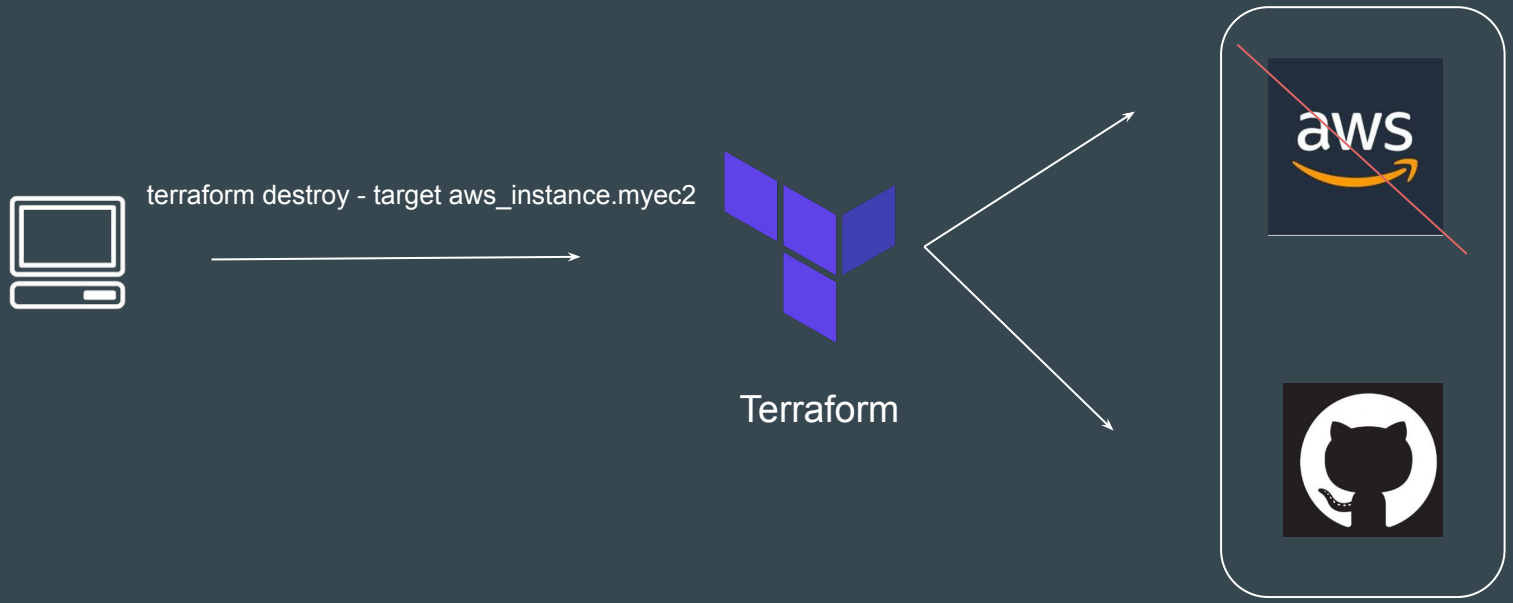
Approach 1 - Destroy ALL

terraform destroy allows us to destroy all the resource that are created within the folder.



Approach 2 - Destroy Some

terraform destroy with -target flag allows us to destroy specific resource.



Terraform Destroy with Target

The -target option can be used to focus Terraform's attention on only a subset of resources.

Combination of : Resource Type + Local Resource Name

Resource Type	Local Resource Name
aws_instance	myec2
github_repository	example

```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
}
```

```
resource "github_repository" "example" {
  name        = "example"
  description = "My awesome codebase"

  visibility = "public"

}
```

AWS Provider - Authentication Configuration

Understanding the Basics

At this stage, we have been manually hardcoding the access / secret keys within the provider block.

Although a working solution, but it is not optimal from security point of view.

aws-provider-config.tf > ...

```
provider "aws" {
  region     = "us-east-1"
  access_key = "AKIAQTSHLD4OASDZ5QJI"
  secret_key = "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"
}

resource "aws_eip" "lb" {
  domain   = "vpc"
}
```

Better Way

We want our code to run successfully without hardcoding the secrets in the provider block.

aws-provider-config.tf > ...

```
provider "aws" {
  region     = "us-east-1"
}

resource "aws_eip" "lb" {
  domain   = "vpc"
}
```

Better Approach

The AWS Provider can source credentials and other settings from the shared configuration and credentials files.

aws-provider-config.tf > ...

```
provider "aws" {
  shared_config_files      = ["/Users/tf_user/.aws/conf"]
  shared_credentials_files = ["/Users/tf_user/.aws/creds"]
  profile                  = "customprofile"
}

resource "aws_eip" "lb" {
  domain   = "vpc"
}
```

Default Configurations

If shared files lines are not added to provider block, by default, Terraform will locate these files at $HOME/.aws/config and $HOME/.aws/credentials on Linux and macOS.

"%USERPROFILE%\.aws\config" and "%USERPROFILE%\.aws\credentials" on Windows.

aws-provider-config.tf > ...

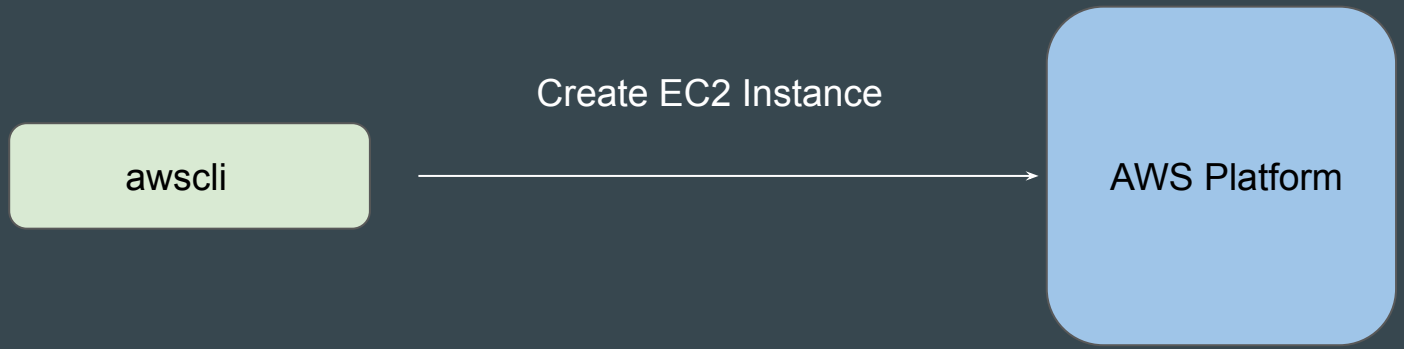
```
provider "aws" {
  region     = "us-east-1"
}

resource "aws_eip" "lb" {
  domain   = "vpc"
}
```

AWS CLI

AWS CLI allows customers to manage AWS resources directly from CLI.

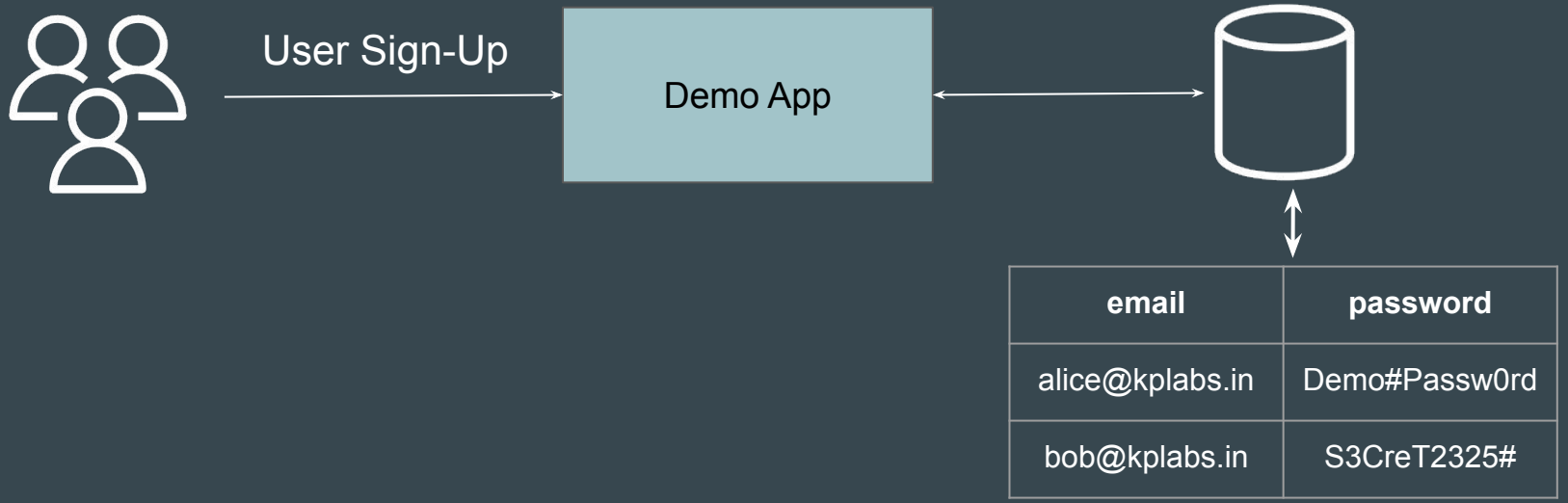
When you configure Access/Secret keys in AWS CLI, the location in which these credentials are stored is the same default location that Terraform searches the credentials from.



Terraform State File

Simple Analogy

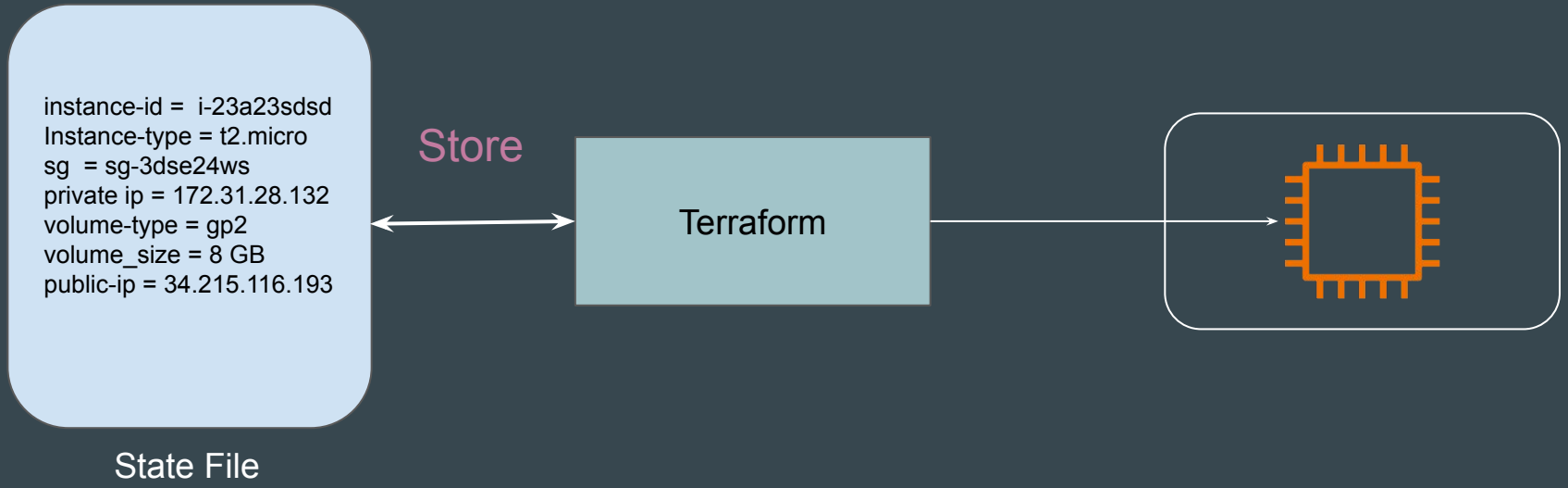
The demo application provides a frontend interface for users and stores all the data in a backend database.



Terraform State File

Terraform stores information about managed infrastructure in a state file.

This state file keeps track of resources created by your configuration.



Point to Note

1. By default, the state information is stored in file named terraform.tfstate

2. Format of state file is JSON

3. Never modify the Terraform State file manually. Any mistake can cause corruption of state file.

Desired State and Current State

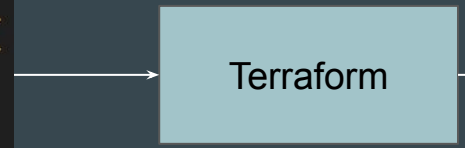
Desired State (The "What You Want")

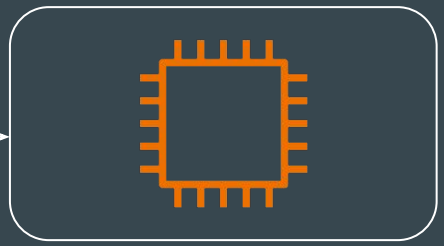
The Desired State is the configuration you write in your Terraform files (usually ending in .tf).

Your desired state is EC2 instance of t2.micro and OS associated with “ami-123”

```
resource "aws_instance" "myec2" {
    ami = "ami-123"
    instance_type = "t2.micro"
}
```

Desired State



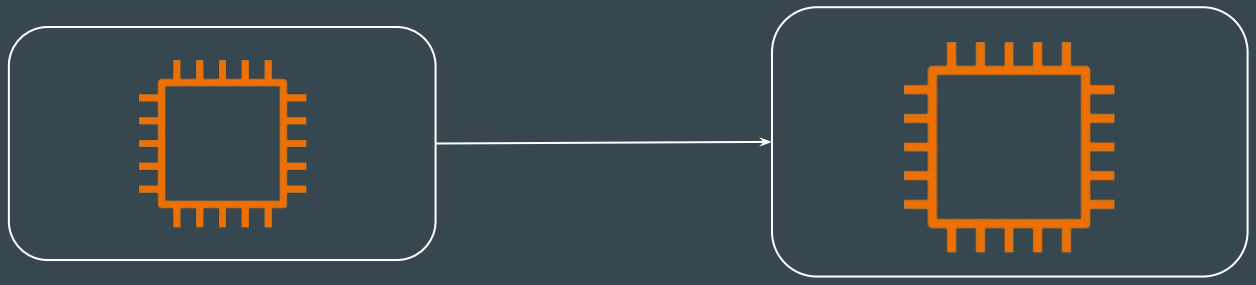


t2.micro

Current State (The "What Actually Exists")

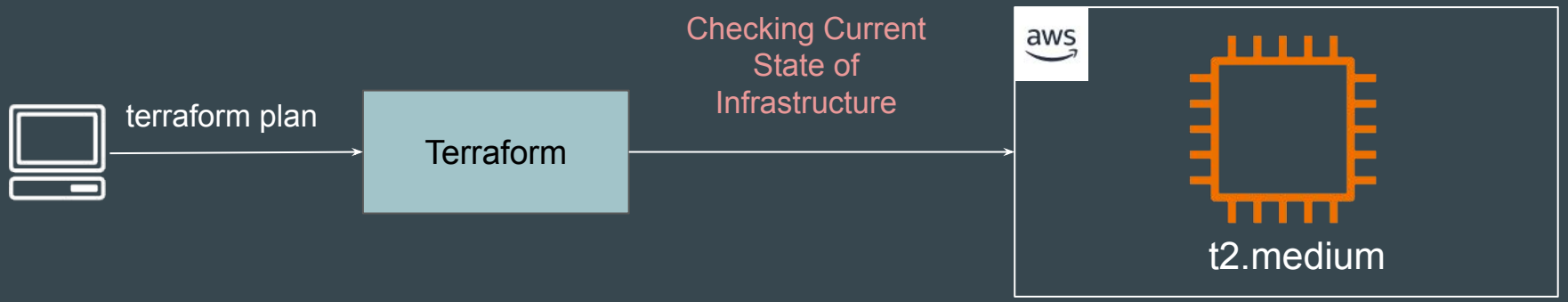
The Current State represents the actual reality of your infrastructure.

Example: Bob manually modified the instance size from t2.micro to t2.medium



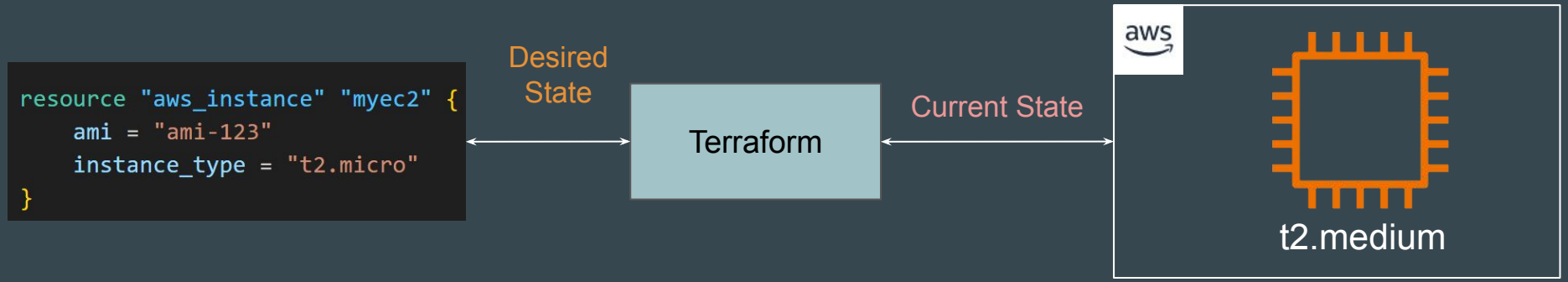
The Reconciliation Process (Part 1)

When you run terraform plan or terraform apply, Terraform first reaches out to the Cloud Provider API to check the Current State of Infrastructure.



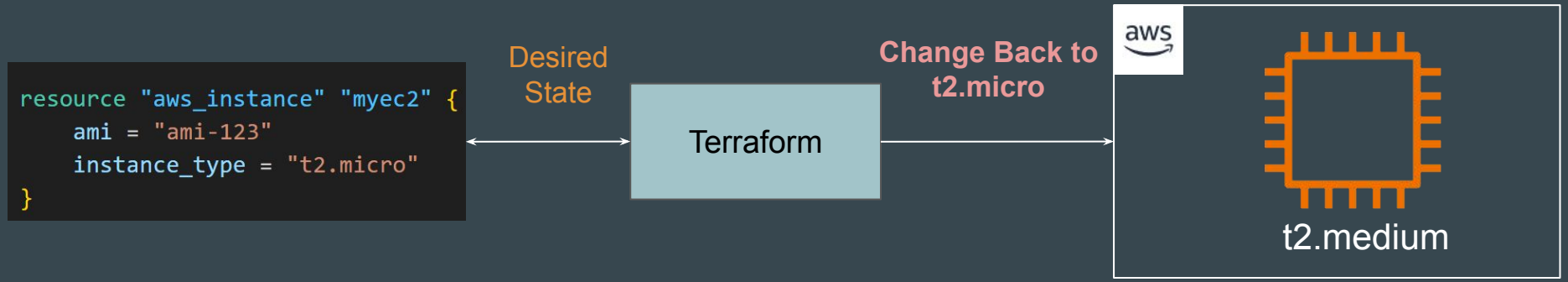
The Reconciliation Process (Part 2)

Terraform compares the Current State (what it just found in the cloud) against your Desired State (your .tf files).



The Reconciliation Process (Part 3)

Based on the difference, Terraform determines what actions are necessary to make the Current State match the Desired State.



Generic Actions

Based on the difference, Terraform determines what actions are necessary to make the Current State match the Desired State.

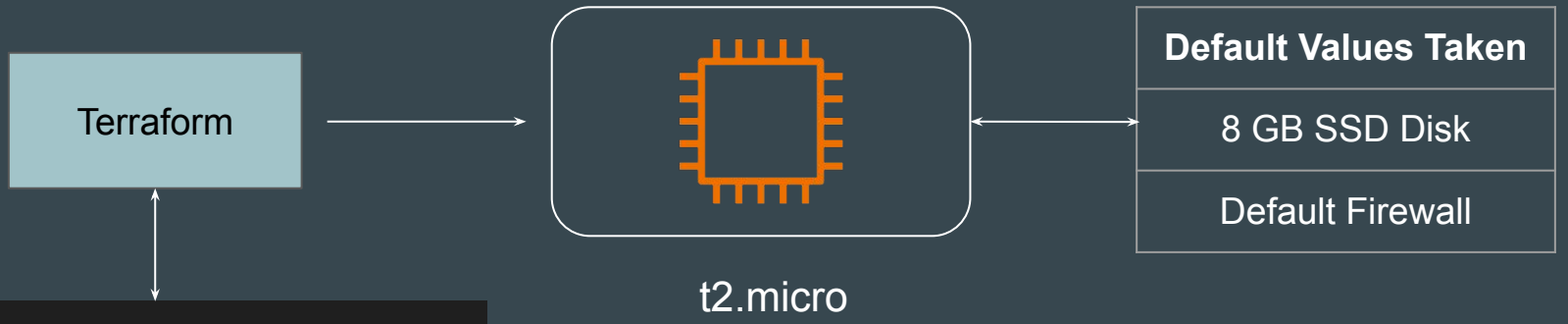
Create	If Desired exists but Current does not.
Update	If both exist, but a property differs.
Delete	If Current exists (and is tracked in the state file) but Desired (the code) has been removed.
No-Op	If Current matches Desired exactly.


CURRENT
STATE
GET BACK TO
DESIRED STATE

More Clarity - Desired State and Current State

Setting the Base

When you launch an EC2 instance without specifying certain configuration parameters, AWS automatically assigns default values to those unspecified settings.

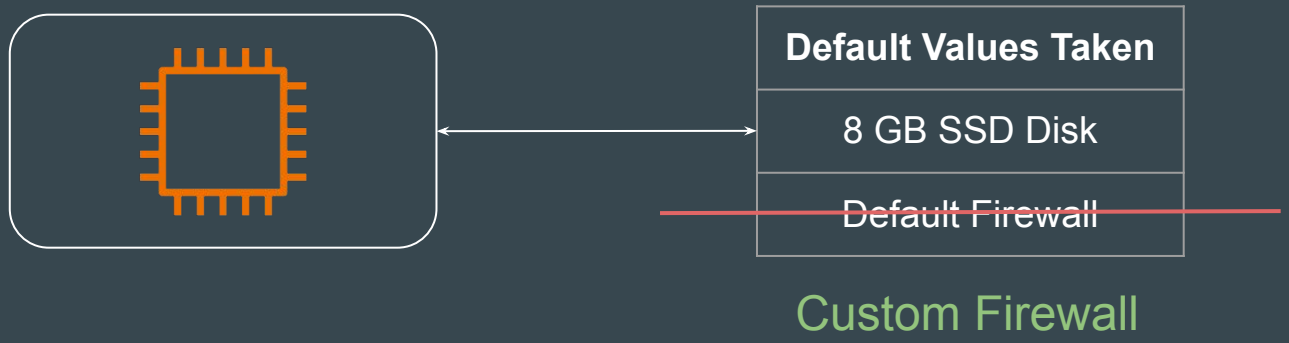



```
resource "aws_instance" "myec2" {
    ami = "ami-123"
    instance_type = "t2.micro"
}
```

Point to Note

These default values are generally not considered to be your desired state.

If you manually change these default values, it will have no impact in next terraform plan and apply stages.



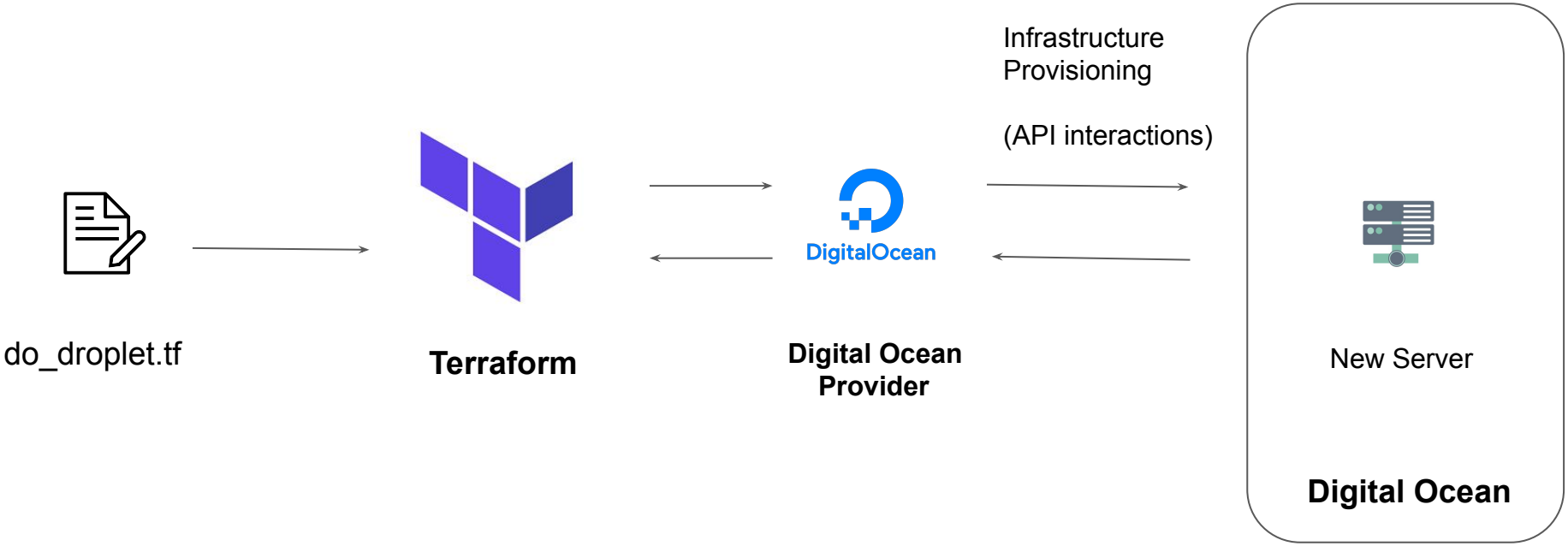
Best Practice

Explicitly Define All Desired Attributes in Your Configuration.

Provider Versioning

Terraform in detail

Provider Architecture

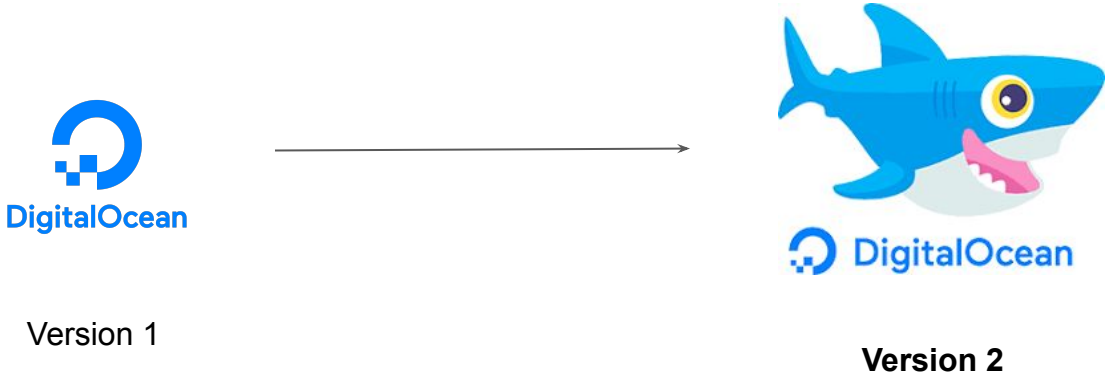


Overview of Provider Versioning

Provider plugins are released separately from Terraform itself.

They have different set of version numbers.

.



Version 1

Version 2

Explicitly Setting Provider Version

During terraform init, if version argument is not specified, the most recent provider will be downloaded during initialization.

For production use, you should constrain the acceptable provider versions via configuration, to ensure that new versions with breaking changes will not be automatically installed.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

Arguments for Specifying provider

There are multiple ways for specifying the version of a provider.

Version Number Arguments	Description
>=1.0	Greater than equal to the version
<=1.0	Less than equal to the version
~>2.0	Any version in the 2.X range.
>=2.10,<=2.30	Any version between 2.10 and 2.30

Dependency Lock File

Terraform dependency lock file allows us to lock to a specific version of the provider.

If a particular provider already has a selection recorded in the lock file, Terraform will always re-select that version for installation, even if a newer version has become available.

You can override that behavior by adding the -upgrade option when you run terraform init,

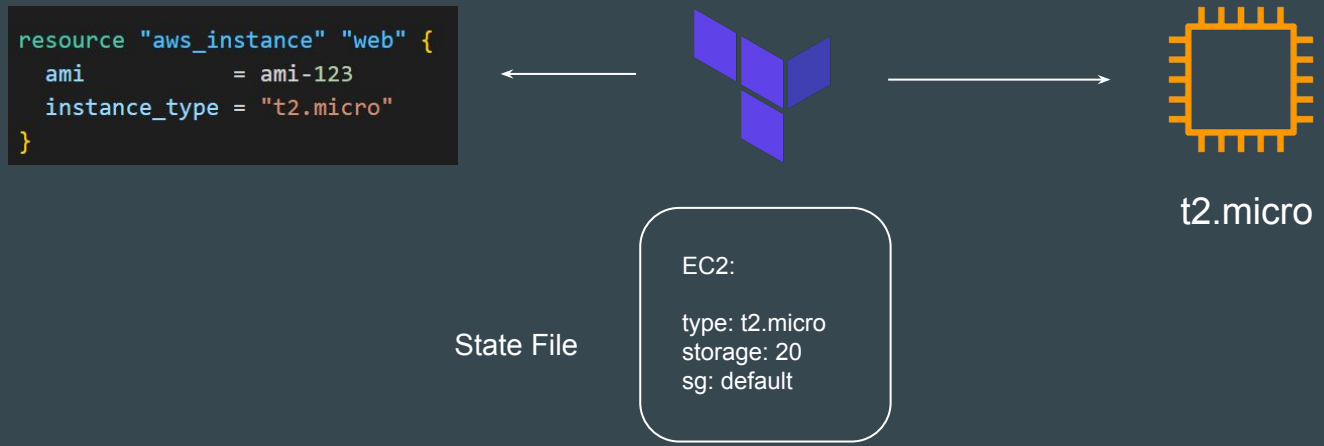
```
provider "registry.terraform.io/hashicorp/aws" {
  version     = "2.70.0"
  constraints = ">= 2.31.0, <= 2.70.0"
  hashes = [
    "h1:fx8tbGVwK1YIDI6UdHLnorC9PA1ZPSWEeW3V3aDCdWY=",
    "zh:01a5f351146434b418f9ff8d8cc956ddc801110f1cc8b139e01be2ff8c544605",
    "zh:1ec08abbaf09e3e0547511d48f77a1e2c89face2d55886b23f643011c76cb247",
    "zh:606d134fef7c1357c9d155aadbee6826bc22bc0115b6291d483bc1444291c3e1",
    "zh:67e31a71a5ecbbc96a1a6708c9cc300bbfe921c322320cdbb95b9002026387e1",
```

Terraform Refresh

Understanding the Challenge

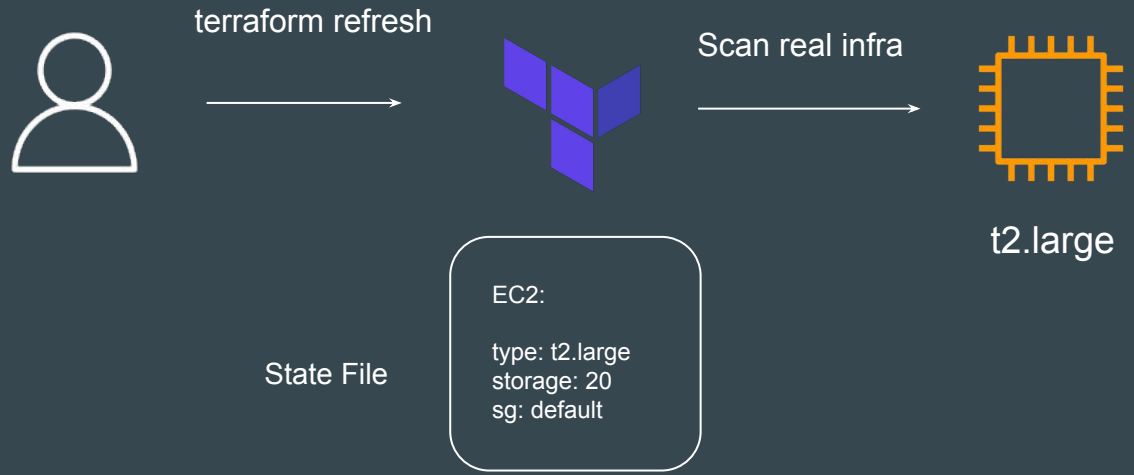
Terraform can create an infrastructure based on configuration you specified.

It can happen that the infrastructure gets modified manually.



Understanding the Challenge

The terraform refresh command will check the latest state of your infrastructure and update the state file accordingly.



Points to Note

You shouldn't typically need to use this command, because Terraform automatically performs the same refreshing actions as a part of creating a plan in both the terraform plan and terraform apply commands.

Understanding the Usage

The terraform refresh command is deprecated in newer version of terraform.

The -refresh-only option for terraform plan and terraform apply was introduced in Terraform v0.15.4.

Lecture Format - Terraform Course

Terraform in detail

Overview of the Format

We tend to use a different folder for each practical that we do in the course.

This allows us to be more systematic and allows easier revisit in-case required.

Lecture Name	Folder Names
Create First EC2 Instance	folder1
Tainting resource	folder2
Conditional Expression	folder3

Find the appropriate code from GitHub

Code in GitHub is arranged according to sections that are matched to the domains in the course.

Every section in GitHub has easy Readme file for quick navigation.

Video-Document Mapper

Sr No	Document Link
1	Understanding Attributes and Output Values in Terraform
2	Referencing Cross-Account Resource Attributes
3	Terraform Variables
4	Approaches for Variable Assignment
5	Data Types for Variables

Destroy Resource After Practical

We know how to destroy resources by now

terraform destroy

After you have completed your practical, make sure you destroy the resource before moving to the next practical.

This is easier if you are maintaining separate folder for each practical.

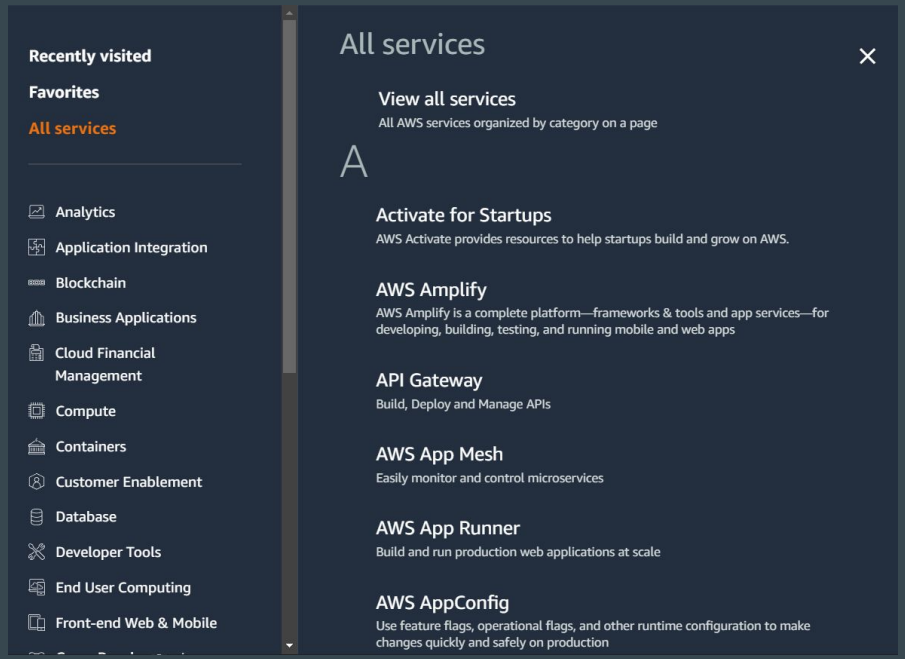
Relax and Have a Meme Before Proceeding



Learning Scope - AWS Services for Terraform Course

Understanding the Basics

AWS has more than 200 services available.

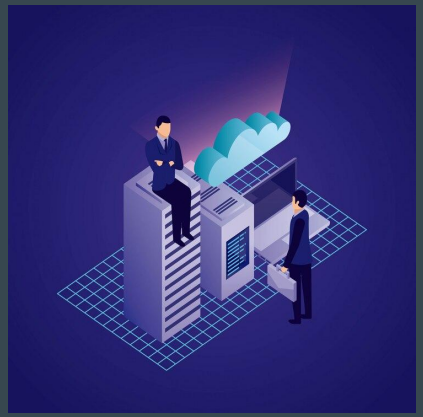


Aim of the Course

Primary aim of this course is to master the core concepts of Terraform.

Terraform = Infrastructure as Code Tool.

To learn Terraform, we need to create infrastructure somewhere.



Services that we Choose

Throughout the course, we use very basic AWS services to demonstrate Terraform concepts.

- Virtual Machine (EC2)
- Firewall (Security Groups)
- AWS Users (IAM Users)
- IP Address (Elastic IP)

Basics of These Services are Covered

We have 100,000+ students from different background who are learning Terraform.

Some are AWS Pros, Some are from Azure/GCP, Some are students

To align everyone on same page, we also cover basics of the AWS service that we use throughout the course.

Example - Creating Firewall Through Terraform

1. Basics of Firewalls in AWS

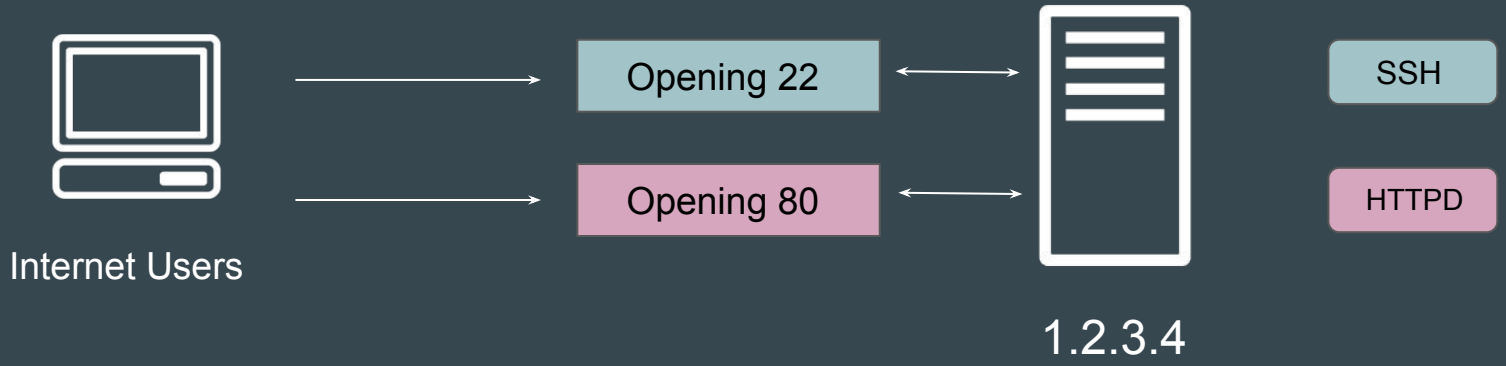
2. Firewall Practical in AWS (GUI Console)

3. Creating Firewall Rules Through Terraform.

Basics of Firewalls

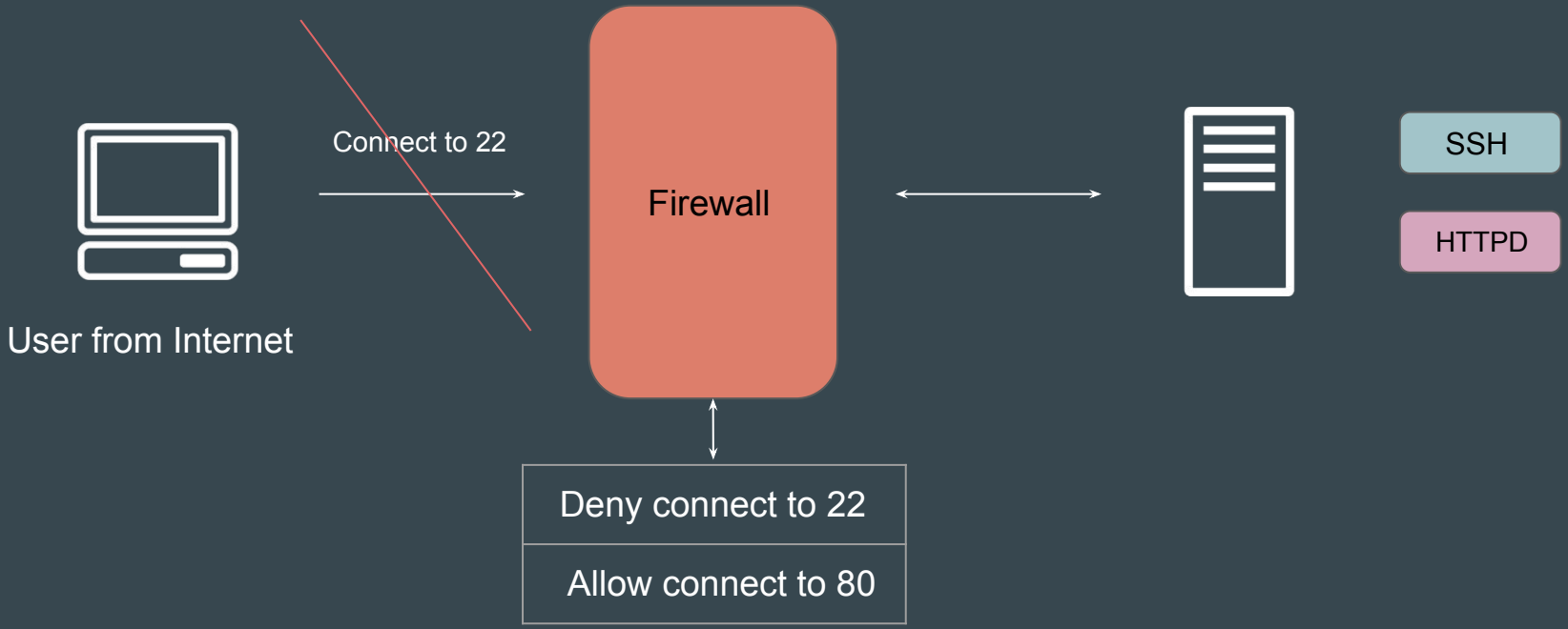
Basics of Ports

A port acts as a endpoint of communication to identify a given application or process on an Linux operating system



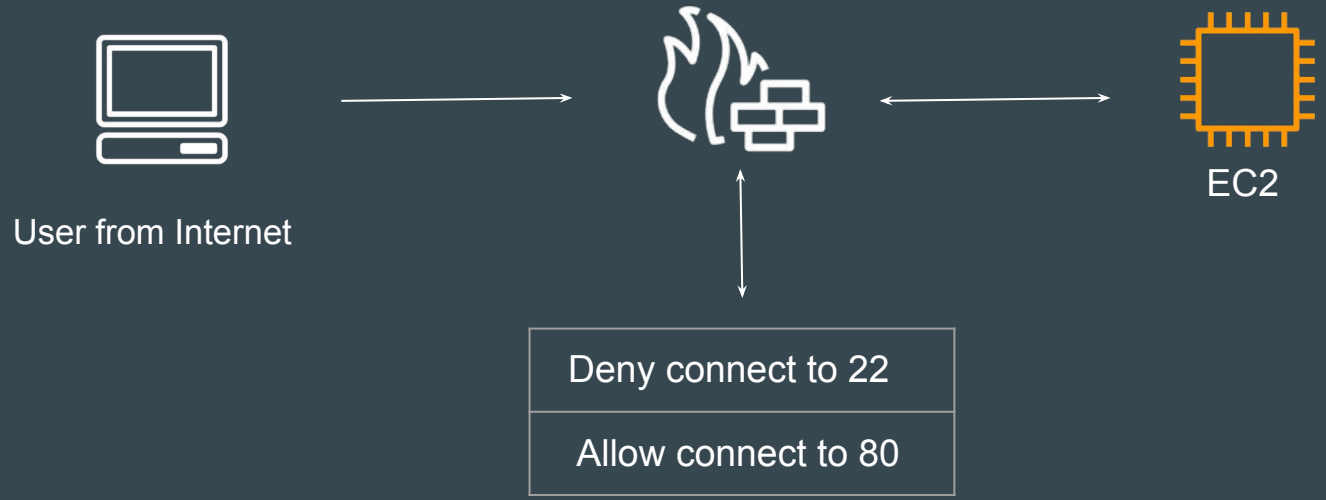
Basics of Firewall

Firewall is a network security system that monitors and controls incoming and outgoing network traffic based on predetermined security rules.

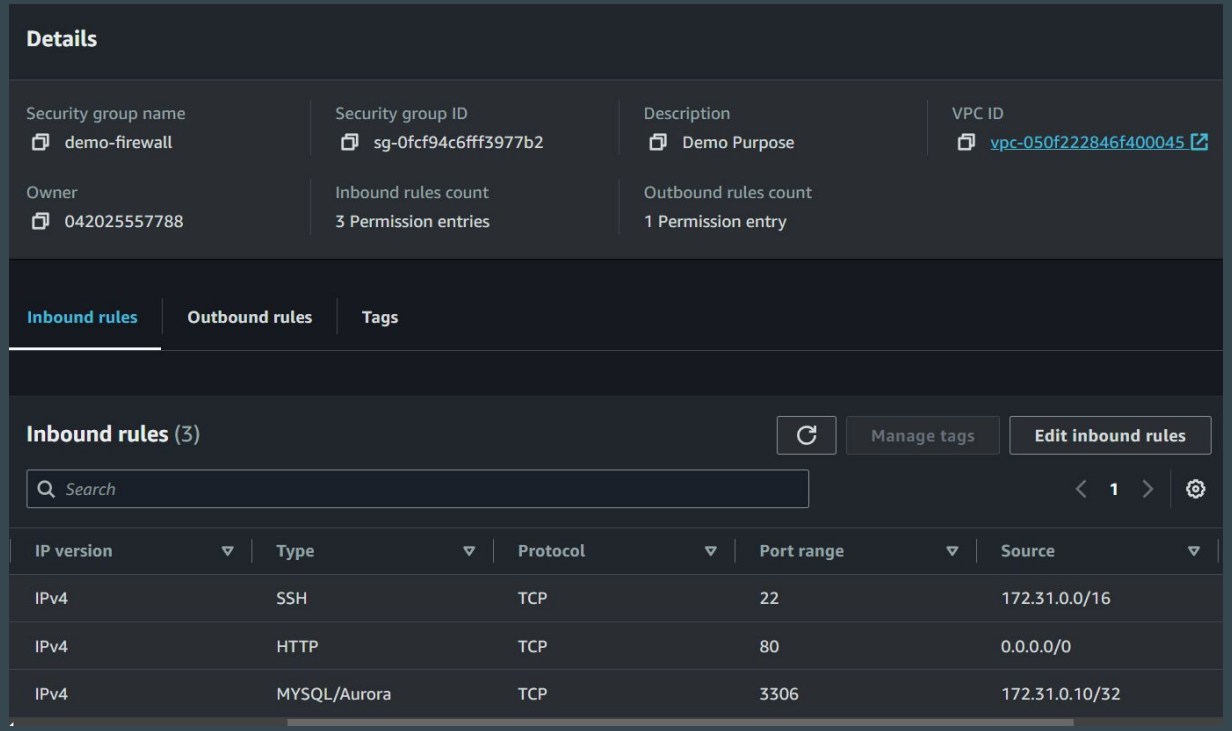


Firewall in AWS

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic.



Sample Security Group with Rules



Details

Security group name	Security group ID	Description	VPC ID
demo-firewall	sg-0fcf94c6fff3977b2	Demo Purpose	vpc-050f222846f400045

Owner	Inbound rules count	Outbound rules count
042025557788	3 Permission entries	1 Permission entry

Inbound rules | Outbound rules | Tags

Inbound rules (3)

Manage tags | Edit inbound rules

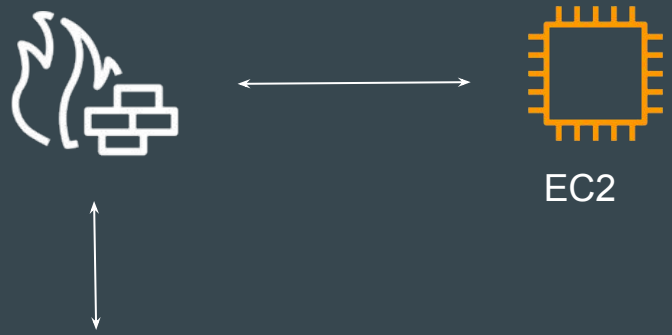
Search

1

IP version	Type	Protocol	Port range	Source
IPv4	SSH	TCP	22	172.31.0.0/16
IPv4	HTTP	TCP	80	0.0.0.0/0
IPv4	MYSQL/Aurora	TCP	3306	172.31.0.10/32

Inbound and Outbound Rules

Firewalls control both inbound and outbound connections to and from the server.



Inbound	Outbound
Allow 80 from 0.0.0.0/0	Allow 3306 to 172.31.10.50

Creating Firewall Rules with Terraform

Architecture of Today's Video

We will create a Firewall (Security Group) in AWS with following configuration



terraform-firewall

Inbound	Outbound
Allow 80 from 0.0.0.0/0	Allow ALL

Reference - Final Code in Video

firewall.tf

firewall.tf > resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4"

```
resource "aws_security_group" "allow_tls" {
  name        = "terraform-firewall"
  description = "Managed from Terraform"
}

resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "0.0.0.0/0"
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}

resource "aws_vpc_security_group_egress_rule" "allow_all_traffic_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1" # semantically equivalent to all ports
}
```

Dealing with Documentation Code Updates - Terraform

Understanding the Challenge

Occasionally in the newer version of Providers, you will see some changes in the way you create a resource.

```hcl
resource "aws_security_group" "allow_tls" {
  name        = "terraform-firewall"
  description = "Managed from Terraform"
}

resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "0.0.0.0/0"
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}
```

New Approach

```hcl
resource "aws_security_group" "old_approach" {
  name        = "allow_tls"
  description = "Allow TLS inbound traffic"


  ingress {
    description      = "TLS from VPC"
    from_port        = 443
    to_port          = 443
    protocol         = "tcp"
    cidr_blocks      = ["10.77.32.50/32"]
  }
```

Old Approach

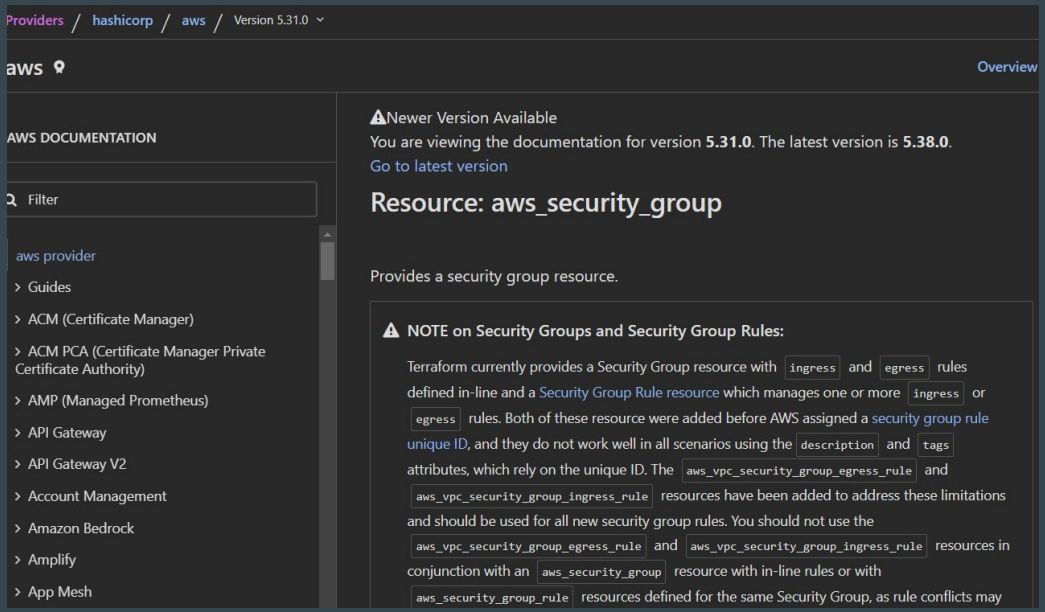
Points to Note

Just because a better approach is recommended, does NOT always mean that the older approach will stop working.

Organizations can continue to use the approach that suits best in it’s environment.

Switching to Older Provider Doc

You can always switch to the older version of provider documentation page to understand the changes.



Closing Pointers

For larger enterprises, it becomes difficult to upgrade their code base to the newer approach that provider recommends.

In such case, they stick with the appropriate provider version that supports the older approach of creating the resource.

Create Elastic IP with Terraform

Basics of Elastic IP in AWS

An Elastic IP address is a static IPv4 address in AWS.

You can create it and associate it with EC2 instance.



Aim of Today's Video

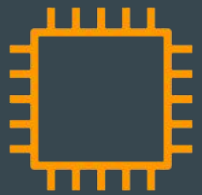
We are going to use Terraform to generate Elastic IP resource in AWS.

Attributes

Basics of Attributes

Each resource has its associated set of attributes.

Attributes are the fields in a resource that hold the values that end up in state.

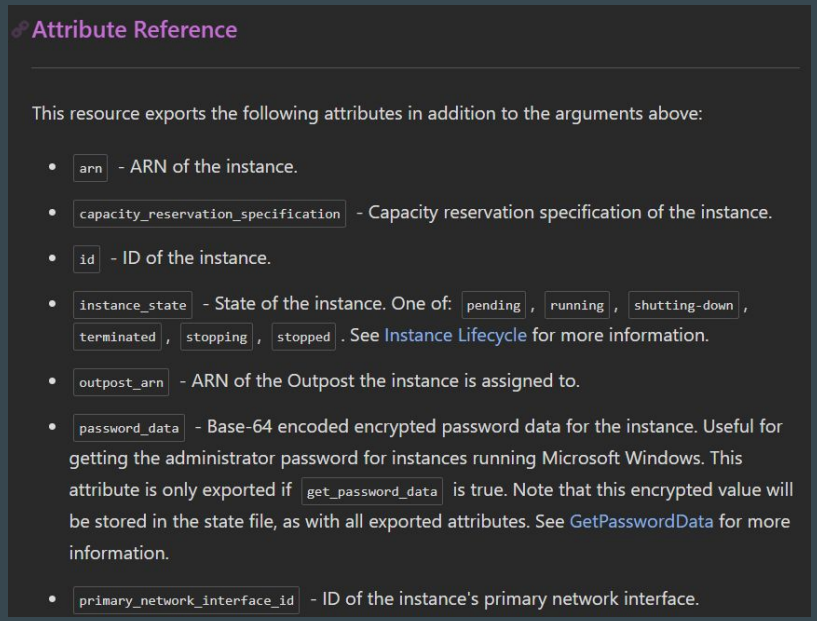




Attributes	Values
ID	i-abcd
public_ip	52.74.32.50
private_ip	172.31.10.50
private_dns	ip-172-31-10-50-.ec2.internal

Points to Note

Each resource type has a predefined set of attributes determined by the provider.



Attribute Reference

This resource exports the following attributes in addition to the arguments above:

- `arn` - ARN of the instance.
- `capacity_reservation_specification` - Capacity reservation specification of the instance.
- `id` - ID of the instance.
- `instance_state` - State of the instance. One of: `pending`, `running`, `shutting-down`, `terminated`, `stopping`, `stopped`. See Instance Lifecycle for more information.
- `outpost_arn` - ARN of the Outpost the instance is assigned to.
- `password_data` - Base-64 encoded encrypted password data for the instance. Useful for getting the administrator password for instances running Microsoft Windows. This attribute is only exported if `get_password_data` is true. Note that this encrypted value will be stored in the state file, as with all exported attributes. See GetPasswordData for more information.
- `primary_network_interface_id` - ID of the instance's primary network interface.

Cross-Resource Attribute References

Typical Challenge

It can happen that in a single terraform file, you are defining two different resources.

However Resource 2 might be dependent on some value of Resource 1.



Elastic IP Address

Security group

Allow 443 from Elastic IP

Understanding The Workflow

eip.tf > ...

```
resource "aws_eip" "lb" {
  domain   = "vpc"
}
```

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "HOW-TO-ADD-ELASTIC-IP-ADDRESS-HERE"
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}
```



Analyzing the Attributes of EIP

We have to find which attribute stores the Public IP associated with EIP Resource.

Attribute Reference

This resource exports the following attributes in addition to the arguments above:

- `allocation_id` - ID that AWS assigns to represent the allocation of the Elastic IP address for use with instances in a VPC.
- `association_id` - ID representing the association of the address with an instance in a VPC.
- `carrier_ip` - Carrier IP address.
- `customer_owned_ip` - Customer owned IP.
- `id` - Contains the EIP allocation ID.
- `private_dns` - The Private DNS associated with the Elastic IP address (if in VPC).
- `private_ip` - Contains the private IP address (if in VPC).
- `public_dns` - Public DNS associated with the Elastic IP address.
- `public_ip` - Contains the public IP address.

Referencing Attribute in Other Resource

We have to find a way in which attribute value of “public_ip” is referenced to the cidr_ipv4 block of security group rule resource.



Elastic IP

Attribute	Value
public_ip	52.72.52.72

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "REFERENCE-PUBLIC-IP-ATTRIBUTE-HERE"
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}
```

Cross Referencing Resource Attribute

Terraform allows us to reference the attribute of one resource to be used in a different resource.

Overall syntax:

<RESOURCE TYPE>.<NAME>.<ATTRIBUTE>

Cross Referencing Resource Attribute

We can specify the resource address with attribute for cross-referencing.



Elastic IP

Attribute	Value
public_ip	52.72.52.72

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "aws_eip.lb.public_ip"
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}
```

String Interpolation in Terraform

${...}): This syntax indicates that Terraform will replace the expression inside the curly braces with its calculated value.

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "${aws_eip.lb.public_ip}/32"
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}
```

Joke Time

Why did the Terraform attribute take a break?

...It was feeling over-referenced.

How did the Terraform attribute become a detective?

...It followed the resource trail.

Output Values

Understanding the Basics

Output values make information about your infrastructure available on the command line, and can expose information for other Terraform configurations to use.



Sample Example

Use-Case:

Create a Elastic IP (Public IP) resource in AWS and output the value of the EIP.

```
Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + demo = (known after apply)
aws_eip.lb: Creating...
aws_eip.lb: Creation complete after 3s [id=eipalloc-0680508decfe8c252]

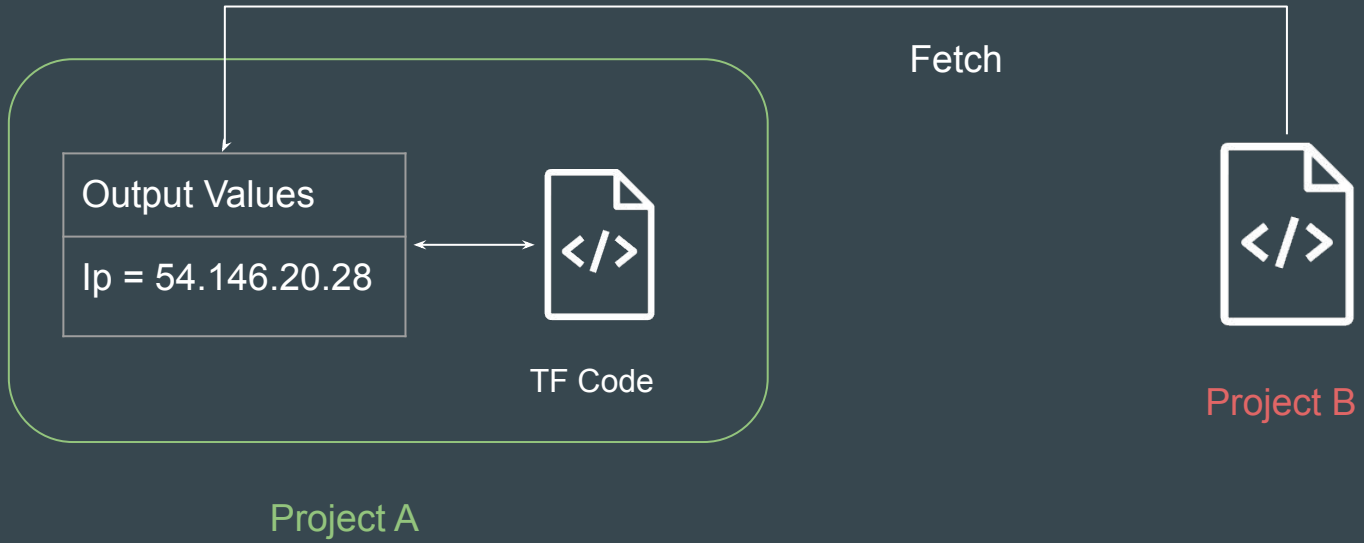
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

demo = "54.146.20.18"
```

Point to Note

Output values defined in Project A can be referenced from code in Project B as well.



Terraform Variables

Understanding the Challenge

Repeated static values in the code can create more work in the future.

Example: VPN IP needs to be whitelisted for 5 ports through Firewall Rules.

Port Number	CIDR Block	Description
80	101.0.62.210/32	VPN IP Whitelist
443	101.0.62.210/32	VPN IP Whitelist
22	101.0.62.210/32	VPN IP Whitelist
21	101.0.62.210/32	VPN IP Whitelist
8080	101.0.62.210/32	VPN IP Whitelist

Reference Screenshot

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "101.0.62.210/32"
  from_port         = 80
  ip_protocol       = "tcp"
  to_port           = 80
}
```

Firewall Rule 1

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "101.0.62.210/32"
  from_port         = 443
  ip_protocol       = "tcp"
  to_port           = 443
}
```

Firewall Rule 2

Better Approach

A better solution would be to define repeated static value in one central place.

Key	Value
vpn_ip	101.0.62.210/32

Central Location

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "fetch-from-central-location"
  from_port         = 443
  ip_protocol       = "tcp"
  to_port           = 443
}
```

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "fetch-from-central-location"
  from_port         = 8080
  ip_protocol       = "tcp"
  to_port           = 8080
}
```

Basics of Variables

Terraform input variables are used to pass certain values from outside of the configuration

Name	Value
vpn_ip	101.0.62.210/32
app_port	8080

Variable File


```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = var.vpn_ip
  from_port         = var.app_port
  ip_protocol       = "tcp"
  to_port           = var.app_port
}
```

Benefits of Variables

1. Update important values in one central place instead of searching and replacing them throughout your code, saving time and potential mistakes.

2. No need to touch the core Terraform configuration file. This can avoid human mistakes while editing.



Variable Definitions File (TFVars)

Understanding the Base

Managing variables in production environment is one of the very important aspect to keep code clean and reusable.

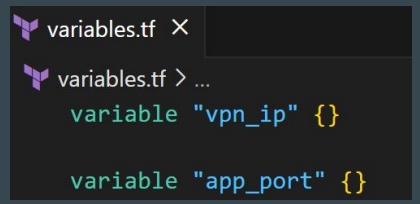
HashiCorp recommends creating a separate file with name of *.tfvars to define all variable value in a project.

How Recommended Folder Structure Looks Like

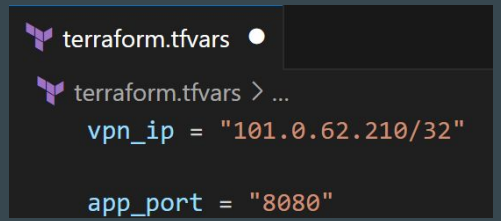
1. Main Terraform Configuration File.
2. variables.tf file that defines all the variables.
3. terraform.tfvars file that defines value to all the variables.



Main Configuration File



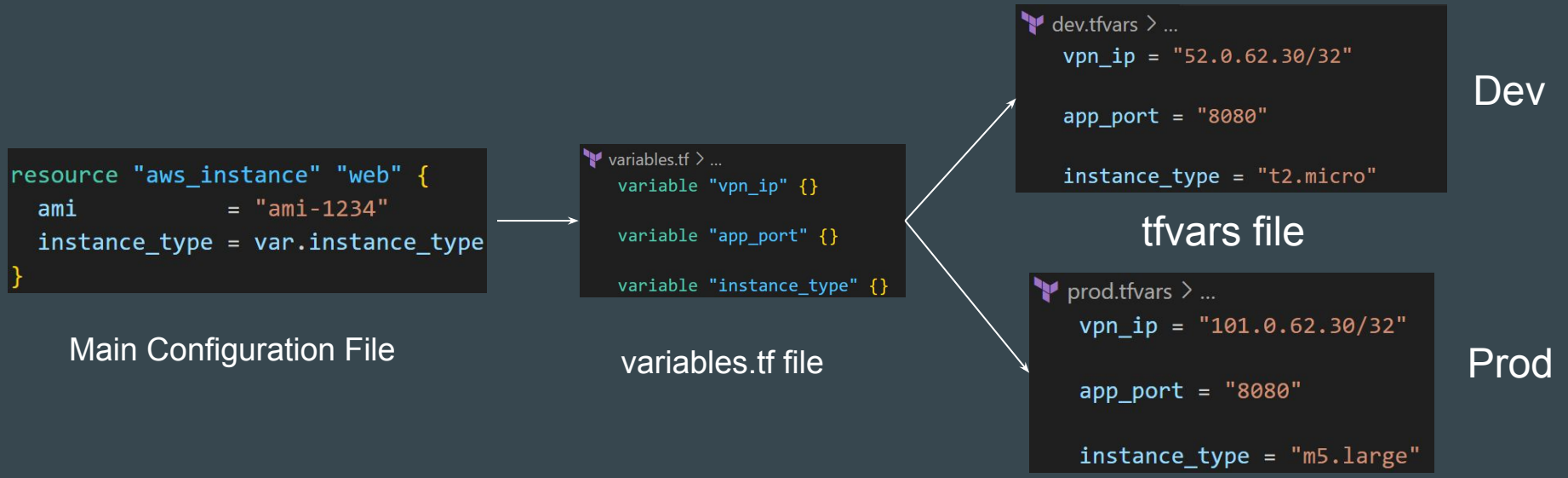
variables.tf File



terraform.tfvars file

Configuration for Different Environments

Organizations can have wide set of environments: Dev, Stage, Prod



Selecting tfvars File

If you have multiple variable definitions file (*.tfvars) file, you can manually define the file to use during command line.

```
C:\Users\zealv\kplabs-terraform>terraform plan -var-file="prod.tfvars"

Terraform used the selected providers to generate the following execution plan.
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.web will be created
  + resource "aws_instance" "web" {
      + ami                                  = "ami-1234"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
```

Point to Note

If file name is terraform.tfvars → Terraform will automatically load values from it.

If file name is different like prod.tfvars → You have to explicitly define the file during plan / apply operation.

```
C:\Users\zealv\kplabs-terraform>terraform plan -var-file="prod.tfvars"

Terraform used the selected providers to generate the following execution plan.
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.web will be created
  + resource "aws_instance" "web" {
      + ami                                  = "ami-1234"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
```

Approach to Variable Assignment

Understanding the Base

By default, whenever you define a variable, you must also set a value associated with it.

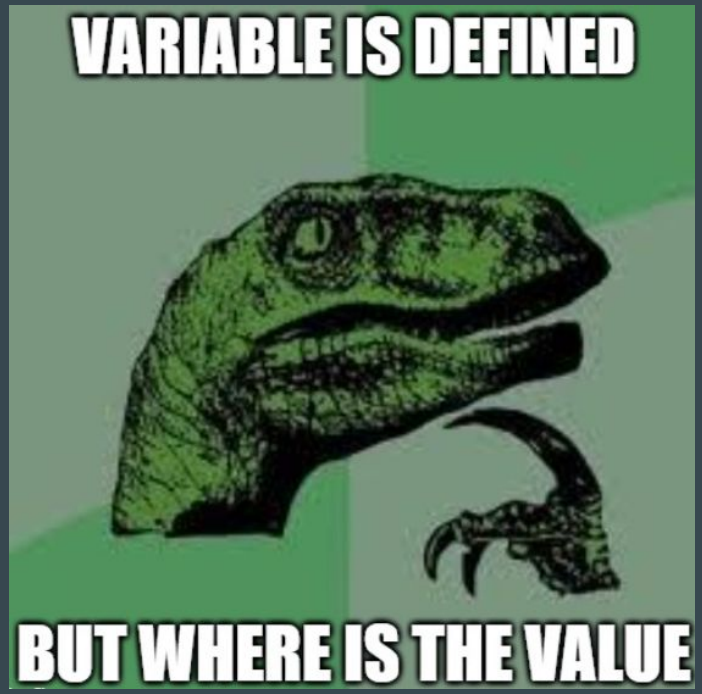
```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = var.instance_type
}
```

Main Configuration File

variables.tf > ...

```
variable "instance_type" {}
```

variables.tf


VARIABLE IS DEFINED
BUT WHERE IS THE VALUE

Add a Value in CLI

If you have not defined a value for a variable, Terraform will ask you to input the value in CLI Prompt when you run terraform plan / apply operation.

```
C:\Users\zealv\kplabs-terraform>terraform plan
var.instance_type
  Enter a value:
```

Declaring Variable Values

When variables are declared in your configuration, they can be set in a number of ways:

1. Variable Defaults.
2. Variable Definition File (*.tfvars)
3. Environment Variables
4. Setting Variables in the Command Line.

Variable Defaults

You can set a default value for a variable.

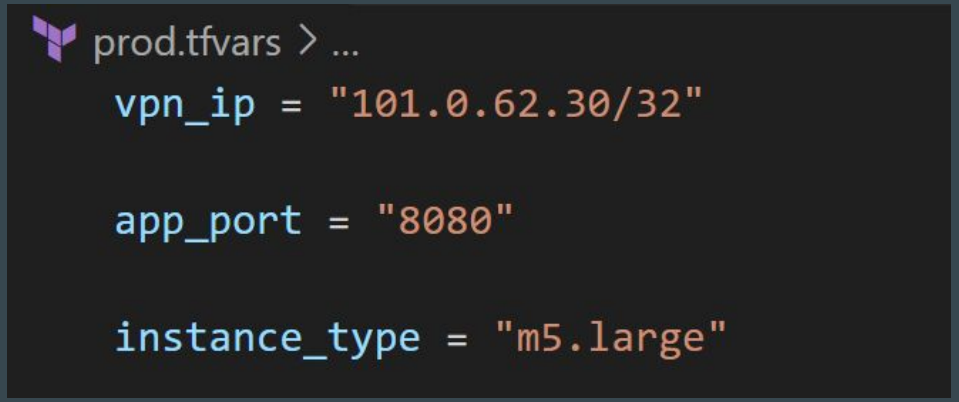
If there is no value supplied, the default value will be taken.

variables.tf > ...

```
variable "app_port" {
    default = "8080"
}
```

Variable Definition File (*.tfvars)

Variable Values can be defined in *.tfvars file.



Setting Variable in Command Line

To specify individual variables on the command line, use the -var option when running the terraform plan and terraform apply commands:

```
C:\Users\zealv\kplabs-terraform>terraform plan -var="instance_type=m5.large"

Terraform used the selected providers to generate the following execution plan. Resource
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.web will be created
  + resource "aws_instance" "web" {
      + ami                                  = "ami-1234"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "m5.large"
```

Setting Variable through Environment Variables

Terraform searches the environment of its own process for environment variables named TF_VAR_ followed by the name of a declared variable.

```
C:\Users\zealv\kplabs-terraform>echo %TF_VAR_instance_type%
t2.large

C:\Users\zealv\kplabs-terraform>terraform plan

Terraform used the selected providers to generate the following execution plan
following symbols:
  + create

Terraform will perform the following actions:

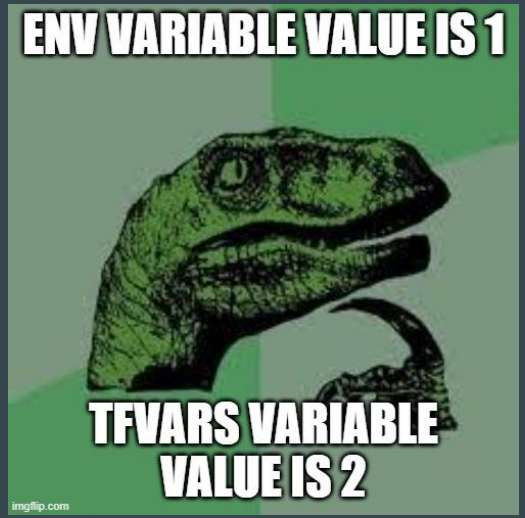
  # aws_instance.web will be created
  + resource "aws_instance" "web" {
      + ami                                  = "ami-1234"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
```

Variable Definition Precedence

Understanding the Base

Values for a variable can be defined at multiple different places.

What if values for a variable are different?



Simple Example

variable “instance_type” {}

1. Default Value is t2.micro
2. Terraform.tfvars value is “t2.small”
3. Environment Variable TF_VAR_instance_type = “t2.large”

Which value will Terraform take?

Variable Definition Precedence

Terraform loads variables in the following order, with later sources taking precedence over earlier ones:

1. Environment variables
2. The terraform.tfvars file, if present.
3. The terraform.tfvars.json file, if present.

4. Any *.auto.tfvars or *.auto.tfvars.json files, processed in lexical order of their filenames.

5. Any -var and -var-file options on the command line

Example 1

ENV Variable of TF_VAR_instance_type = “t2.micro”

Value in terraform.tfvars = “t2.large”

Final Result = “t2.large”

Example 2

1. ENV Variable of TF_VAR_instance_type = “t2.micro”
2. Value in terraform.tfvars = “t2.large”
3. terraform plan -var="instance_type=m5.large"

Final Result = “m5.large”

Data Types

Setting the Base

Data type refers to the type of value.

Depending on the requirement, you can use wide variety of values in Terraform configuration.

Example Data Type	Data Type
“Hello World”	String
7575	Number

Restricting Variable Value to Data Type

We can restrict the value of a variable to a data type.

Example:

Only numbers should be allowed in AWS Usernames.

```
variable "username" {
  type = number
}
```

Data Types in Terraform

Data Types	Description
string	a sequence of Unicode characters representing some text, like "hello".
number	A Numeric value
bool	a boolean value, either true or false
list	a sequence of values, like ["us-west-1a", "us-west-1c"]
set	a collection of unique values that do not have any secondary identifiers or ordering.
map	a group of values identified by named labels, like {name = "Mabel", age = 52}.
null	a value that represents absence or omission.

Data Type - List

List Data Type

Allows us to store collection of values for a single variable / argument.

Represented by a pair of square brackets containing a comma-separated sequence of values, like ["a", 15, true].

Useful when multiple values needs to be added for a specific argument

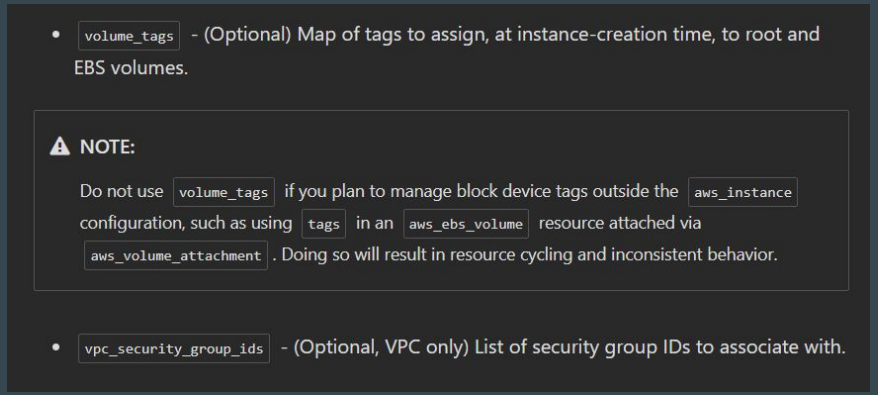
```
variable "my-list" {
  type = list
  default = ["mumbai","bangalore","delhi"]
}
```

Data Type and Documentation

Arguments for a resource requires specific data types.

Some argument requires list, some requires map and so on.

The details of data type expected for an argument is mentioned in documentation.



- `volume_tags` - (Optional) Map of tags to assign, at instance-creation time, to root and EBS volumes.

> ⚠ **NOTE:**
>
> Do not use `volume_tags` if you plan to manage block device tags outside the `aws_instance` configuration, such as using `tags` in an `aws_ebs_volume` resource attached via `aws_volume_attachment`. Doing so will result in resource cycling and inconsistent behavior.

- `vpc_security_group_ids` - (Optional, VPC only) List of security group IDs to associate with.

Use-Case 1: List Data Type

EC2 instance can have one or more security groups.

Requirement:

Create EC2 instance with 2 security groups attached.

Specify the Type of Values in List

We can also specify the type of values expected in a list.

```
variable "my-list" {
  type = list(number)
  default  = ["1","2","3"]
}
```

Map - Data Type

Map Data Type

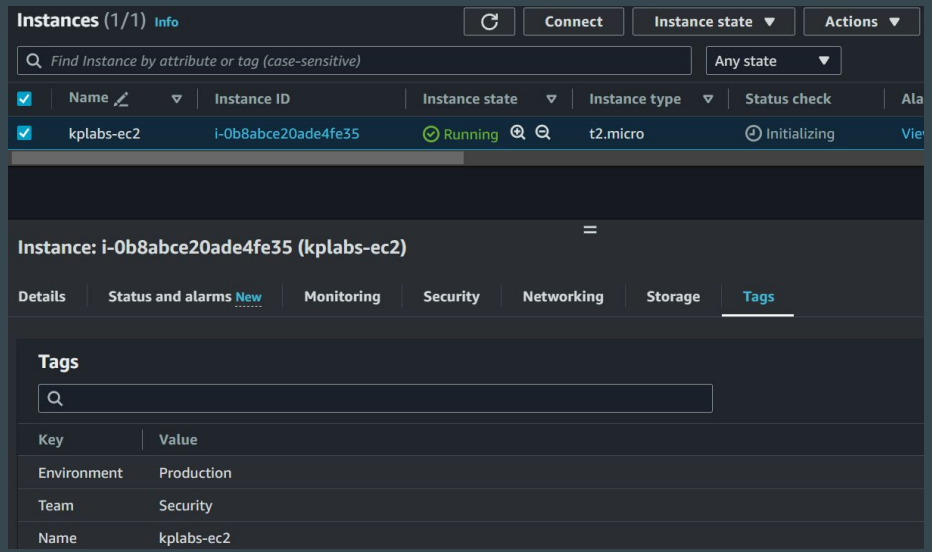
A map data type represents a collection of key-value pair elements

```
variable "instance_tags" {
  type = map
  default = {
    Name = "app-server"
    Environment = "development"
    Team = "payments"
  }
}
```

Use-Case of Map

We can add multiple tags to AWS resources.

These tags are key-value pairs.

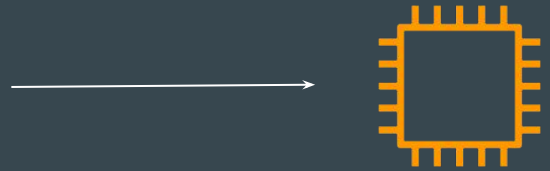


The COUNT Meta-Argument

Understanding the Challenge

By default, a resource block configures one real infrastructure object.

```
resource "aws_instance" "first_ec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
}
```



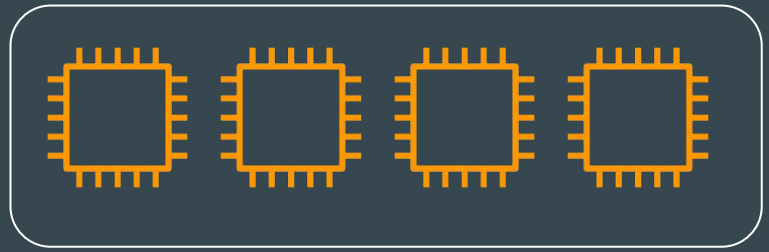
```
resource "aws_instance" "second_ec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
}
```



Understanding the Use-Case

Sometimes you want to manage several similar objects (like a fixed pool of compute instances) without writing a separate block for each one.

```
resource "aws_instance" "second_ec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
    <something-logic-here>
}
```

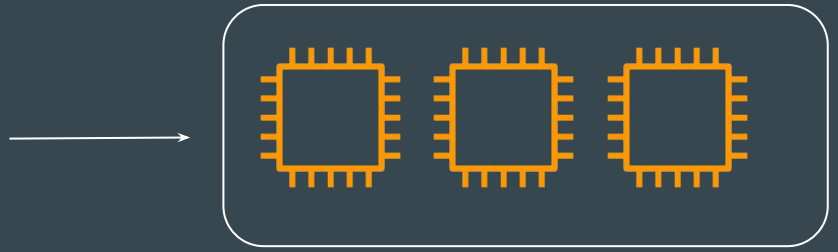


Pool of Servers

Introducing Count Argument

The count argument accepts a whole number, and creates that many instances of the resource.

```
resource "aws_instance" "second_ec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
    count = 3
}
```

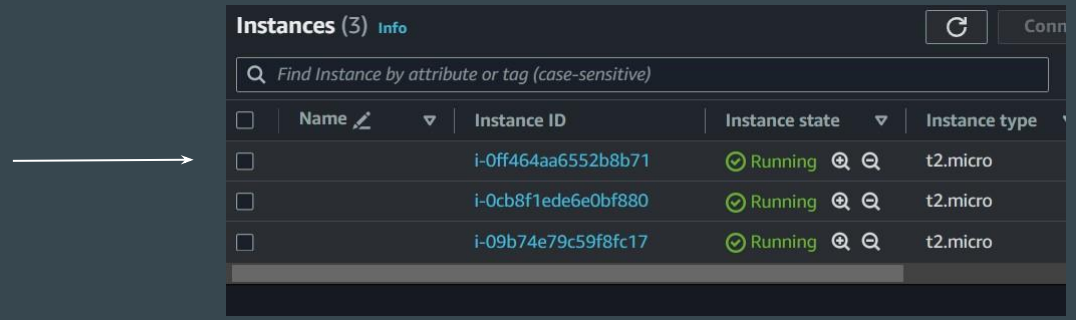


Pool of Servers

Challenges with Count

The instances created through count and identical copies, but you might want to customize certain properties for each one.

```
resource "aws_instance" "first_ec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
    count = 3
}
```



Example - IAM User

For many resources, exact identical copies are not required and will not work.

Example: You cannot have multiple AWS Users with exact same name.

```
resource "aws_iam_user" "lb" {
  name = "developer-user"
  count = 3
}
```



```
Error: creating IAM User (developer-user): operation error IAM: CreateUser, https response error Status
: 44c3ff43-0bd4-430e-a838-ba5e43aa569a, EntityAlreadyExists: User with name developer-user already exists

  with aws_iam_user.lb[0],
  on iam.tf line 2, in resource "aws_iam_user" "lb":
   2: resource "aws_iam_user" "lb" {
```

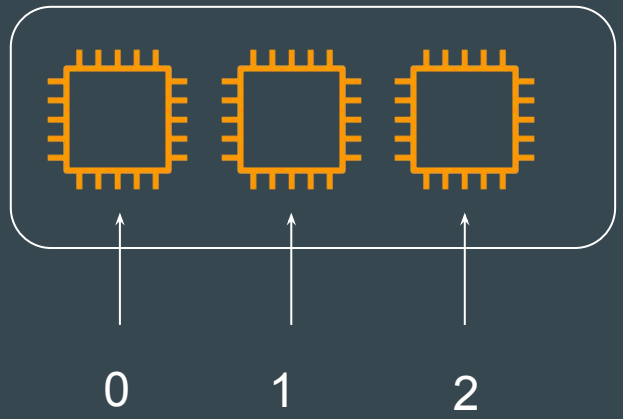
COUNT.INDEX

Introducing Count Index

When using count, you can also make use of count.index which allows better flexibility.

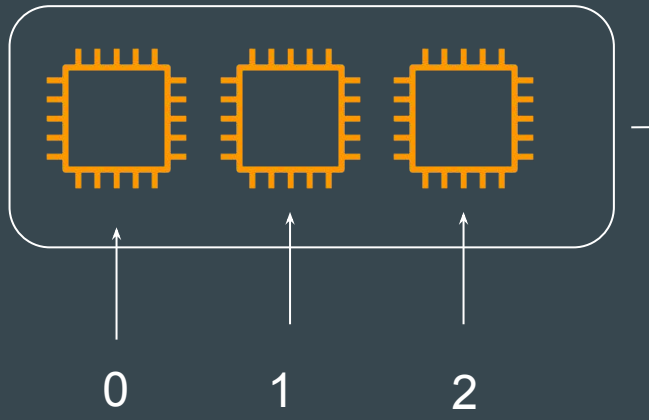
This attribute holds a distinct index number, starting from 0, that uniquely identifies each instance created by the count meta-argument.

```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
    count = 3
}
```



Tabular Representation

Following representation shows each EC2 instance's resource address that contains the index.



Resource Address	Description
aws_instance.myec2[0]	First EC2 Instance
aws_instance.myec2[1]	Second EC2 Instance
aws_instance.myec2[2]	Third EC2 Instance

CLI Output

Within CLI output, you will be able to see the index value of resource.

```
# aws_instance.first_ec2[0] will be created
+ resource "aws_instance" "first_ec2" {
    + ami                                  = "ami-00c39f71452c08778"
    + arn                                  = (known after apply)
    + associate_public_ip_address          = (known after apply)
    + availability_zone                    = (known after apply)
    + cpu_core_count                       = (known after apply)
    + cpu_threads_per_core                 = (known after apply)
    + disable_api_stop                     = (known after apply)
    + disable_api_termination              = (known after apply)
    + ebs_optimized                        = (known after apply)
    + get_password_data                    = false
    + host_id                              = (known after apply)
    + host_resource_group_arn              = (known after apply)
    + iam_instance_profile                 = (known after apply)
    + id                                   = (known after apply)
```

First EC2

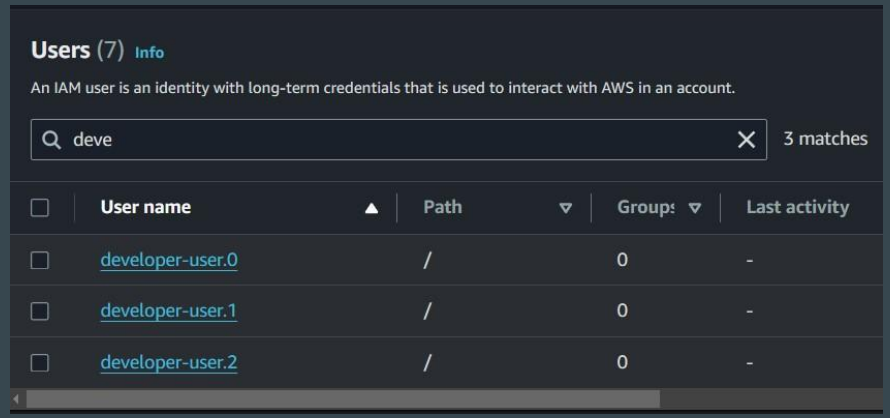
```
# aws_instance.first_ec2[1] will be created
+ resource "aws_instance" "first_ec2" {
    + ami                                  = "ami-00c39f71452c08778"
    + arn                                  = (known after apply)
    + associate_public_ip_address          = (known after apply)
    + availability_zone                    = (known after apply)
    + cpu_core_count                       = (known after apply)
    + cpu_threads_per_core                 = (known after apply)
    + disable_api_stop                     = (known after apply)
    + disable_api_termination              = (known after apply)
    + ebs_optimized                        = (known after apply)
    + get_password_data                    = false
    + host_id                              = (known after apply)
    + host_resource_group_arn              = (known after apply)
    + iam_instance_profile                 = (known after apply)
    + id                                   = (known after apply)
```

Second EC2

Example - IAM User Use-Case

The ${count.index} is dynamic expression that utilizes the count.index attribute so that each username will be unique.

```
resource "aws_iam_user" "lb" {
  name = "developer-user.${count.index}"
  count = 3
}
```

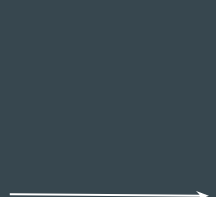


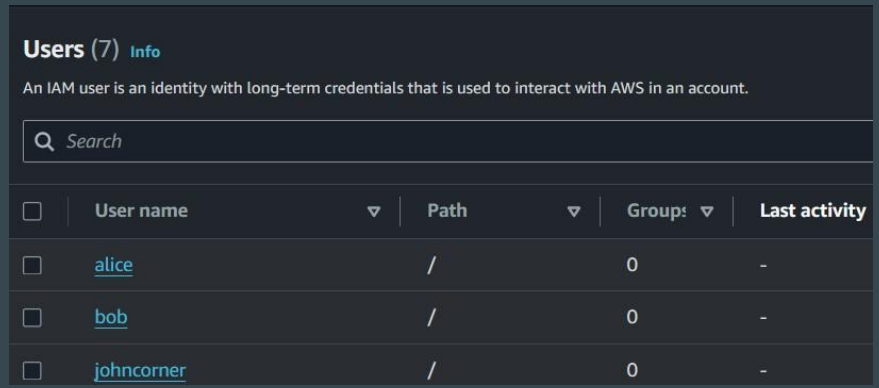
Enhancing with Count Index

You can use count.index to iterate through the list to have more customization.

```
variable "dev_names" {
  type = list
  default = ["alice", "bob","johncorner"]
}


resource "aws_iam_user" "lb" {
  name = var.dev_names[count.index]
  count = 3
}
```

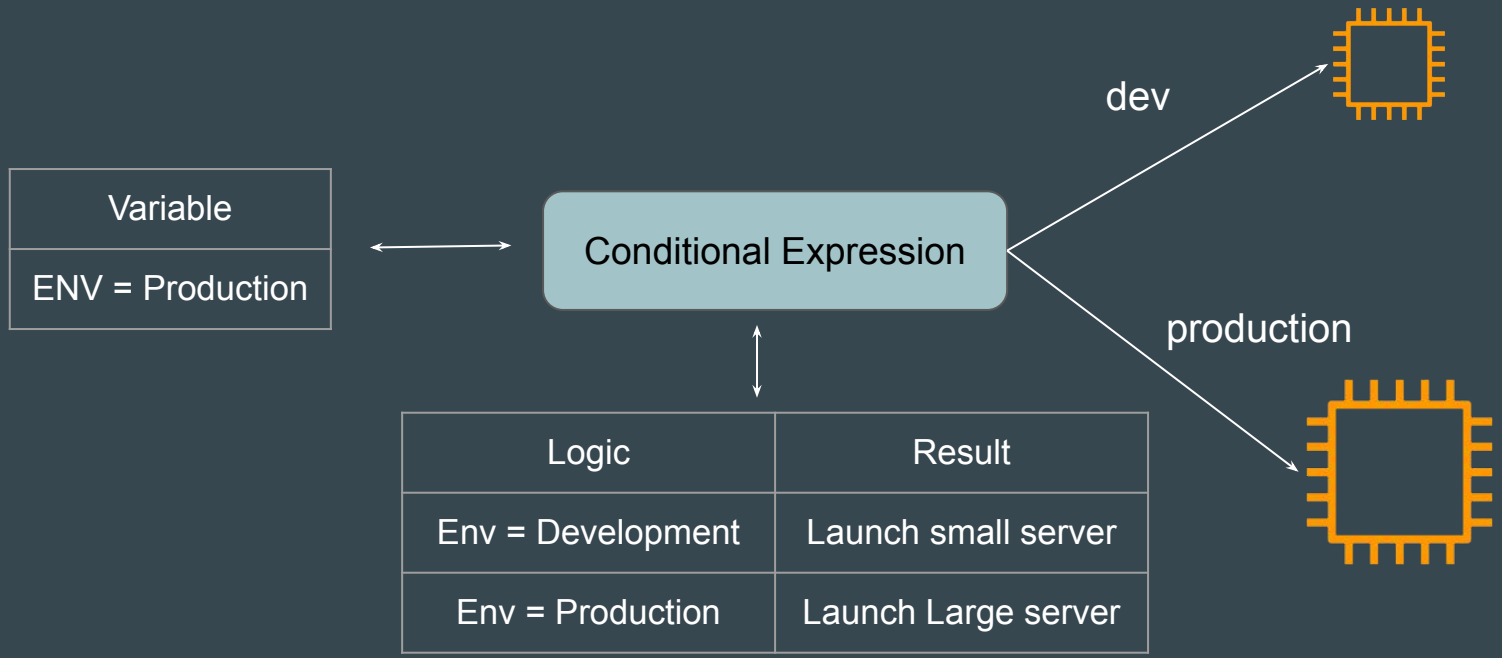




Conditional Expressions

Setting the Base

Conditional expressions in Terraform allow you to choose between two values based on a condition



Syntax of Conditional Expression

The syntax of a conditional expression is as follows:

```
condition ? true_val : false_val
```

If condition is true then the result is true_val. If condition is false then the result is false_val.

Conditional Expression Based on Use-Case

If Environment is Development, t2.micro instance type should be used.

If Environment is NOT development, m5.large instance type should be used.

```
variable "environment" {
    default = "development"
}

resource "aws_instance" "example" {
  instance_type = var.environment == "development" ? "t2.micro" : "m5.large"
  ami           = "ami-12345678"
}
```

Conditional Expression with Multiple Variables

In the following example, only if env=production and region=us-east-1, the larger instance type of m5.large can be used.

```
variable "environment" {
    default = "production"
}

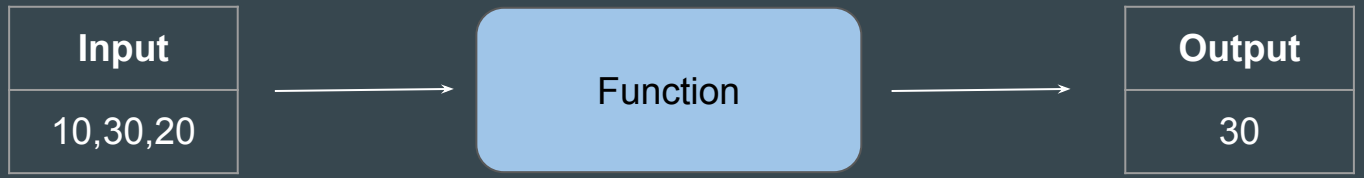
variable "region" {
    default = "ap-south-1"
}

resource "aws_instance" "example" {
  instance_type = var.environment == "production" && var.region == "us-east-1" ? "m5.large" : "t2.micro"
  ami           = "ami-12345678"
}
```

Terraform Functions

Basics of Function

A function is a block of code that performs a specific task.



max ()

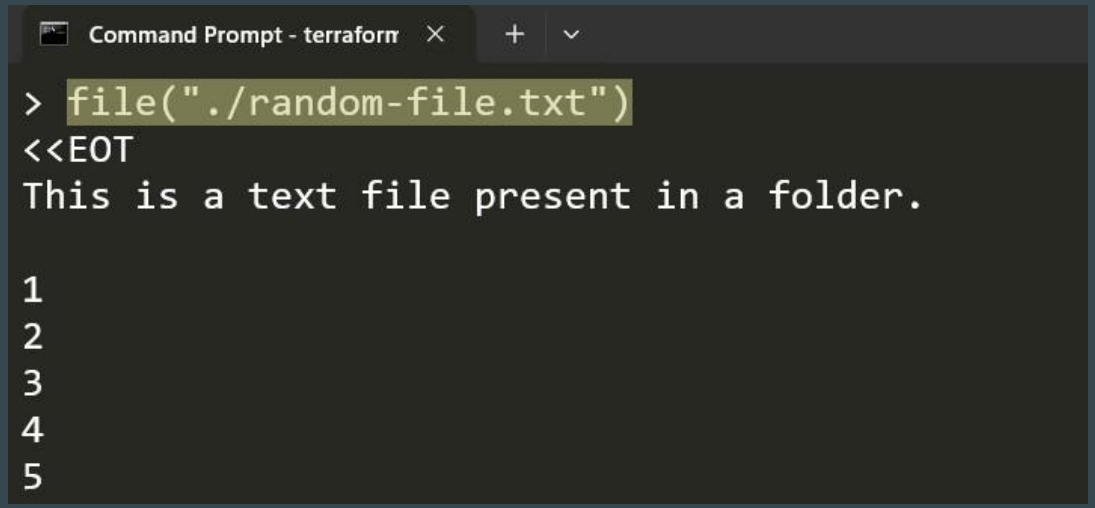
Function 1 - MAX

max () takes one or more numbers and returns the greatest number.

```
C:\Users\zealv\kplabs-terraform>terraform console
> max(10,20,30)
30
```

Function 2 - FILE

file () reads the contents of a file at the given path and returns them as a string.



Introducing Terraform Console

Terraform Console provides an interactive environment specifically designed to test functions and experiment with expressions before integrating them into your main code.

```
C:\Users\zealv\kplabs-terraform>terraform console
> max(10,20,30)
30
```

Importance of File Function

file reads the contents of a file at the given path and returns them as a string.

```
resource "aws_iam_user_policy" "lb_ro" {
  name = "demo-user-policy"
  user = aws_iam_user.this.name

  policy = jsonencode({
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": "ec2:*",
            "Effect": "Allow",
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": "elasticloadbalancing:*",
            "Resource": "*"
        },
```

Before



```
resource "aws_iam_user_policy" "lb_ro" {
  name = "demo-user-policy"
  user = aws_iam_user.this.name

  policy = file("./ec2-policy.json")
}
```

After

Functions in Terraform

Terraform has wide variety of functions available to achieve different set of use-cases.

Functions are grouped into categories. Some of these include:

Function Categories	Functions Available
Numeric Functions	abs, ceil, floor, max, min
String Functions	concat, replace, split, tolower,toupper
Collection Functions	element, keys, length, merge, sort
Filesystem Functions:	file, filebase64, dirname

Important Point to Note

The Terraform language does not support user-defined functions, and so only the functions built in to the language are available for use

The documentation includes a page for all of the available built-in functions.

Challenge - Analyzing Code Containing Functions

Setting the Base

As part of this challenge, you will be given a code that contains multiple sets of Terraform Functions.

You have to analyze what this code does without running the “apply” operation.

```
variable "ami" {
  type = map
  default = {
    "us-east-1" = "ami-08a0d1e16fc3f61ea"
    "us-west-2" = "ami-0d6621c01e8c2de2c"
    "ap-south-1" = "ami-0470e33cd681b2476"
  }
}


resource "aws_instance" "app-dev" {
   ami = lookup(var.ami,var.region)
   instance_type = "t2.micro"
   count = length(var.tags)

   tags = {
     Name = element(var.tags,count.index)
     CreationDate = formatdate("DD MMM YYYY hh:mm ZZZ",timestamp())
   }
}
```

Overall Workflow

1. Analyze what exactly the given code in GitHub will do without running the “apply operation”.

2. Analyze the outcome by applying function using Terraform Console and reading the documentation.

3. Make a note of it.

4. Run the “terraform apply” operation to verify if it matches your findings.

Solution - Analyzing Code Containing Functions

1- Analyzing Lookup Function

lookup retrieves the value of a single element from a map, given its key.

Format: lookup(map, key, default)

```
> lookup({a="ay", b="bee"}, "a", "what?")
ay
> lookup({a="ay", b="bee"}, "c", "what?")
what?
```

Testing Lookup Function

To test lookup function, add the details that are part of the map associated with variable of ami and the default value of variable of region.

```
variable "ami" {
  type = map
  default = {
    "us-east-1" = "ami-08a0d1e16fc3f61ea"
    "us-west-2" = "ami-0d6621c01e8c2de2c"
    "ap-south-1" = "ami-0470e33cd681b2476"
  }
}
```

```
variable "region" {
  default = "us-east-1"
}
```

terraform console

```
> lookup({"us-east-1" = "ami-08a0d1e16fc3f61ea","us-west-2" = "ami-0d6621c01e8c2de2c","ap-south-1" = "ami-0470e33cd681b2476"}
,"us-east-1")
"ami-08a0d1e16fc3f61ea"
```

2 - Analyzing Length Function

length determines the length of a given list, map, or string.

```
> length([])
0
> length(["a", "b"])
2
> length({"a" = "b"})
1
> length("hello")
5
```

Testing Length Function

Code: count = length(var.tags)

```
variable "tags" {
  type = list
  default = ["firstec2","secondec2"]
}
```



```
> length(["firstec2","secondec2"])
2
```

3 - Analyzing Element Function

element retrieves a single element from a list.

Format: element(list, index)

```
> element(["a", "b", "c"], 1)
b
```

Testing Element Function

Code: Name = element(var.tags,count.index)

```
variable "tags" {
  type = list
  default = ["firstec2","secondec2"]
}
```



```
> element(["firstec2","secondec2"],0)
"firstec2"
```

```
> element(["firstec2","secondec2"],1)
"secondec2"
```

4 - Analyzing TimeStamp Function

timestamp returns a UTC timestamp string in RFC 3339 format.

```
> formatdate("DD MMM YYYY hh:mm ZZZ", "2018-01-02T23:12:01Z")
02 Jan 2018 23:12 UTC
> formatdate("EEEE, DD-MMM-YY hh:mm:ss ZZZ", "2018-01-02T23:12:01Z")
Tuesday, 02-Jan-18 23:12:01 UTC
> formatdate("EEE, DD MMM YYYY hh:mm:ss ZZZ", "2018-01-02T23:12:01-08:00")
Tue, 02 Jan 2018 23:12:01 -0800
> formatdate("MMM DD, YYYY", "2018-01-02T23:12:01Z")
Jan 02, 2018
> formatdate("HH:mmaa", "2018-01-02T23:12:01Z")
11:12pm
```

Testing TimeDate Function

A simple call to the timestamp () returns the timestamp value

```
> timestamp()
"2024-06-16T13:43:13Z"
```

5 - Analyzing Formatdate Function

formatdate converts a timestamp into a different time format.

```
> formatdate("DD MMM YYYY hh:mm ZZZ", "2018-01-02T23:12:01Z")
02 Jan 2018 23:12 UTC
> formatdate("EEEE, DD-MMM-YY hh:mm:ss ZZZ", "2018-01-02T23:12:01Z")
Tuesday, 02-Jan-18 23:12:01 UTC
> formatdate("EEE, DD MMM YYYY hh:mm:ss ZZZ", "2018-01-02T23:12:01-08:00")
Tue, 02 Jan 2018 23:12:01 -0800
> formatdate("MMM DD, YYYY", "2018-01-02T23:12:01Z")
Jan 02, 2018
> formatdate("HH:mmaa", "2018-01-02T23:12:01Z")
11:12pm
```

5 - Testing Formatdate Function

Code Block:

```
CreationDate = formatdate("DD MMM YYYY hh:mm ZZZ",timestamp())
```

```
> formatdate("DD MMM YYYY hh:mm ZZZ",timestamp())
"16 Jun 2024 13:46 UTC"
```

Final Result

1. Two set of EC2 instances will be created.

2. Name of EC2 will be “firstec2”, and “secondec2”

3. EC2 will have a tag of creation date with the timestamp value

You are Awesome

Learning “Terraform Function” is a longer learning journey compared to other topics.

In today’s video, we learnt the practical aspect of Function in Terraform Code.



Local Values

Understanding the Challenge

Various resources in your project can have common values like tags.

Repeating these values across multiple resource blocks increases the code length and makes it difficult to manage in larger projects.

```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
    tags = {
      Team = "payments-team"
  }
}
```

```
resource "aws_security_group" "allow_tls" {
  name        = "firewall_sg"
  tags = {
      Team = "payments-system"
  }
}
```

Solution using Variables

One solution is to centralize these common values using Variables

```
variable "payment_tag" {
    type = map
    default  = {
        Team = "payments-team"
    }
}
```

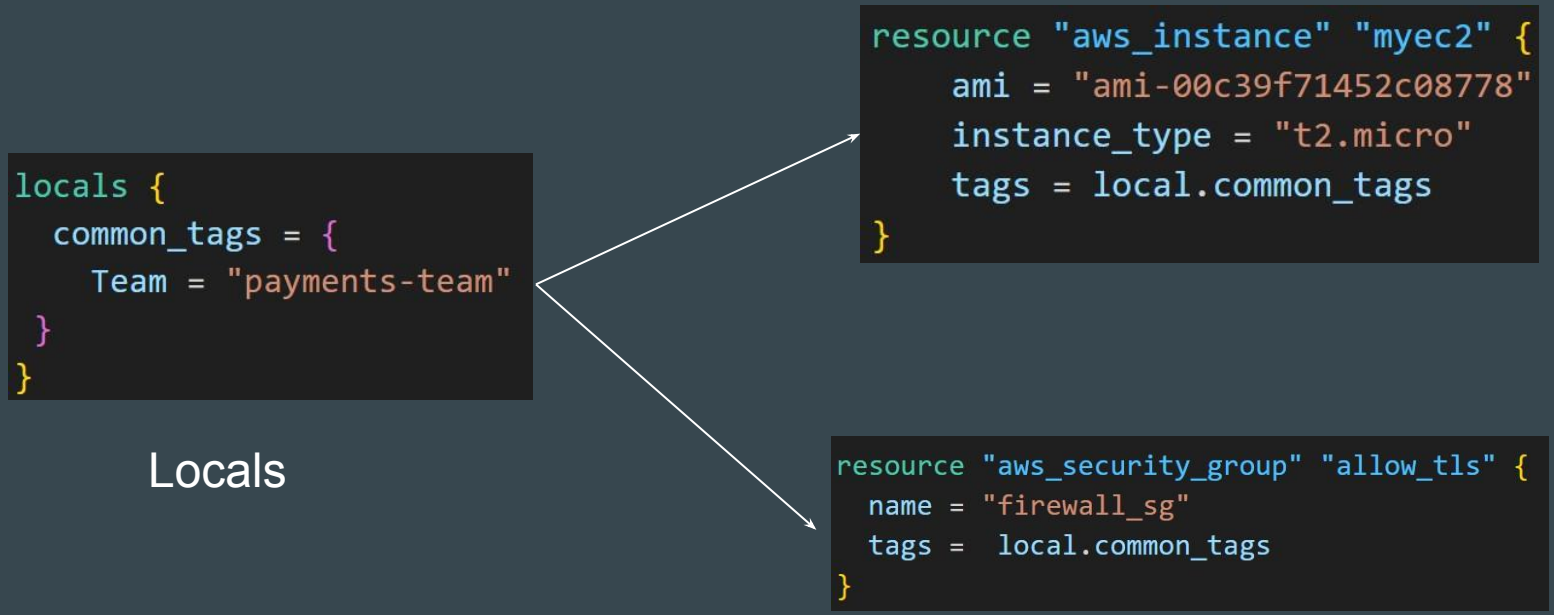
Variable

```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
    tags = var.payment_tag
}
```

```
resource "aws_security_group" "allow_tls" {
  name = "firewall_sg"
  tags =  var.payment_tag
}
```

Introducing Local Values

Local Values are similar to Variables in a sense that it allows you to store data centrally and that can be referenced in multiple parts of configuration.



Additional Benefit of Locals

You can add expressions to locals, which allows you to compute values dynamically

```
locals {
  # Ids for multiple sets of EC2 instances, merged together
  instance_ids = concat(aws_instance.blue.*.id, aws_instance.green.*.id)
}

locals {
  # Common tags to be assigned to all resources
  common_tags = {
    Service = local.service_name
    Owner   = local.owner
  }
}
```

Locals vs Variables

Variable value can be defined in wide variety of places like terraform.tfvars, ENV Variables, CLI and so on.

Locals are more of a private resource. You have to directly modify the code.

Locals are used when you want to avoid repeating the same expression multiple times.

Important Points to Note

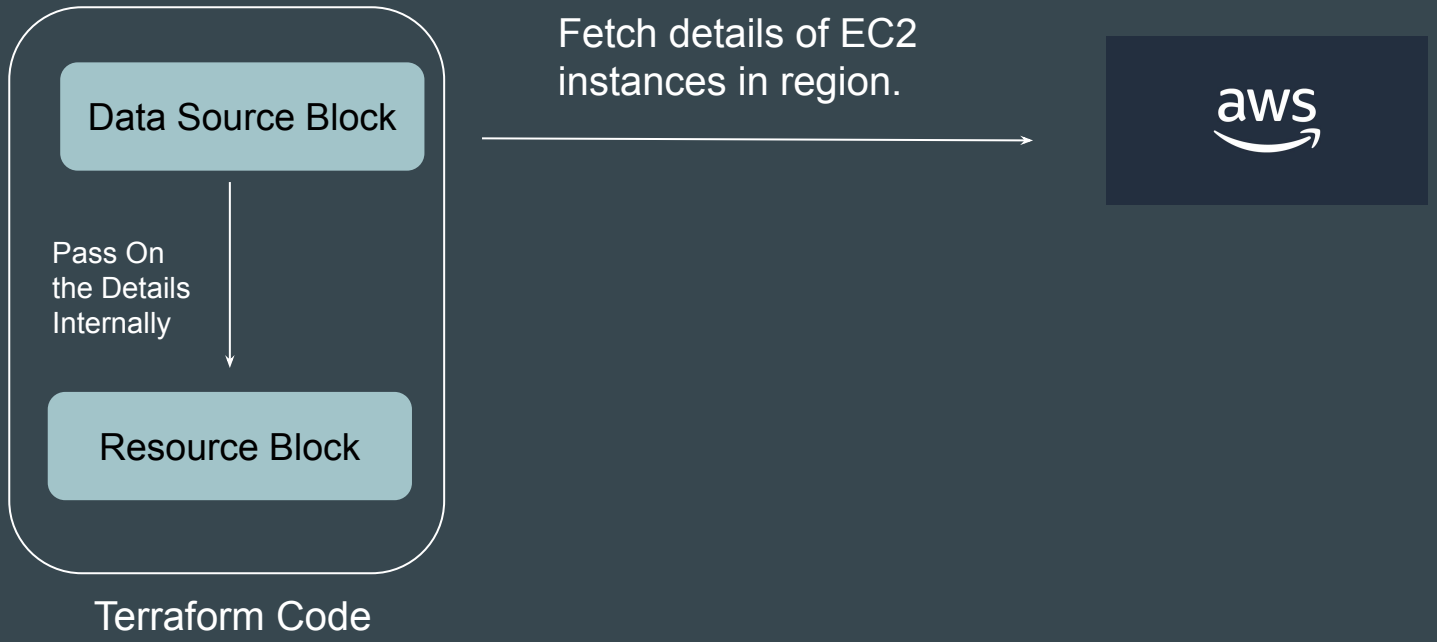
Local values are often referred to as just "locals"

Local values are created by a locals block (plural), but you reference them as attributes on an object named local (singular)

Data Sources

Introducing Data Sources

Data sources allow Terraform to use / fetch information defined outside of Terraform



Example 1 - Reading Info of DO Account

Following data source code is used to get information on your DigitalOcean account.

```
data "digitalocean_account" "example" {}
```

Example 2 - Reading a File

Following data source allows you to read contents of a file in your local filesystem.

```
data "local_file" "foo" {
  filename = "${path.module}/demo.txt"
}
```

Clarity regarding path.module

${path.module} returns the current file system path where your code is located.

```
data "local_file" "foo" {
  filename = "${path.module}/demo.txt"
}
```

Example 3 - Fetch EC2 Instance Details

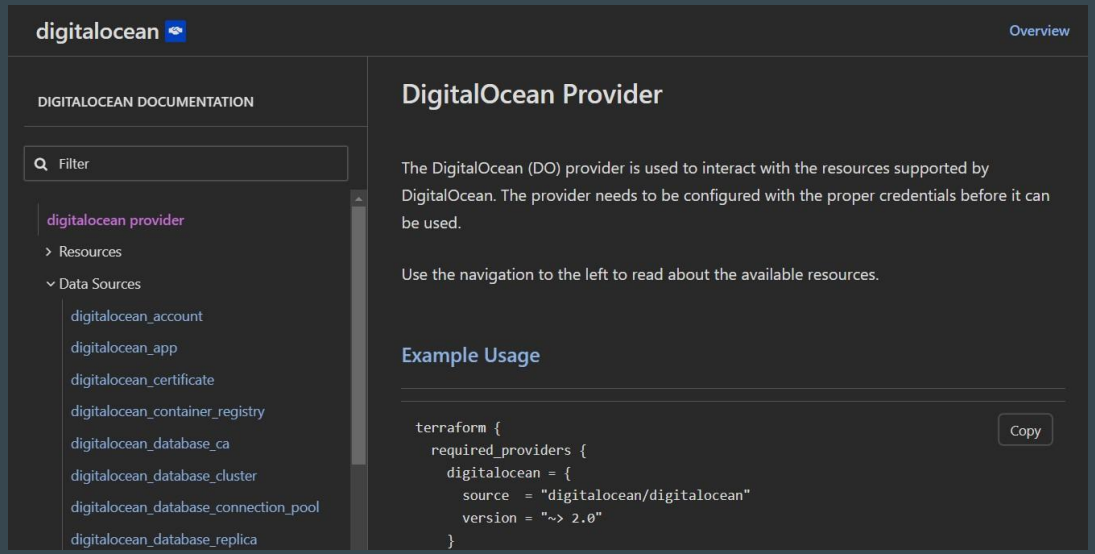
Following data source code is used to fetch details about the EC2 instance in your AWS region.

```
data "aws_instances" "example" {}
```

Data Sources Documentation Reference

Finding Available Data Sources

List of available data source are associated with each resource of a provider.



Data Sources Format

Understanding the Basic Structure

A data source is accessed via a special kind of resource known as a data resource, declared using a data block:

Following data block requests that Terraform read from a given data source ("aws_instance") and export the result under the given local name ("foo").

```
data "aws_instance" "foo" {}
```

Filter Structure

Within the block body (between { and }) are query constraints defined by the data source.

```
data "aws_instance" "foo" {
  filter {
    name   = "tag:Team"
    values = ["Production"]
  }
}
```

Fetching Latest OS Image Using Data Sources

Understanding the Requirement

You have been given a requirement to write a Terraform code that creates EC2 instance using latest OS Image of Amazon Linux.

Approach that New User will Take

We want to use the latest OS image for creating server in AWS.

Steps that we typically follow:

1. Go to EC2 Console.
2. Fetch the latest AMI ID
3. Add that AMI ID in Terraform code.



Sample Reference Code

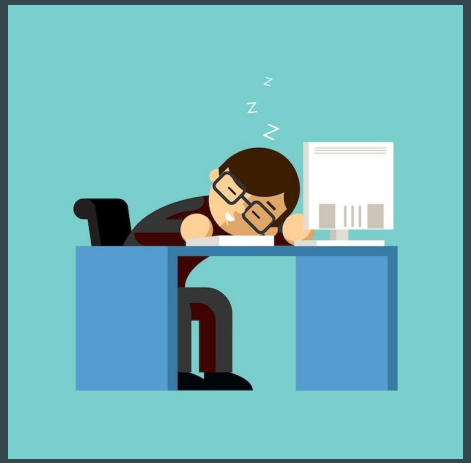
```
resource "aws_instance" "web" {
  ami           = "ami-0440d3b780d96b29d"
  instance_type = "t2.micro"
}
```



Hard Coded static value

Static Information is Boring

Hardcoding static details in your Terraform code will lead you to repeatedly modify your code to meet changing requirements.

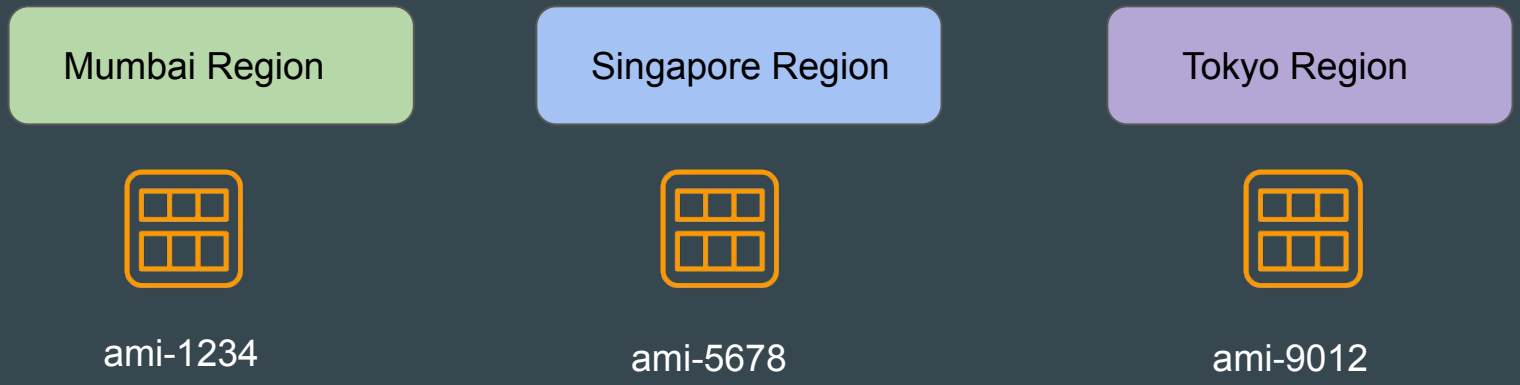


Another Challenge with Static Values

In many of the cases, the static value changes depending on the region.

Example: AMI IDs are specific to region.

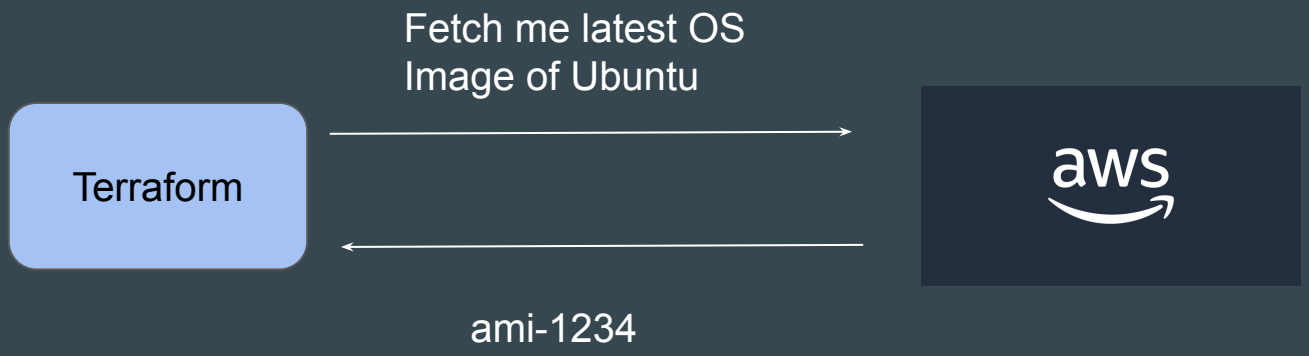
Hardcoded AMI in code will only work for single region.



Time to be Pros - Dynamic Configuration

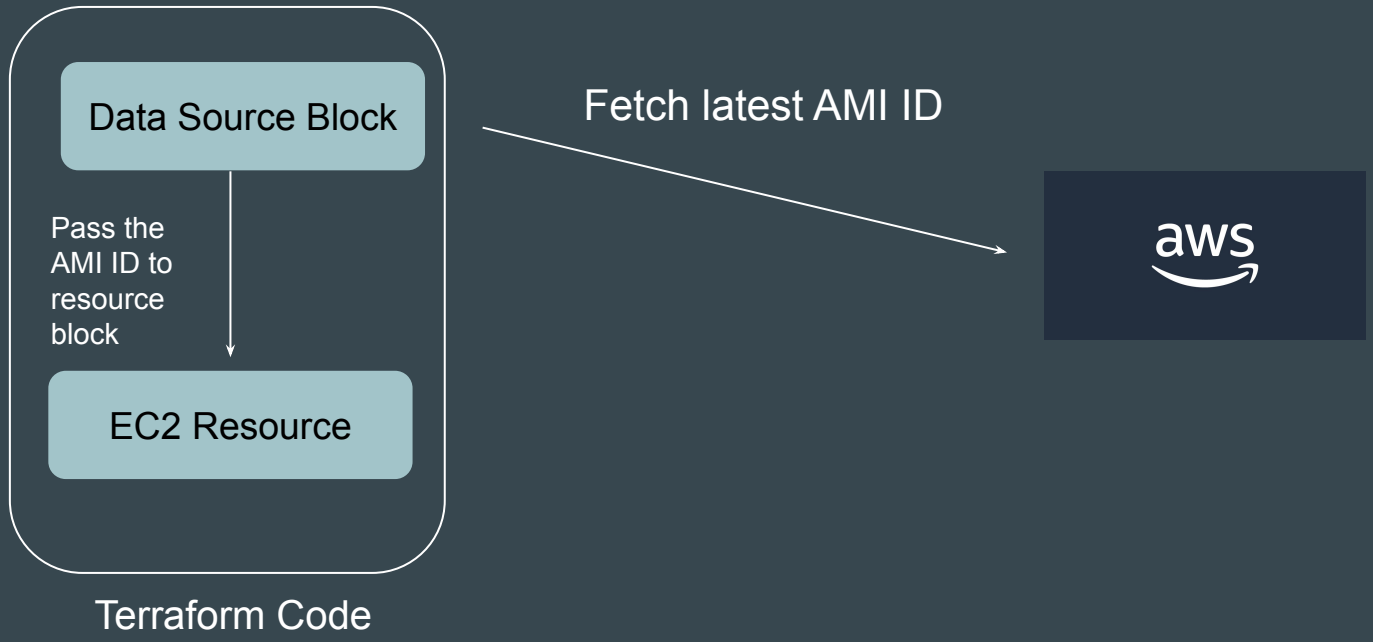
We want Terraform to automatically query the latest OS image in AWS or any other provide and use that for creating server.

We need code which works for all region without modification.



Introducing Data Sources

Data sources allow Terraform to use information defined outside of Terraform and we can use that information to provision resources.



Debugging Terraform

Basics of Debugging

Debugging is the process of finding the root cause of a specific issue.

30-40% of the time of a System Administrator goes into Debugging.



Example - SSH Verbosity

One of the important requirement in Debugging is getting detailed Log

Depending on the application, the approach to get detailed logs will differ.

```
C:\Users\zealv\OneDrive\Desktop>ssh -v -i dp_rsa root@64.227.154.149
OpenSSH_for_Windows_8.6p1, LibreSSL 3.4.3
debug1: Authenticator provider $SSH_SK_PROVIDER did not resolve; disabling
debug1: Connecting to 64.227.154.149 [64.227.154.149] port 22.
debug1: Connection established.
debug1: identity file dp_rsa type -1
debug1: identity file dp_rsa-cert type -1
debug1: Local version string SSH-2.0-OpenSSH_for_Windows_8.6
debug1: Remote protocol version 2.0, remote software version OpenSSH_9.6p1 Ubuntu-3ubuntu13.4
debug1: compat_banner: match: OpenSSH_9.6p1 Ubuntu-3ubuntu13.4 pat OpenSSH* compat 0x04000000
debug1: Authenticating to 64.227.154.149:22 as 'root'
debug1: load_hostkeys: fopen C:\\Users\\zealv/.ssh/known_hosts2: No such file or directory
debug1: load_hostkeys: fopen __PROGRAMDATA__\\ssh/ssh_known_hosts: No such file or directory
debug1: load_hostkeys: fopen __PROGRAMDATA__\\ssh/ssh_known_hosts2: No such file or directory
debug1: SSH2_MSG_KEXINIT sent
debug1: SSH2_MSG_KEXINIT received
debug1: kex: algorithm: curve25519-sha256
debug1: kex: host key algorithm: ssh-ed25519
debug1: kex: server->client cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none
debug1: kex: client->server cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none
debug1: expecting SSH2_MSG_KEX_ECDH_REPLY
```

Debugging in Terraform

Similar to SSH Verbosity, even Terraform allows us to set wide variety of log levels for getting detailed logs for debugging purpose.

```
root@test-kplabs:~# terraform plan
2024-08-24T05:01:48.844Z [INFO]  Terraform version: 1.9.5
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/go-tfe v1.58.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/hcl/v2 v2.20.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/terraform-svchost v0.1.1
2024-08-24T05:01:48.845Z [DEBUG] using github.com/zclconf/go-cty v1.14.4
2024-08-24T05:01:48.845Z [INFO]  Go runtime version: go1.22.5
2024-08-24T05:01:48.845Z [INFO]  CLI args: []string{"terraform", "plan"}
2024-08-24T05:01:48.845Z [TRACE] Stdout is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stderr is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stdin is a terminal
2024-08-24T05:01:48.845Z [DEBUG] Attempting to open CLI config file: /root/.terraformrc
2024-08-24T05:01:48.845Z [DEBUG] File doesn't exist, but doesn't need to. Ignoring.
2024-08-24T05:01:48.845Z [DEBUG] ignoring non-existing provider search directory terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /var/lib/snapd/desktop/terraform/plugins
2024-08-24T05:01:48.848Z [INFO]  CLI command args: []string{"plan"}
```

Understanding the Basics

Terraform has detailed logs that you can enable by setting the TF_LOG environment variable to any value.

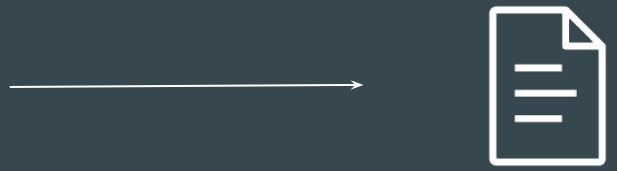
You can set TF_LOG to one of the log levels (in order of decreasing verbosity)

Log Level
TRACE
DEBUG
INFO
WARN
ERROR

Storing the Logs to File

To persist logged output you can set TF_LOG_PATH in order to force the log to always be appended to a specific file when logging is enabled

```
root@test-kplabs:~# terraform plan
2024-08-24T05:01:48.844Z [INFO]  Terraform version: 1.9.5
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/go-tfe v1.58.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/hcl/v2 v2.20.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/terraform-svchost v0.1.1
2024-08-24T05:01:48.845Z [DEBUG] using github.com/zclconf/go-cty v1.14.4
2024-08-24T05:01:48.845Z [INFO]  Go runtime version: go1.22.5
2024-08-24T05:01:48.845Z [INFO]  CLI args: []string{"terraform", "plan"}
2024-08-24T05:01:48.845Z [TRACE] Stdout is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stderr is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stdin is a terminal
2024-08-24T05:01:48.845Z [DEBUG] Attempting to open CLI config file: /root/.terraformrc
2024-08-24T05:01:48.845Z [DEBUG] File doesn't exist, but doesn't need to. Ignoring.
2024-08-24T05:01:48.845Z [DEBUG] ignoring non-existing provider search directory terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /var/lib/snapd/desktop/terraform/plugins
2024-08-24T05:01:48.848Z [INFO]  CLI command args: []string{"plan"}
```

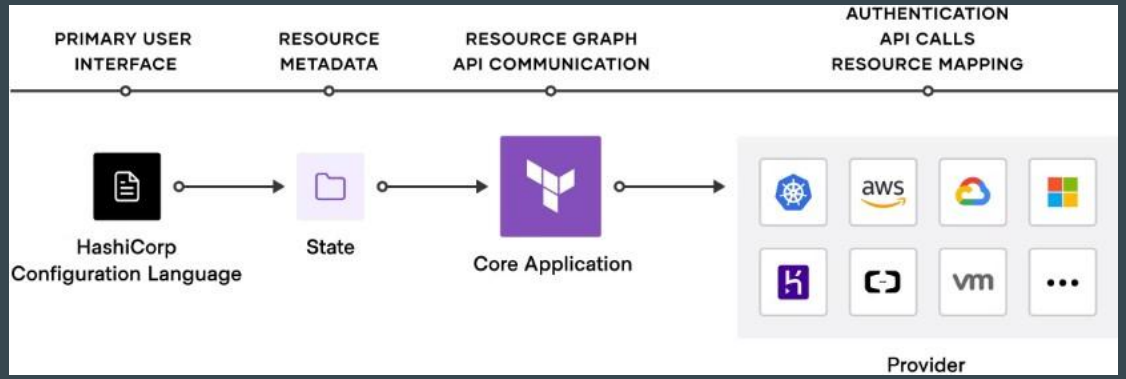


Terraform Troubleshooting Model

Terraform Troubleshooting Model

There are four potential types of issues that you could experience with Terraform

Language, State, Core, and Provider Errors.



1 - Language Errors

In most of the cases, the errors that you will face will be related to this.

When Terraform encounters a syntax error in your configuration, it prints out the line numbers and an explanation of the error.

```
C:\kplabs-terraform>terraform plan

  Error: Unclosed configuration block

    on demo.tf line 11, in resource "aws_iam_user" "dev":
    11: resource "aws_iam_user" "dev" {

  There is no closing brace for this block before the end of the file.
  elsewhere in this file.
```

2 - State Errors

If state is out of sync, Terraform may destroy or change your existing resources.

If state is locked, you will also be blocked from running write operations.

```
C:\kplabs-terraform>terraform plan

Error: Error acquiring the state lock

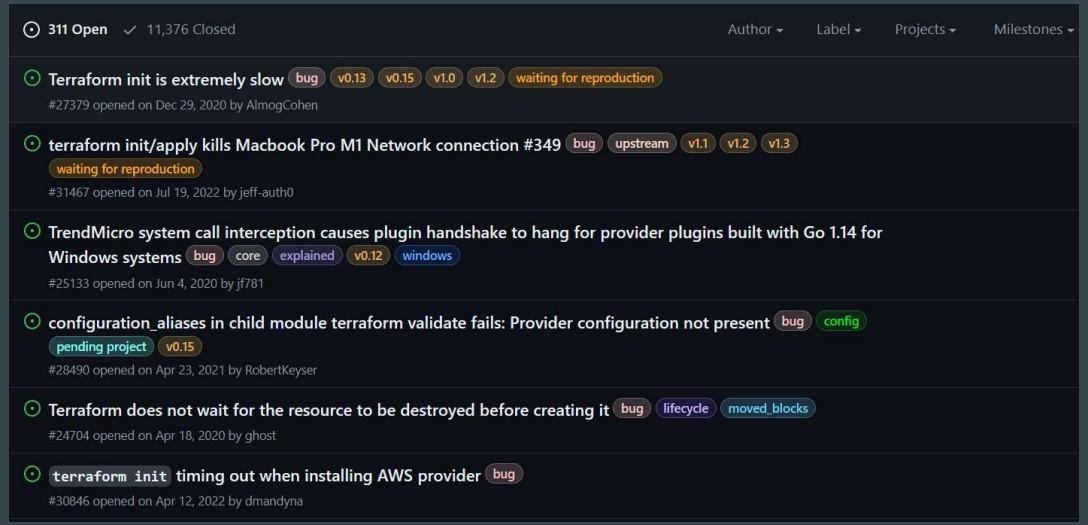
Error message: Failed to read state file: The state file could not be read: read terraform.tfstate
access the file because another process has locked a portion of the file.

Terraform acquires a state lock to protect the state from being written
by multiple users at the same time. Please resolve the issue above and try
again. For most commands, you can disable locking with the "-lock=false"
flag, but this is not recommended.
```

3 - Core errors

These errors are directly related to the main Terraform application.

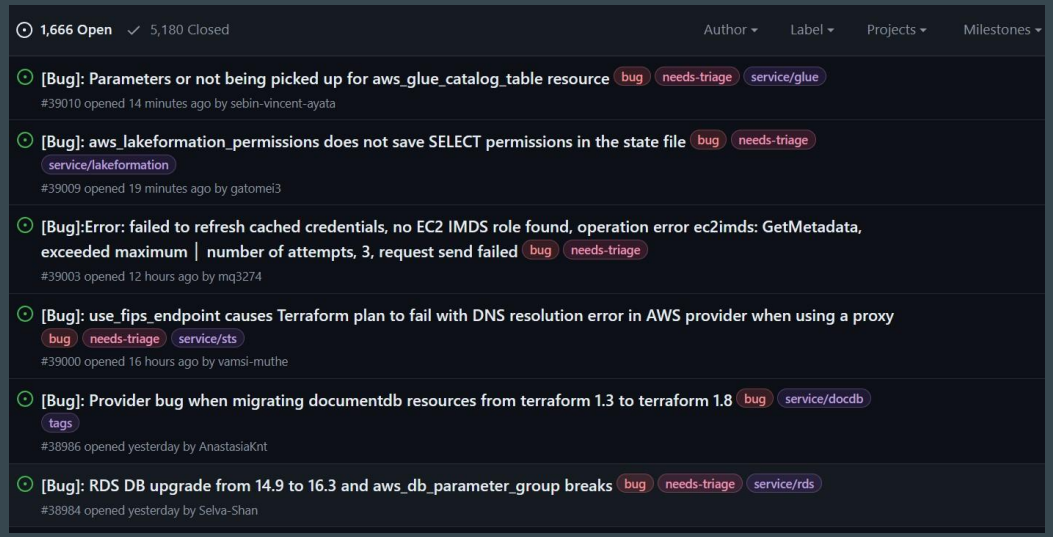
Errors produced at this level may be a bug.



4 - Provider errors

These set of errors are primarily related to the provider plugins.

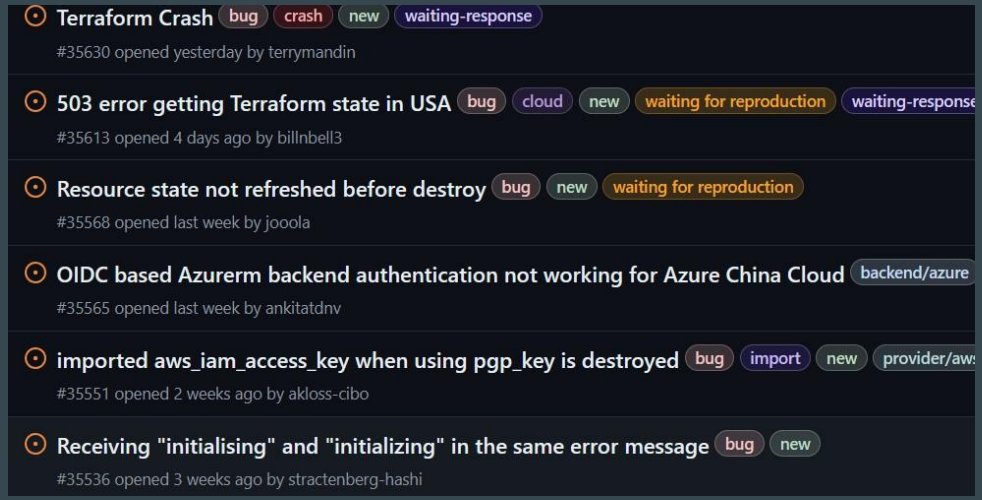
Use the Provider GitHub page for reporting and identifying the issue.



Reporting Terraform Bugs

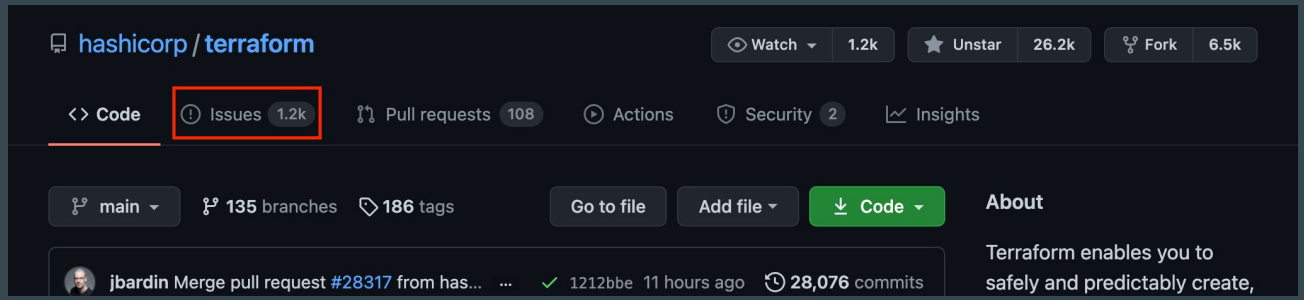
Reporting Bugs

You can report bugs in the Terraform Core GitHub page or appropriate provider page.

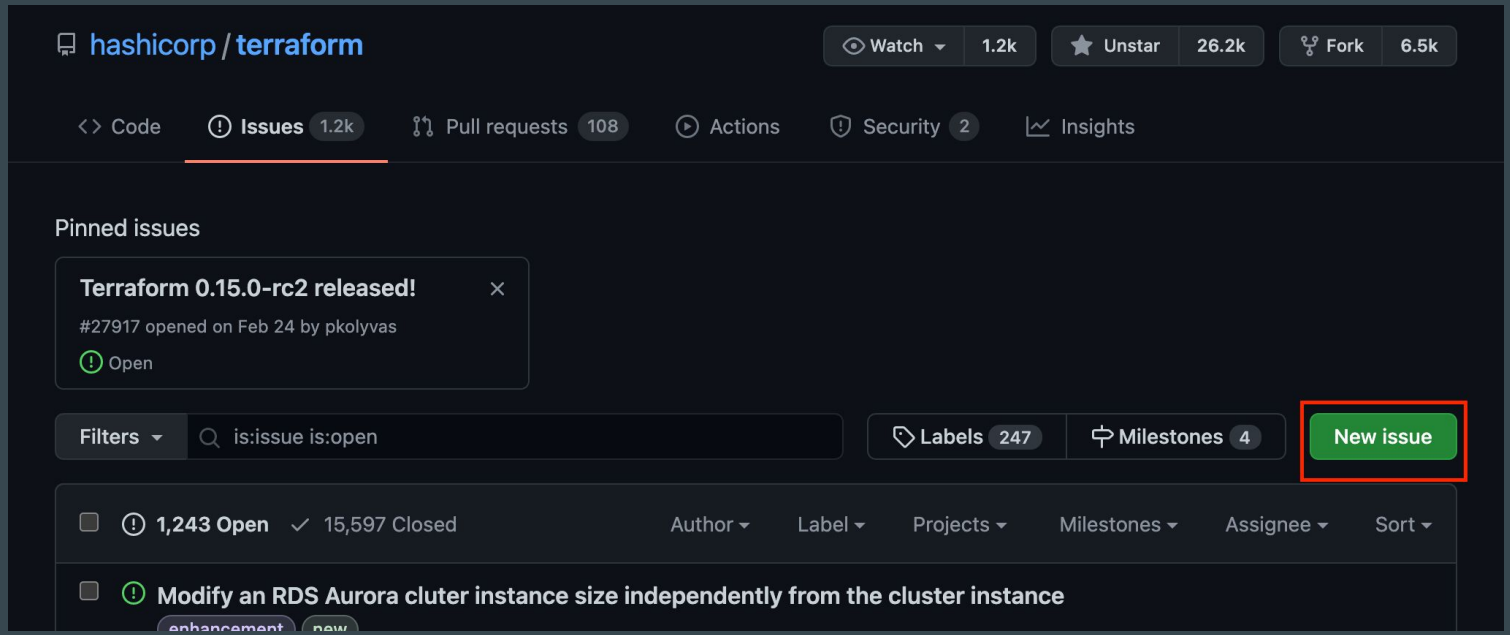


1 - Navigate to Issues

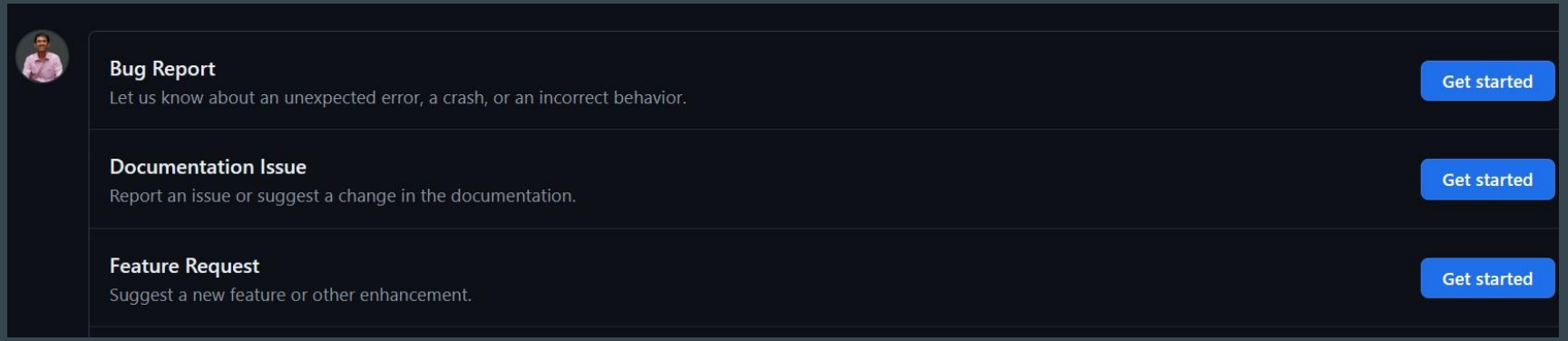
First, navigate to the Terraform GitHub repository and choose "Issues" from the top tabs.



2 - Choose "New Issue".



3 - Click “Get Started”



4 - Fill Core Terraform Template

Terraform Version *

Run `terraform version` to show the version, and paste the result below. If you are not running the latest version of Terraform, please try upgrading because your issue may have already been fixed.

```
...output of `terraform version`...
```

Terraform Configuration Files *

Paste the relevant parts of your Terraform configuration between the ``` marks below. For Terraform configs larger than a few resources, or that involve multiple files, please make a GitHub repository that we can clone, rather than copy-pasting multiple files in here.

```terraform
...terraform config...
```

Debug Output *

Full debug output can be obtained by running Terraform with the environment variable `TF_LOG=trace`. Please create a GitHub Gist containing the debug output. Please do *not* paste the debug output in the issue, since debug output is long. Debug output may contain sensitive information. Please review it before posting publicly.

...link to gist...

terraform fmt

Importance of Readability

When multiple engineers work on the same codebase, varying coding styles can make code reviews painful and debugging difficult.

The terraform fmt command formats Terraform configuration file contents so that it matches the canonical format and style



Before fmt

demo.tf > ...

```
resource "local_file" "foo" {
      content  = "foo!"
  filename = "${path.module}/demo.txt"
}
```

terraform fmt

After fmt

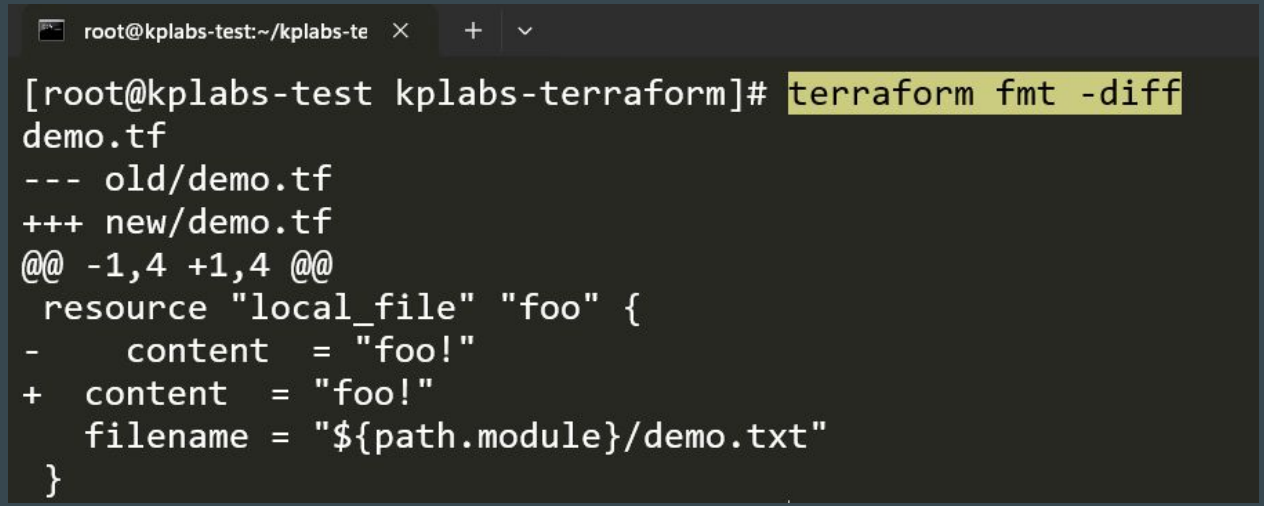
demo.tf > ...

```
resource "local_file" "foo" {
  content  = "foo!"
  filename = "${path.module}/demo.txt"
}
```

Visualizing Changes

If you want to see exactly what terraform fmt is changing, use the -diff flag.

This is helpful for understanding the specific style adjustments being applied.



Recursive Formatting

By default, terraform fmt only looks at the current directory.

If you have a complex project with subdirectories, use the -recursive flag.

```
C:\kplabs-terraform>terraform fmt -recursive
demo.tf
folder-1\01.tf
```

Check Mode (Dry Run)

In CI/CD pipelines, you often want to verify if the code is formatted correctly without actually modifying it.

The -check flag returns 0 if all files are formatted correctly and 3 if any files require formatting.

```
C:\kplabs-terraform>terraform fmt -check
demo.tf
```

Point to Note

terraform fmt does not change the behavior of your infrastructure; it only changes how the code looks.

Terraform - Load Order and Semantics

Setting the Base

Terraform normally loads all of the .tf and .tf.json files within a directory

The files loaded must end in either .tf or .tf.json



Point to Note

Since terraform loads all of the .tf and .tf.json files within a directory, it expects each one to define a distinct set of configuration objects. If two files attempt to define the same object, Terraform returns an error.

Example: The following two files are in same folder of “load-order”. It will result in error when terraform plan/apply operations run.

load-order > a_resource.tf > ...

```
resource "local_file" "file_one" {
  filename = "${path.module}/file_one.txt"
  content  = "I am File One."
}
```

a_resource.tf

load-order > c_resource.tf > ...

```
1
2  resource "local_file" "file_one" {
3    filename = "${path.module}/file_two.txt"
4    content  = "I am File two."
5  }
```

c_resource.tf

Point to Note

Terraform loads all configuration files within the directory specified in alphabetical order.

Terraform does not automatically read sub-directories unless explicitly called.

Organizing code into networking.tf, compute.tf is for humans, not Terraform.

Dynamic Blocks

Setting the Base

One of Terraform's powerful features for keeping your code DRY (Don't Repeat Yourself) is the Dynamic Block.

If you have ever find yourself copy-pasting the same configuration block (like ingress rules, etc) over and over again within a single resource, Dynamic Blocks are the solution.

```
resource "aws_security_group" "demo_sg" {
  name        = "sample-sg"

  ingress {
    from_port   = 8200
    to_port     = 8200
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

```
resource "aws_security_group" "demo_sg" {
  name        = "sample-sg"

  ingress {
    from_port   = 8300
    to_port     = 8300
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

Introducing Dynamic Blocks

A dynamic block allows you to generate these nested blocks programmatically based on a variable (list or map), rather than writing them out manually.

```
resource "aws_security_group" "dynamicsg" {
  name        = "dynamic-sg"

  dynamic "ingress" {
    for_each = var.sg_ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```



```
variable "sg_ports" {
  type        = list(number)
  default     = [8200, 8201,8300, 9200, 9500]
}
```

Terraform Validate

Terraform in detail

Overview of Terraform Validate

Terraform Validate primarily checks whether a configuration is syntactically valid.

It can check various aspects including unsupported arguments, undeclared variables and others.

```
resource "aws_instance" "myec2" {
  ami           = "ami-082b5a644766e0e6f"
  instance_type = "t2.micro"
  sky = "blue"
}
```



```
bash-4.2# terraform validate

Error: Unsupported argument

  on validate.tf line 10, in resource "aws_instance" "myec2":
  10:   sky = "blue"

An argument named "sky" is not expected here.
```

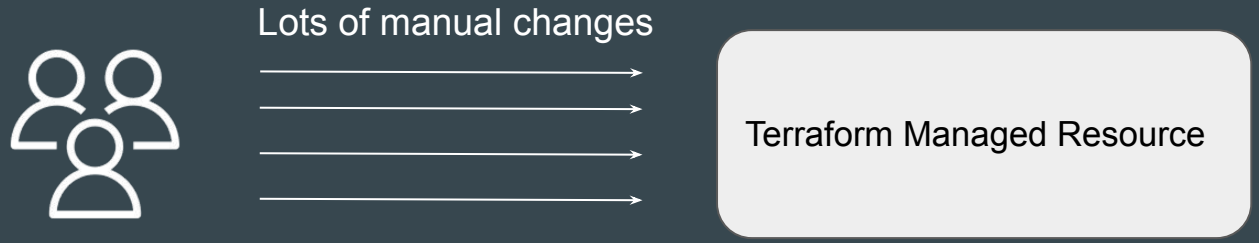
Terraform Taint

Understanding the Use-Case

You have created a new resource via Terraform.

Users have made a lot of manual changes (both infrastructure and inside the server)

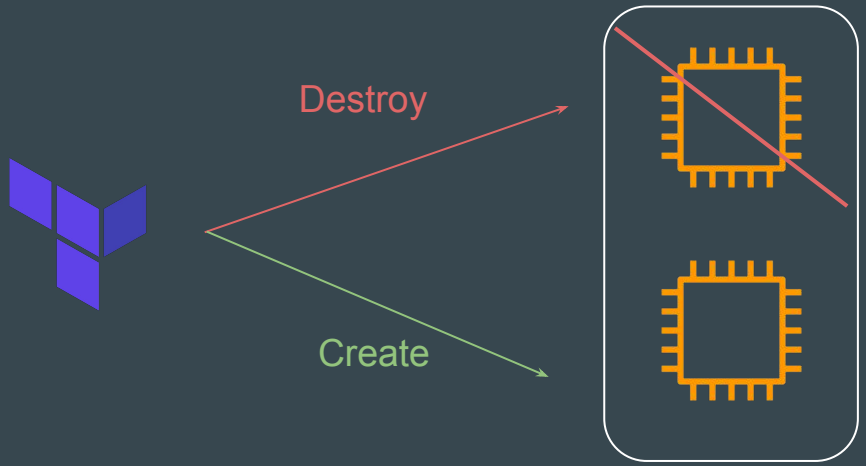
Two ways to deal with this: Import Changes to Terraform / Delete & Recreate the resource



Recreating the Resource

The -replace option with terraform apply to force Terraform to replace an object even though there are no configuration changes that would require it.

terraform apply -replace="aws_instance.web"



Points to Note

Similar kind of functionality was achieved using terraform taint command in older versions of Terraform.

For Terraform v0.15.2 and later, HashiCorp recommend using the -replace option with terraform apply

Splat Expression

Terraform Expressions

Overview of Spalat Expression

Splat Expression allows us to get a list of all the attributes.

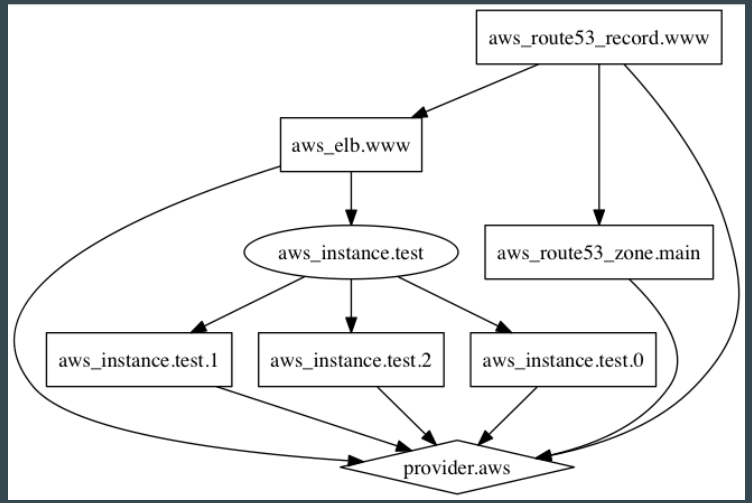
```
resource "aws_iam_user" "lb" {
  name = "iamuser.${count.index}"
  count = 3
  path = "/system/"
}

output "arns" {
  value = aws_iam_user.lb[*].arn
}
```

Terraform Graph

Understanding the Base Structure

Terraform graph refers to a visual representation of the dependency relationships between resources defined in your Terraform configuration.



Summary and Conclusion

Terraform graphs are a valuable tool for visualizing and understanding the relationships between resources in your infrastructure defined with Terraform.

It can improve your overall workflow by aiding in planning, debugging, and managing complex infrastructure configurations.

Saving Terraform Plan to File

Setting the Base

Terraform allows saving a plan to a file.

terraform plan -out ec2.plan


```
Saved the plan to: ec2.plan

To perform exactly these actions, run the following command to apply:
    terraform apply "ec2.plan"
```

Apply from Plan File

You can run the terraform apply by referencing the plan file.

This ensures the infrastructure state remains exactly as shown in the plan to ensure consistency.

```
C:\Users\zealv\kplabs-terraform>terraform apply infra.plan
aws_security_group.allow_tls: Creating...
aws_security_group.allow_tls: Creation complete after 4s [id=sg-02ed88b1d80a484a6]
aws_vpc_security_group_ingress_rule.app_port: Creating...
aws_vpc_security_group_ingress_rule.ssh_port: Creating...
aws_vpc_security_group_ingress_rule.ftp_port: Creating...
aws_vpc_security_group_ingress_rule.app_port: Creation complete after 1s [id=sgr-0b8f10164824c9027]
aws_vpc_security_group_ingress_rule.ssh_port: Creation complete after 1s [id=sgr-042688b669ba3d824]
aws_vpc_security_group_ingress_rule.ftp_port: Creation complete after 2s [id=sgr-0962b3406710b00ba]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

Exploring Terraform Plan File

The saved Terraform plan file will be a binary file.

You can use the terraform show command to read the contents in detail.


```
{
  "format_version": "1.2",
  "terraform_version": "1.9.1",
  "variables": {
    "app_port": {
      "value": "8080"
    },
    "ftp_port": {
      "value": "21"
    },
    "ssh_port": {
      "value": "22"
    },
    "vpn_ip": {
      "value": "200.20.30.50/32"
    }
  },
  "planned_values": {
    "root_module": {
      "resources": [
        {
          "address": "aws_security_group.allow_tls",
          "mode": "managed",
          "type": "aws_security_group",
          "name": "allow_tls",
          "provider_name": "registry.terraform.io/hashicorp/aws",
          "schema_version": 1,
          "values": {
            "description": "Managed from Terraform",
            "name": "terraform-firewall",
```

Use-Cases of Saving Plan to a File

Many organizations require documented proof of planned changes before implementation.

These changes will further be reviewed and approved.

Running apply from plan ensures consistent desired outcome.

Terraform Output

Terraform in detail

Terraform Output

The terraform output command is used to extract the value of an output variable from the state file.

```
C:\Users\Zeal Vora\Desktop\terraform\terraform output>terraform output iam_names
[
  "iamuser.0",
  "iamuser.1",
  "iamuser.2",
]
```

Terraform Settings

Setting the Base

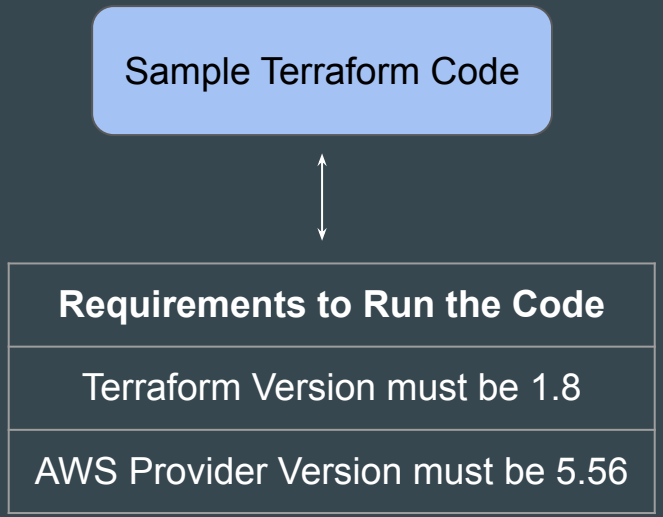
We can use the provider block to define various aspects of the provider, like region, credentials and so on.

```
provider "aws" {
  region     = "us-east-1"
  access_key = "AKIAIOSFODNN7EXAMPLE"
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
}

resource "aws_security_group" "sg_01" {
  name = "app_firewall"
}
```

Specific Version to Run Your Code

In a Terraform project, your code might require a very specific set of versions to run.



Introducing Terraform Settings

Terraform Settings are used to configure project-specific Terraform behaviors, such as requiring a minimum Terraform version to apply to your configuration.

Terraform settings are gathered together into terraform blocks:

```
terraform {
  # <setting-1>
  # <setting-2>
}
```

1 - Specifying a Required Terraform Version

If your code is compatible with specific versions of Terraform, you can use the required_version block to add your constraints.

```
terraform {
  required_version = "1.8"
}
```

2 - Specifying Provider Requirements

The required_providers block can be used to specify all of the providers required by your Terraform code.

You can further fine-tune to include a specific version of the provider plugins.

```
terraform {
  required_providers {
    aws = {
      version = "5.56"
      source = "hashicorp/aws"
    }
  }
}
```

Flexibility in Settings Block

There are a wide variety of options that can be specified in the Terraform block.

terraform {....}

Options That Can be Defined
Required Terraform Version
Required Provider and Version
BackEnd Configuration
Experimental Features

Point to Note

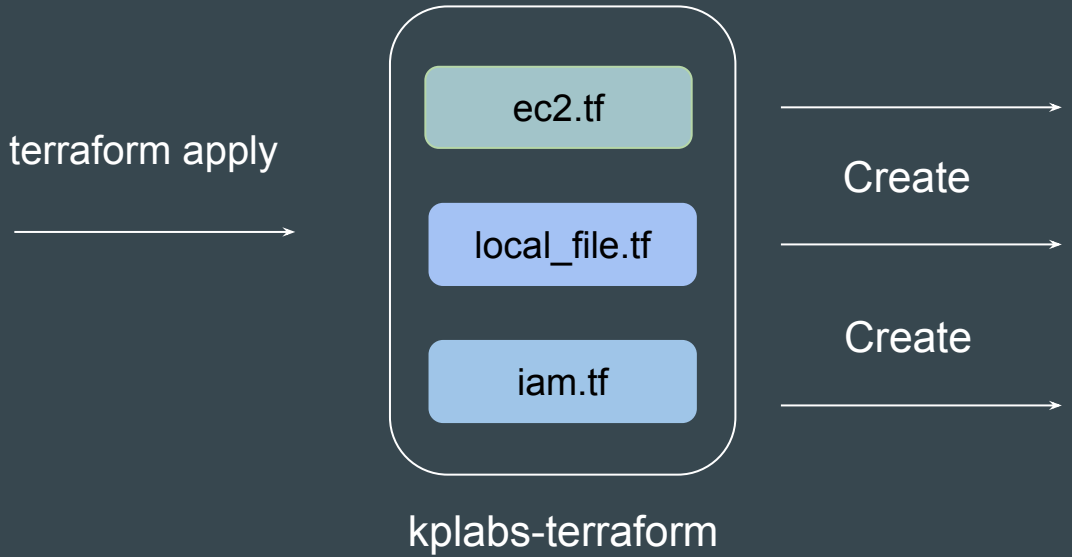
It is a good practice to include the terraform { } block to include details like required_providers as part of your project.

The provider { } block is still important to specify various other aspects like regions, credentials, alias and others.

Resource Targeting

Setting the Base

In a typical Terraform workflow, you apply the entire plan at once. This is also the default behaviour.



Understanding Resource Targeting

Resource targeting in Terraform allows you to apply changes to a specific subset of resources rather than applying changes to your entire infrastructure.

terraform plan <local_file>



```
resource "aws_iam_user" "dev" {
  name = "kplabs-user-01"
}

resource "aws_security_group" "prod" {
  name        = "terraform-firewalls"
}

resource "local_file" "foo" {
  content  = "foo!"
  filename = "${path.module}/foo.txt"
}
```

Using the -target flag

You can use Terraform's -target option to target specific resources, modules as part of operation.

terraform plan -target local_file.foo



```
resource "aws_iam_user" "dev" {
  name = "kplabs-user-01"
}

resource "aws_security_group" "prod" {
  name        = "terraform-firewalls"
}

resource "local_file" "foo" {
  content  = "foo!"
  filename = "${path.module}/foo.txt"
}
```

Use-Cases

A project has 10 resources. Multiple team members are working on each resource's Terraform code to release some updates.

When you run terraform plan, you see Terraform trying to make changes to all 10 resources.

There is a critical issue and you need to add a port 80 in security group that is managed via terraform without having to update other 9 resources.

Points to Note

Occasionally you may want to apply only part of a plan, such as when Terraform's state has become out of sync with your resources due to a network failure, a problem with the upstream cloud platform, or a bug in Terraform or its providers.

To support this, Terraform lets you target specific resources when you plan, apply, or destroy your infrastructure.

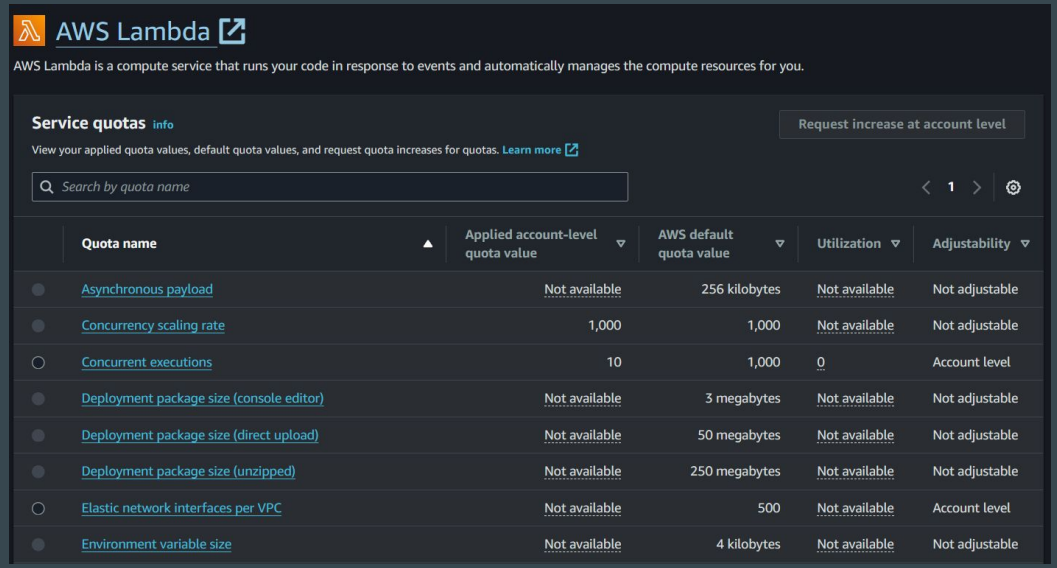
Targeting individual resources can be useful for troubleshooting errors, but should not be part of your normal workflow.

Dealing with Larger Infrastructure

Cloud does not mean Unlimited

Using a Cloud provider does not mean you have access to unlimited resources.

There are some limits that apply to all of the services.



AWS Lambda

AWS Lambda is a compute service that runs your code in response to events and automatically manages the compute resources for you.

Service quotas info

Request increase at account level

View your applied quota values, default quota values, and request quota increases for quotas. Learn more

Search by quota name

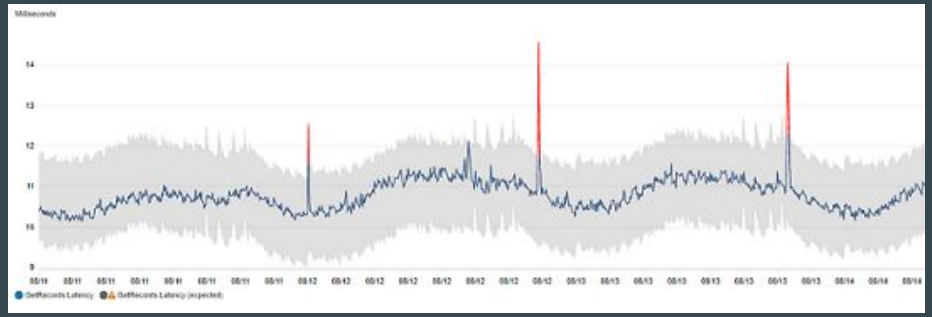
1

Quota name	Applied account-level quota value	AWS default quota value	Utilization	Adjustability
Asynchronous payload	Not available	256 kilobytes	Not available	Not adjustable
Concurrency scaling rate	1,000	1,000	Not available	Not adjustable
Concurrent executions	10	1,000	0	Account level
Deployment package size (console editor)	Not available	3 megabytes	Not available	Not adjustable
Deployment package size (direct upload)	Not available	50 megabytes	Not available	Not adjustable
Deployment package size (unzipped)	Not available	250 megabytes	Not available	Not adjustable
Elastic network interfaces per VPC	Not available	500	Not available	Account level
Environment variable size	Not available	4 kilobytes	Not available	Not adjustable

Less Known Limit - API Limit in AWS

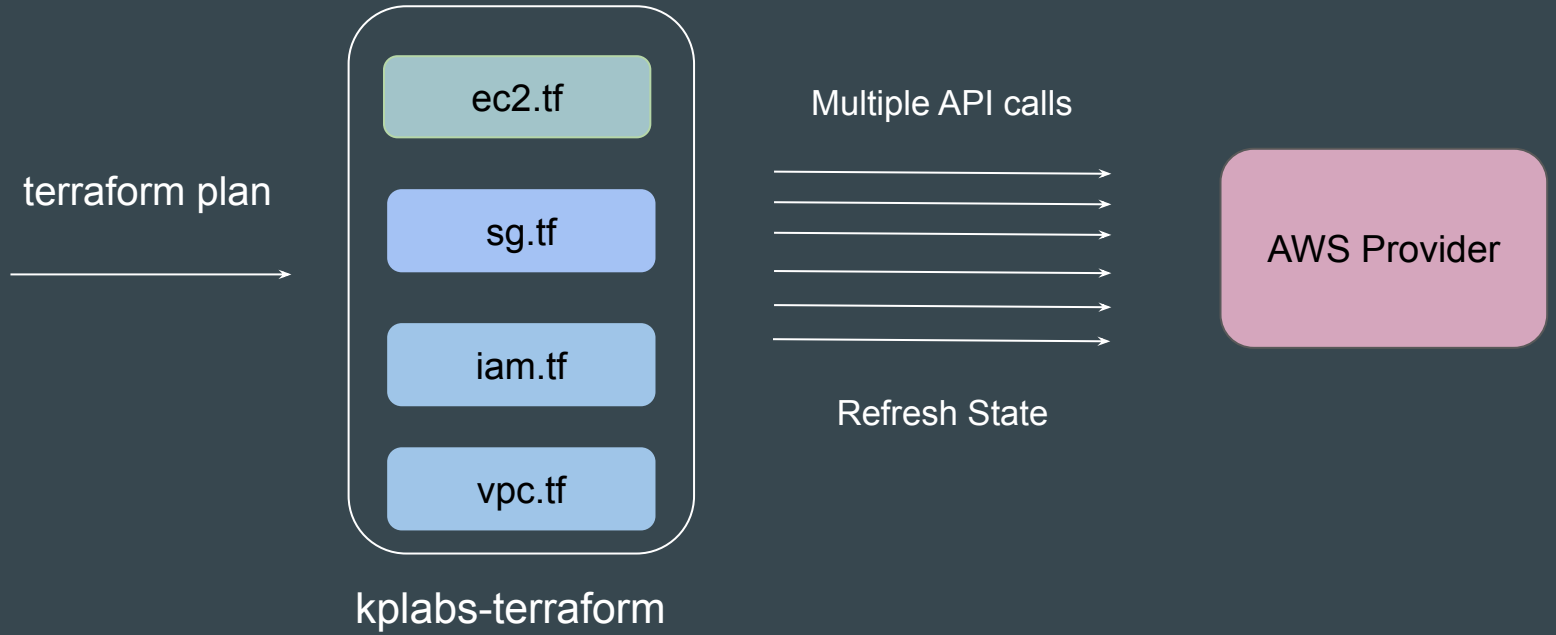
When you're architecting for the cloud, you need to keep API throttling in mind, particularly the types of calls and the frequency with which they are called.

When the allotted rate limit for an API call is exceeded, you'll receive an error response and the call will be throttled.



Key Consideration in Terraform

When a single project has large set of resources that are managed, even a simple terraform plan can lead to higher number of API calls to the provider.



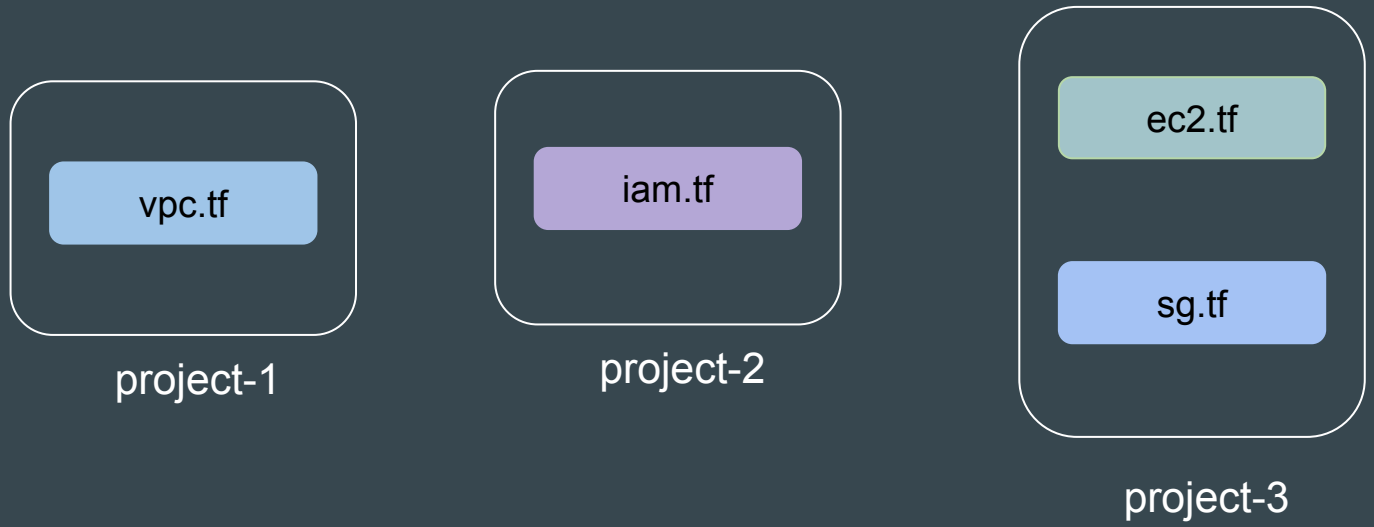
Time to Slow Down

Whenever you are designing Terraform projects, you have to take in mind the API limits of the provider to avoid throttling that will further impact production systems.



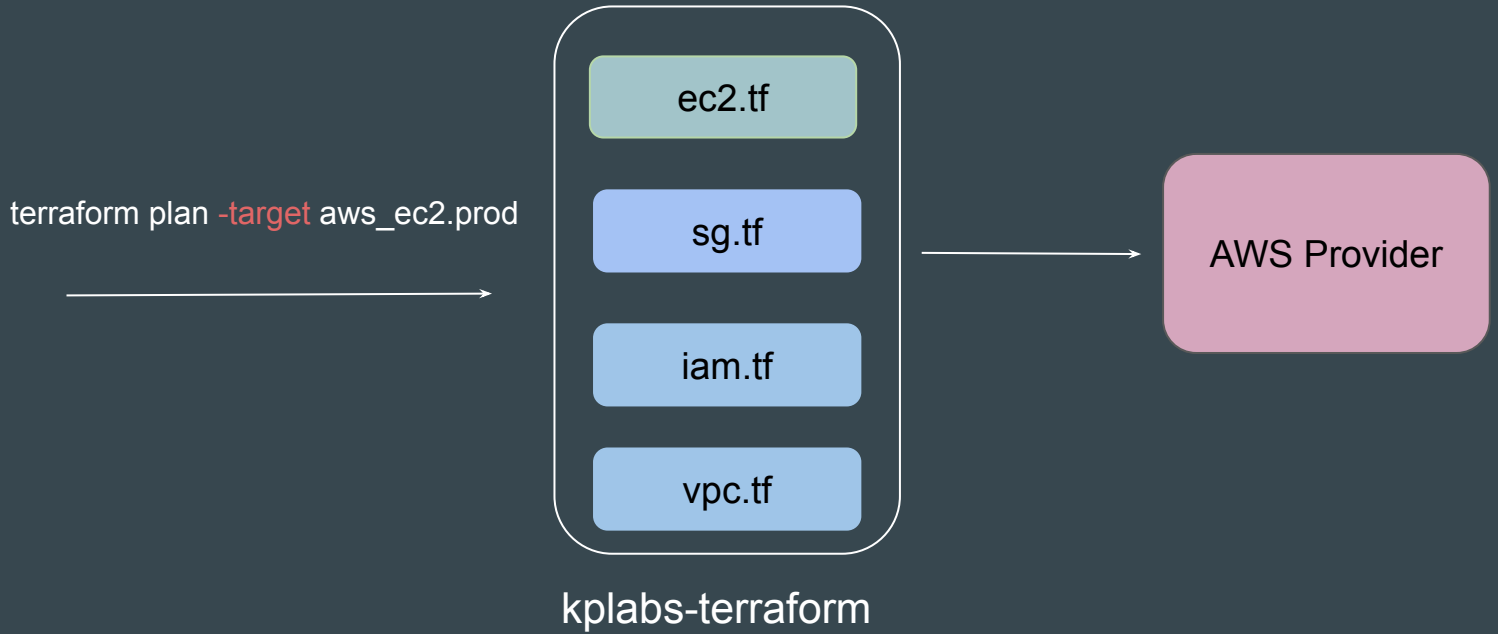
1 - Convert Big Project to Multiple Smaller Project

Whenever you are designing Terraform projects, you have to take in mind the API limits of the provider to avoid throttling that will further impact production systems.



2 - Use Resource Targeting

Using resource targeting, the number of calls that will be made to a provider will be much less.



3 - Setting refresh=false

The -refresh=false option is used in normal planning mode to skip the default behavior of refreshing Terraform state before checking for configuration changes.

By disabling automatic refresh, you can potentially speed up the planning and application process, especially if you're confident that the existing state is accurate.

```
C:\kplabs-terraform\larger-infra>terraform plan -refresh=false

Terraform used the selected providers to generate the following execution plan.
following symbols:
  + create

Terraform will perform the following actions:

  # aws_security_group.allow_tl will be created
  + resource "aws_security_group" "allow_tl" {
      + arn                    = (known after apply)
      + description            = "Managed from Terraform"
      + egress                 = (known after apply)
      + id                     = (known after apply)
      + ingress                = (known after apply)
      + name                   = "test-firewall"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + revoke_rules_on_delete = false
      + tags_all               = (known after apply)
      + vpc_id                 = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Reference Screenshot

```
C:\kplabs-terraform\larger-infra>terraform plan
aws_security_group.allow_tls: Refreshing state... [id=sg-0f6ab960cd5974e3d]
module.vpc.aws_vpc.this[0]: Refreshing state... [id=vpc-0ab5b8b88189cc37f]
aws_vpc_security_group_ingress_rule.allow_tls_ipv6: Refreshing state... [id=sgr-02ba31b9105691dd4]
aws_vpc_security_group_egress_rule.allow_all_traffic_ipv4: Refreshing state... [id=sgr-0684d5de21661e90a]
module.vpc.aws_default_security_group.this[0]: Refreshing state... [id=sg-0a4a6df13518ee751]
module.vpc.aws_default_route_table.default[0]: Refreshing state... [id=rtb-0469b753720e5d5c3]
module.vpc.aws_subnet.public[2]: Refreshing state... [id=subnet-0cce48829889218ba]
module.vpc.aws_subnet.private[2]: Refreshing state... [id=subnet-0c6ffa119e73801cf]
module.vpc.aws_subnet.private[1]: Refreshing state... [id=subnet-0b3710a0a30a92e61]
module.vpc.aws_subnet.public[1]: Refreshing state... [id=subnet-0ef80e31f37e114fc]
module.vpc.aws_subnet.private[0]: Refreshing state... [id=subnet-030725dbbfadbffec]
module.vpc.aws_internet_gateway.this[0]: Refreshing state... [id=igw-07acd64321b4c86ee]
module.vpc.aws_subnet.public[0]: Refreshing state... [id=subnet-079ea1df9674ab8e9]
module.vpc.aws_default_network_acl.this[0]: Refreshing state... [id=acl-0af44b9fd2dcd2899]
module.vpc.aws_route_table.private[2]: Refreshing state... [id=rtb-0bc2f3592aeed257d]
module.vpc.aws_route_table.private[0]: Refreshing state... [id=rtb-0ee65dcb6f2ba2749]
module.vpc.aws_route_table.private[1]: Refreshing state... [id=rtb-0e0317617030fd2eb]
module.vpc.aws_vpn_gateway.this[0]: Refreshing state... [id=vgw-056912b1df8e05812]
module.vpc.aws_route_table.public[0]: Refreshing state... [id=rtb-09e5cb13d415c2b37]
module.vpc.aws_eip.nat[1]: Refreshing state... [id=eipalloc-0893b740cbd72963c]
module.vpc.aws_eip.nat[2]: Refreshing state... [id=eipalloc-0eb138eaa99086e42]
module.vpc.aws_eip.nat[0]: Refreshing state... [id=eipalloc-0940cf568ed9a4e4a]
module.vpc.aws_route_table_association.private[1]: Refreshing state... [id=rtbassoc-064613d9444a71f46]
module.vpc.aws_route_table_association.private[0]: Refreshing state... [id=rtbassoc-0d5464665a321b560]
module.vpc.aws_route_table_association.private[2]: Refreshing state... [id=rtbassoc-075364cdbf6f869d6]
module.vpc.aws_route_table_association.public[2]: Refreshing state... [id=rtbassoc-0e4962322428456cd]
module.vpc.aws_route_table_association.public[1]: Refreshing state... [id=rtbassoc-0663f72daafec7542]
module.vpc.aws_route_table_association.public[0]: Refreshing state... [id=rtbassoc-00232967b2084e45f]
module.vpc.aws_route.public_internet_gateway[0]: Refreshing state... [id=r-rtb-09e5cb13d415c2b371080289494]
```

terraform plan

```
C:\kplabs-terraform\larger-infra>terraform plan -refresh=false

Terraform used the selected providers to generate the following execution plan.
following symbols:
  + create

Terraform will perform the following actions:

  # aws_security_group.allow_tl will be created
  + resource "aws_security_group" "allow_tl" {
      + arn                    = (known after apply)
      + description            = "Managed from Terraform"
      + egress                 = (known after apply)
      + id                     = (known after apply)
      + ingress                = (known after apply)
      + name                   = "test-firewall"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + revoke_rules_on_delete = false
      + tags_all               = (known after apply)
      + vpc_id                 = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

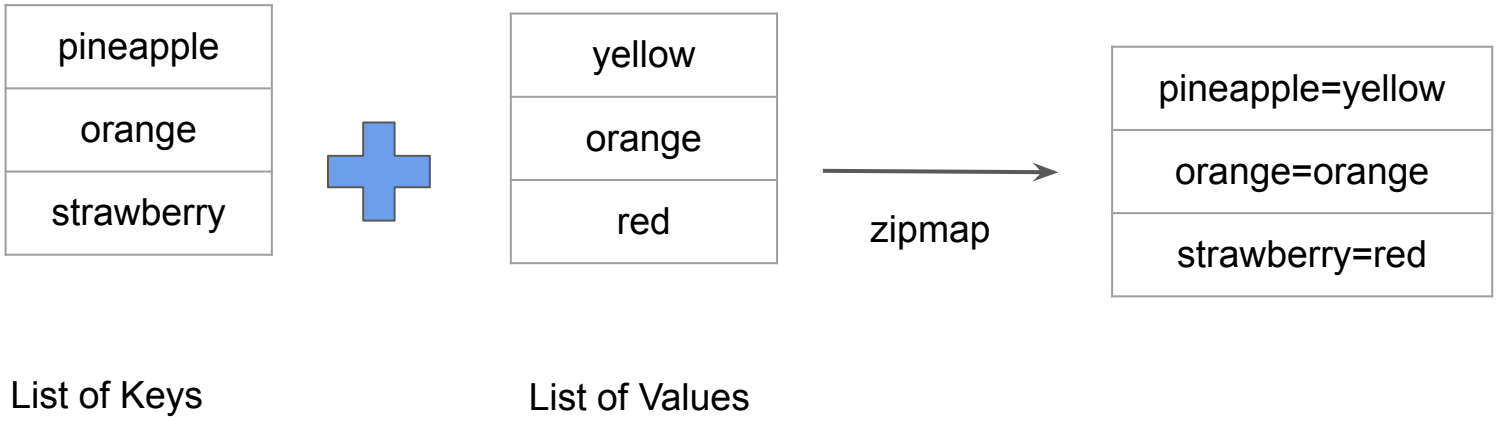
terraform plan -refresh=false

Zipmap

Terraform Function

Overview of Zipmap

The zipmap function constructs a map from a list of keys and a corresponding list of values.



Sample Output of Zipmap Function

```
←[J> zipmap["pineapple","oranges","strawberry"], ["yellow","orange","red"])
{
  "oranges" = "orange"
  "pineapple" = "yellow"
  "strawberry" = "red"
}
```

Simple Use-Case

You are creating multiple IAM users.

You need output which contains direct mapping of IAM names and ARNs

```
zipmap = {
  "demo-user.0" = "arn:aws:iam::018721151861:user/system/demo-user.0"
  "demo-user.1" = "arn:aws:iam::018721151861:user/system/demo-user.1"
  "demo-user.2" = "arn:aws:iam::018721151861:user/system/demo-user.2"
}
```

Comments in Terraform Code

Commenting the Code!

Overview of Comments

A comment is a text note added to source code to provide explanatory information, usually about the function of the code

```
'''In this program, we check if the number is positive or
negative or zero and
display an appropriate message'''

num = 3.4

# Try these two variations as well:
# num = 0
# num = -4.5

if num > 0:
    print("Positive number")
elif num == 0:
    print("Zero")
else:
    print("Negative number")
```

Comments in Terraform

The Terraform language supports three different syntaxes for comments:

Type	Description
#	begins a single-line comment, ending at the end of the line.
//	also begins a single-line comment, as an alternative to #.
/* and */	are start and end delimiters for a comment that might span over multiple lines.

Resource Behavior and Meta-Argument

Understanding the Basics

A resource block declares that you want a particular infrastructure object to exist with the given settings

```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
}
```

How Terraform Applies a Configuration

Create resources that exist in the configuration but are not associated with a real infrastructure object in the state.

Destroy resources that exist in the state but no longer exist in the configuration.

Update in-place resources whose arguments have changed.

Destroy and re-create resources whose arguments have changed but which cannot be updated in-place due to remote API limitations.

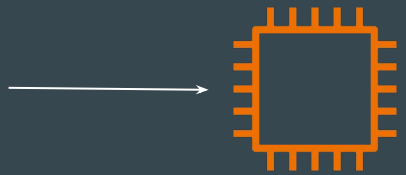
Understanding the Limitations

What happens if we want to change the default behavior?

Example: Some modification happened in Real Infrastructure object that is not part of Terraform but you want to ignore those changes during terraform apply.

```
resource "aws_instance" "web" {
  ami           = "ami-00c39f71452c08778"
  instance_type = "t3.micro"

  tags = {
    Name = "HelloWorld"
  }
}
```



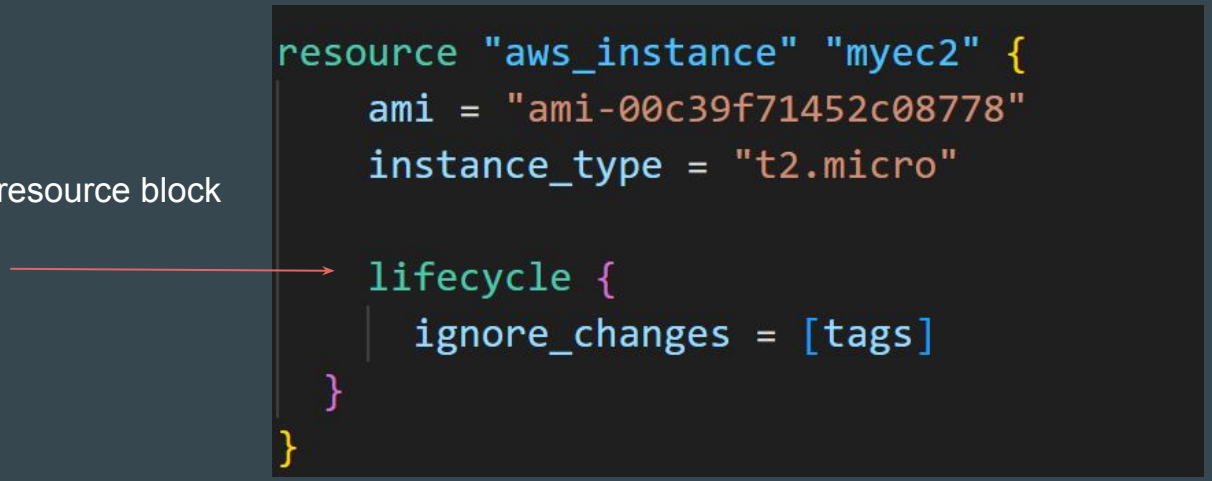
Name	HelloWorld

Env	Production

Solution - Using Meta Arguments

Terraform allows us to include meta-argument within the resource block which allows some details of this standard resource behavior to be customized on a per-resource basis.

Inside resource block



Different Meta-Arguments

Meta-Argument	Description
depends_on	Handle hidden resource or module dependencies that Terraform cannot automatically infer.
count	Accepts a whole number, and creates that many instances of the resource
for_each	Accepts a map or a set of strings, and creates an instance for each item in that map or set.
lifecycle	Allows modification of the resource lifecycle.
provider	Specifies which provider configuration to use for a resource, overriding Terraform's default behavior of selecting one based on the resource type name

Meta Argument - LifeCycle

Basics of Lifecycle Meta-Argument

Some details of the default resource behavior can be customized using the special nested lifecycle block within a resource block body:

```
resource "aws_instance" "myec2" {
    ami = "ami-0f34c5ae932e6f0e4"
    instance_type = "t2.micro"

    tags = {
        Name = "HelloEarth"
    }

    lifecycle {
        ignore_changes = [tags]
    }
}
```

Arguments Available

There are four argument available within lifecycle block.

Arguments	Description
create_before_destroy	New replacement object is created first, and the prior object is destroyed after the replacement is created.
prevent_destroy	Terraform to reject with an error any plan that would destroy the infrastructure object associated with the resource
ignore_changes	Ignore certain changes to the live resource that does not match the configuration.
replace_triggered_by	Replaces the resource when any of the referenced items change

Replace Triggered By

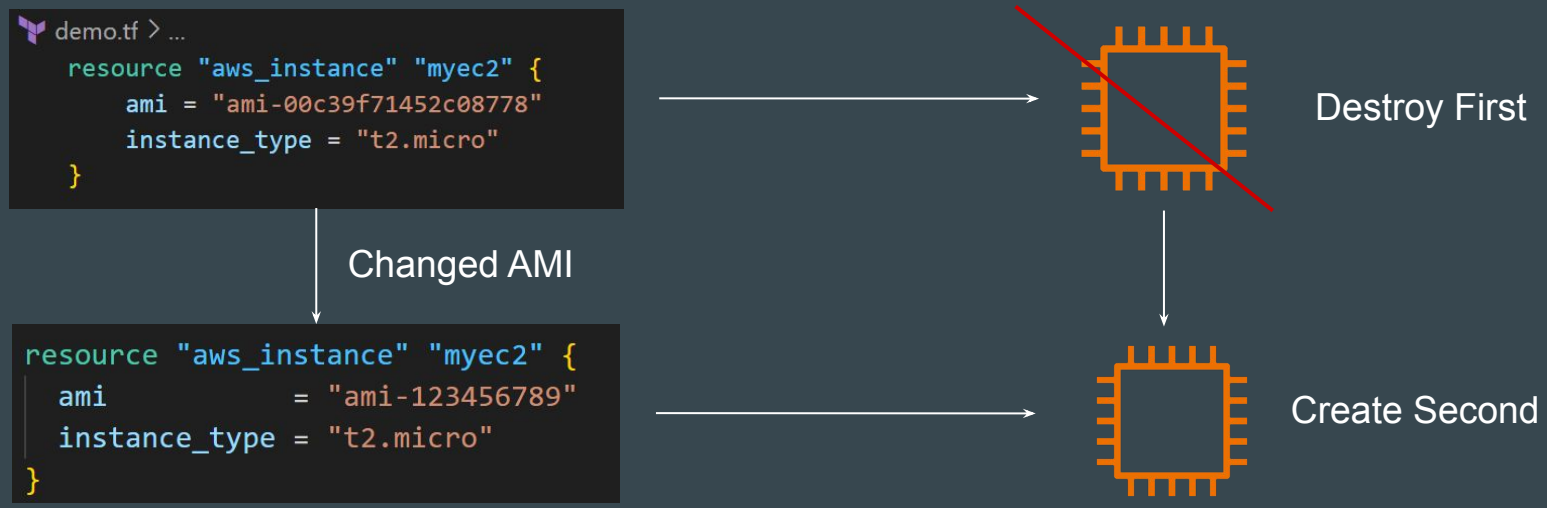
Replaces the resource when any of the referenced items change.

```
resource "aws_appautoscaling_target" "ecs_target" {
  # ...
  lifecycle {
    replace_triggered_by = [
      # Replace `aws_appautoscaling_target` each time this instance of
      # the `aws_ecs_service` is replaced.
      aws_ecs_service.svc.id
    ]
  }
}
```

Create Before Destroy Argument

Understanding the Default Behavior

By default, when Terraform must change a resource argument that cannot be updated in-place due to remote API limitations, Terraform will instead destroy the existing object and then create a new replacement object with the new configured arguments.



Create Before Destroy Argument

The create_before_destroy meta-argument changes this behavior so that the new replacement object is created first, and the prior object is destroyed after the replacement is created.

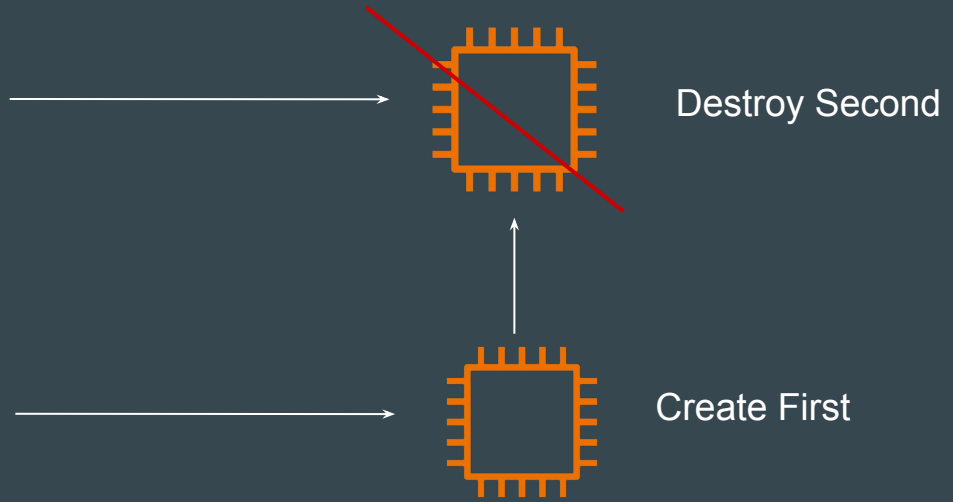
```
resource "aws_instance" "myec2" {
    ami = "ami-053b0d53c279acc90"
    instance_type = "t2.micro"

    lifecycle {
      create_before_destroy = true
  }
}
```

Changed AMI

```
resource "aws_instance" "myec2" {
    ami = "ami-123456789"
    instance_type = "t2.micro"

    lifecycle {
      create_before_destroy = true
  }
}
```



Join us in our Adventure

Be Awesome



kplabs.in/chat



kplabs.in/linkedin

LifeCycle - Prevent Destroy Argument

Prevent Destroy Argument

This meta-argument, when set to true, will cause Terraform to reject with an error any plan that would destroy the infrastructure object associated with the resource, as long as the argument remains present in the configuration.

```
resource "aws_instance" "myec2" {
  ami           = "ami-123456789"
  instance_type = "t2.micro"

  lifecycle {
   prevent_destroy = true
 }
}
```

Points to Note

This can be used as a measure of safety against the accidental replacement of objects that may be costly to reproduce, such as database instances.

Since this argument must be present in configuration for the protection to apply, note that this setting does not prevent the remote object from being destroyed if the resource block were removed from configuration entirely.

LifeCycle - Ignore Changes Argument

Ignore Changes

In cases where settings of a remote object is modified by processes outside of Terraform, the Terraform would attempt to "fix" on the next run.

In order to change this behavior and ignore the manually applied change, we can make use of ignore_changes argument under lifecycle.

```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"

    lifecycle {
      ignore_changes = [tags]
  }
}
```

Points to Note

Instead of a list, the special keyword all may be used to instruct Terraform to ignore all attributes, which means that Terraform can create and destroy the remote object but will never propose updates to it.

```
resource "aws_instance" "myec2" {
    ami = "ami-0f34c5ae932e6f0e4"
    instance_type = "t2.micro"

    tags = {
        Name = "HelloEarth"
    }

    lifecycle {
        ignore_changes = all
    }
}
```

Challenges with Count

Meta-Argument

Revising the Basics

Resource are identified by the index value from the list.

```
variable "iam_names" {
  type = list
  default = ["user-01","user-02","user-03"]
}

resource "aws_iam_user" "iam" {
  name = var.iam_names[count.index]
  count = 3
  path = "/system/"
}
```



Resource Address	Infrastructure
aws_iam_user.iam[0]	user-01
aws_iam_user.iam[1]	user-02
aws_iam_user.iam[2]	user-03

Challenge - 1

If the order of elements of index is changed, this can impact all of the other resources.

```
variable "iam_names" {
  type = list
  default = ["user-0","user-01","user-02","user-03"]
}

resource "aws_iam_user" "iam" {
  name = var.iam_names[count.index]
  count = 4
  path = "/system/"
}
```


Resource Address	Infrastructure
aws_iam_user.iam.[0]	user-01
aws_iam_user.iam.[1]	user-02
aws_iam_user.iam.[2]	user-03

Important Note

If your resources are almost identical, count is appropriate.

If distinctive values are needed in the arguments, usage of for_each is recommended.

```
resource "aws_instance" "server" {
  count = 4 # create four similar EC2 instances
  ami           = "ami-a1b2c3d4"
  instance_type = "t2.micro"
}
```

Resource Dependencies

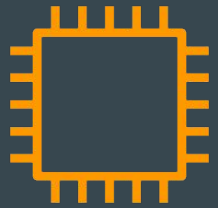
Sample Requirement

An application deployed on an EC2 instance must store its data in external storage before it can initialize.

Resources Needed: S3 Bucket (for external storage) and EC2 Instance



Step 1: Create S3 Bucket



Step 2: Create EC2 Instance

Understanding the Challenge

In a scenario where resources are defined independently, there is no guaranteed order stating which resource will be created first.

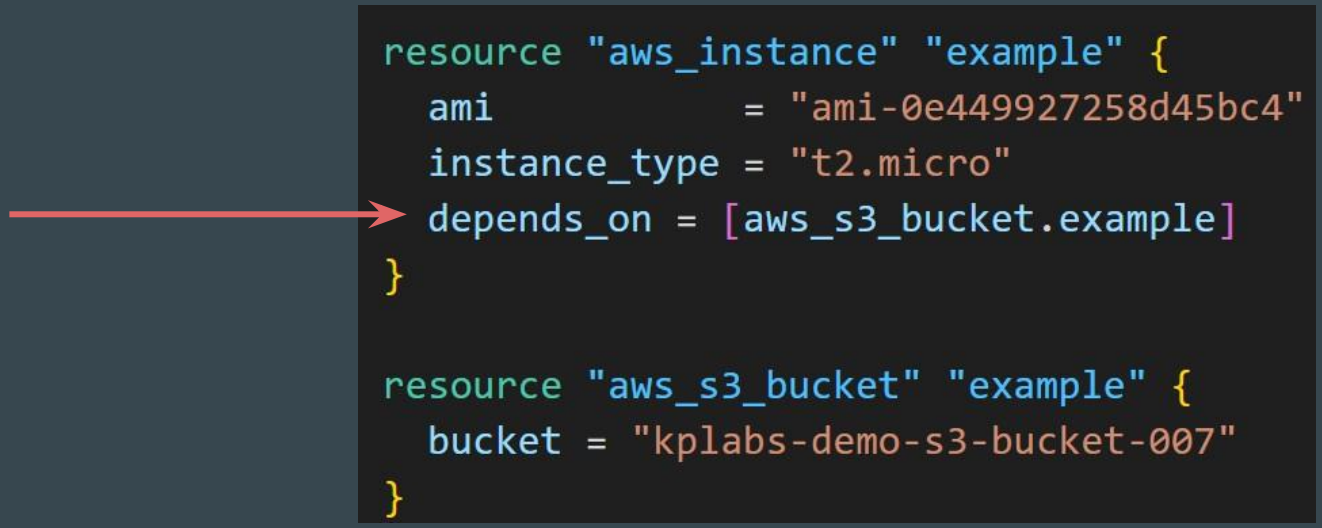
Sometimes the EC2 instance may be created first, and sometimes the S3 bucket may be created first.

```
resource "aws_instance" "example" {
  ami           = "ami-0e449927258d45bc4"
  instance_type = "t2.micro"
}

resource "aws_s3_bucket" "example" {
  bucket = "kplabs-demo-s3-bucket-007"
}
```

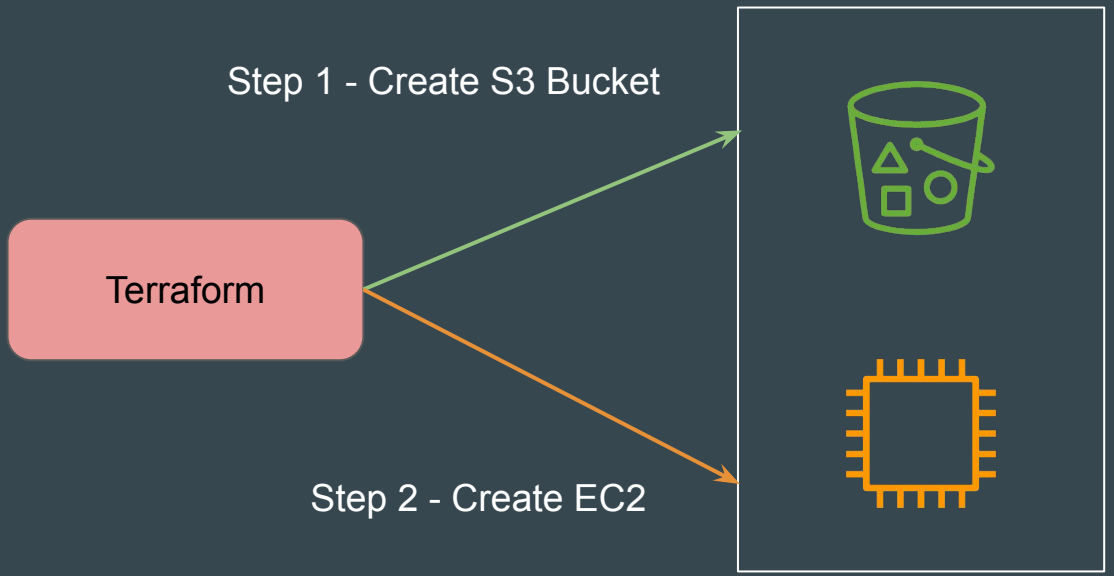
Introducing depends_on Meta Argument

The depends_on meta-argument instructs Terraform to complete all actions on the dependency object before performing actions on the object declaring the dependency.



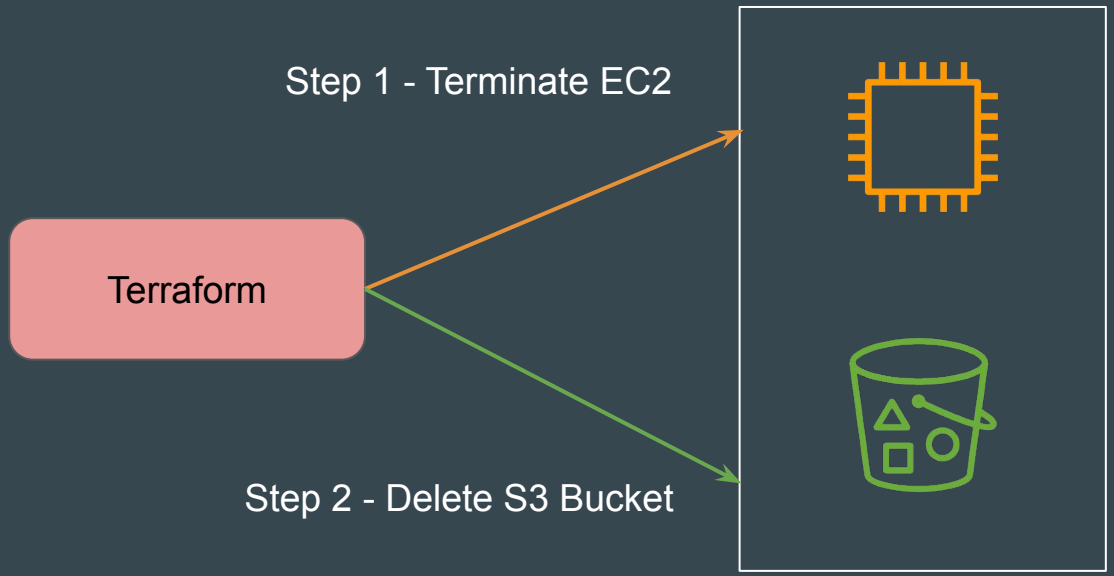
Order of Creation

The depends_on meta-argument explicitly tells Terraform that the aws_instance.example must be created after aws_s3_bucket.example.



Order of Deletion

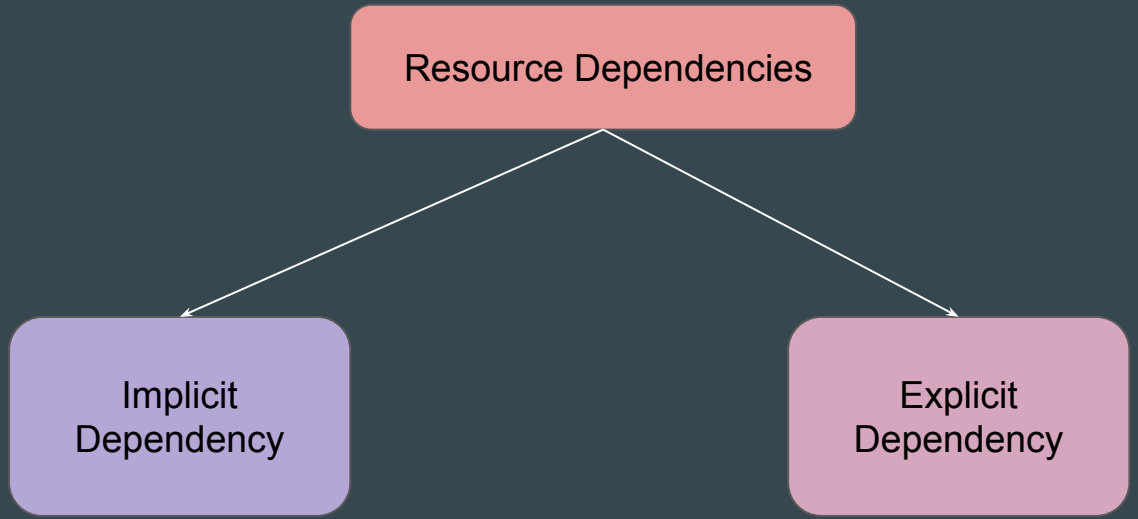
When you run terraform destroy, the order is reversed to ensure dependencies are not broken during deletion.



Implicit vs Explicit Dependencies

Setting the Base

There are two ways to define dependencies in Terraform.



Explicit Dependency

Explicit dependencies are declared using the depends_on meta-argument.

You use this when there's no direct attribute reference, but you still need to control the order of resource creation.

```
resource "aws_instance" "example" {
  ami           = "ami-0e449927258d45bc4"
  instance_type = "t2.micro"
  depends_on = [aws_s3_bucket.example]
}

resource "aws_s3_bucket" "example" {
  bucket = "kplabs-demo-s3-bucket-007"
}
```

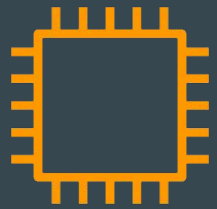
Sample Requirement

EC2 instance should only allow communication from trusted set of IP addresses.

Resources Needed: EC2 Instance + Security Group (Firewall)



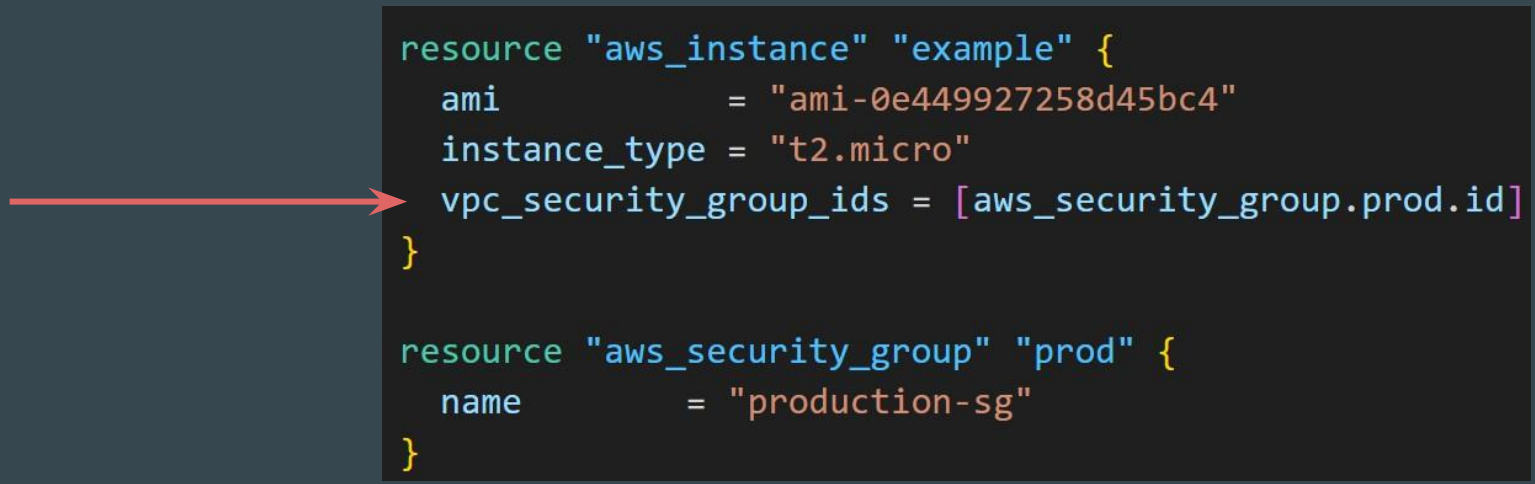
Step 1: Create Security Group



Step 2: Create EC2 Instance

Introducing Implicit Dependency

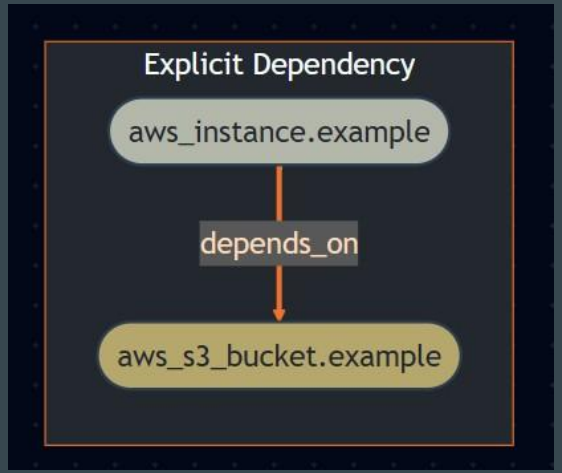
Since in aws_instance resource there is a reference to the ID of the aws_security_group resource, Terraform automatically understands that the security group must be created before the EC2 instance

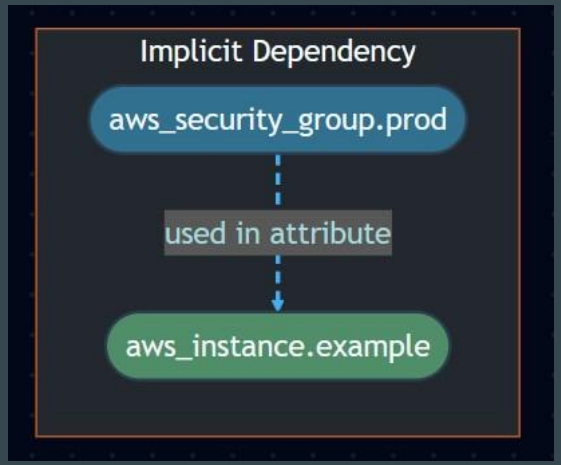


```
resource "aws_instance" "example" {
  ami                    = "ami-0e449927258d45bc4"
  instance_type          = "t2.micro"
  vpc_security_group_ids = [aws_security_group.prod.id]
}

resource "aws_security_group" "prod" {
  name        = "production-sg"
}
```

Final Revision





SET - Data Type

Revising List Data Type

Lists are used to store multiple items in a single variable.

These items can be duplicates as well.

```
C:\Users\zealv\kplabs-terraform>terraform apply -auto-approve
var.my-list
  Enter a value: ["hello","world","hello"]


Changes to Outputs:
  + variable_output = [
      + "hello",
      + "world",
      + "hello",
    ]

You can apply this plan to save these new output values to the Terraform

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

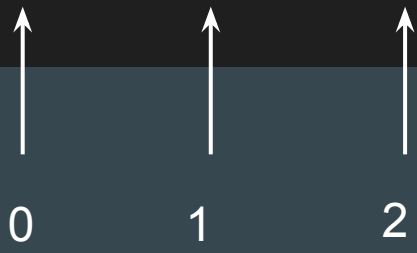
Outputs:

variable_output = tolist([
  "hello",
  "world",
  "hello",
])
```

List and Index

List items are indexed, the first item has index [0], the second item has index [1] etc.

```
variable "user_names" {
    type = list
    default = ["alice","bob","john"]
}
```



SET Data Type

Sets can only store unique elements.

Any duplicates are automatically removed.

```
C:\Users\zealv\kplabs-terraform>terraform apply -auto-approve
var.my-set
  Enter a value: ["hello","world","hello"]


Changes to Outputs:
  + variable_output = [
      + "hello",
      + "world",
    ]

You can apply this plan to save these new output values to the Terraform

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

variable_output = toset([
  "hello",
  "world",
])
```

Point to Note

While defining a SET, you need to also define the type of value that is expected.

```
variable "my-set" {
    type = set(string)
}
```

SET is Unordered

A set does not store the order of the elements.

Terraform only tracks the presence of elements, not their order.

If the elements in a set change order, Terraform won't detect that as a change. However, if an element is added or removed, Terraform will apply updates accordingly.

```
variable "user_names" {
    type = list
    default = ["alice","bob","john"]
}
```

```
variable "user_names" {
    type = set(string)
    default = ["john","bob","alice"]
}
```

The for_each Meta-Argument

Setting the Base

By default, a resource block configures one real infrastructure object.

However, sometimes you want to manage several similar objects (like a fixed pool of compute instances) without writing a separate block for each one.

Terraform has two ways to do this: count and for_each.

```
resource "aws_iam_user" "lb" {
  name = "alice"
}
```

Creating 5 IAM Users

If we want to create multiple resources with different configuration, we have to add multiple different resource blocks.

iam.tf

```
resource "aws_iam_user" "lb" {
  name = "alice"
}

resource "aws_iam_user" "lb" {
  name = "bob"
}

resource "aws_iam_user" "lb" {
  name = "johh"
}

resource "aws_iam_user" "lb" {
  name = "james"
}

resource "aws_iam_user" "lb" {
  name = "will"
}
```

Introducing for_each

If a resource block includes a for_each meta argument whose value is a map or a set of strings, Terraform creates one instance for each member of that map or set.

```
variable "user_names" {
    type = set(string)
    default = ["alice","bob"]
}

resource "aws_iam_user" "lb" {
  for_each = var.user_names
  name = each.value
}
```



```
C:\kplabs-terraform>terraform plan

Terraform used the selected providers to generate the following
following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_user.lb["alice"] will be created
  + resource "aws_iam_user" "lb" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "alice"
      + path          = "/"
      + tags_all      = (known after apply)
      + unique_id     = (known after apply)
    }

  # aws_iam_user.lb["bob"] will be created
  + resource "aws_iam_user" "lb" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "bob"
      + path          = "/"
      + tags_all      = (known after apply)
      + unique_id     = (known after apply)
    }
```

Point to Note

In blocks where for_each is set, an additional each object is available.

These object has two attributes:

Each Object	Description
each.key	The map key (or set member) corresponding to this instance.
each.value	The map value corresponding to this instance

Example - for_each with Map

When for_each is used with map, we can make use of each object to extract both key and value from the given map.

```
variable "mymap" {
    default = {
        dev = "ami-123"
        prod = "ami-456"
    }
}

resource "aws_instance" "web" {
  for_each      = var.mymap
  ami           = each.value
  instance_type = "t3.micro"

  tags = {
    Name = each.key
  }
}
```

Data Type - Object

Revising Map Data Type

In a basic setup, a map is a collection of key-value pairs where all values must be of the same type whereas keys are string.

Strict structure is not required explicitly.

```
variable "my-map" {
  type = map(number)
}

output "variable_value" {
  value = var.my-map
}
```



```
C:\kplabs-terraform>terraform plan
var.my-map
  Enter a value: {"Name"="123","Location"="456"}


Changes to Outputs:
  + variable_value = {
      + Location = 456
      + Name     = 123
    }
```

Introducing Object Data Type

An object is also a collection of key-value pairs, but each value can be of a different type.

A proper structure is generally required while defining object data type.

```
variable "my-object" {
  type = object({Name = string, userID = number})
}

output "variable_value" {
  value = var.my-object
}
```



```
C:\kplabs-terraform>terraform plan
var.my-object
  Enter a value: {"Name"="Zeal","userID"=123}


Changes to Outputs:
  + variable_value = {
      + Name   = "Zeal"
      + userID = 123
    }
```

Proper Structure is Required

It is important to have a structure of attributes allowed as part of the object.

```
variable "my-map" {
  type = map
}

variable "my-object" {
  type = object
}
```



This will work

This will NOT work

Keep in Mind the Syntax

object(...): a collection of named attributes that each have their own type.

The schema for object types is { <KEY> = <TYPE>, <KEY> = <TYPE>, ... } — a pair of curly braces containing a comma-separated series of <KEY> = <TYPE> pairs.

Extra attributes are discarded during type conversion.

Example - Extra Attributes Provided

In the following example, an additional key=value pair is provided that is not part of the object structure. We see on how it gets discarded in the output.

```
variable "my-object" {
  type = object({Name = string, userID = number})
}

output "variable_value" {
  value = var.my-object
}
```



```
C:\kplabs-terraform>terraform plan
var.my-object
  Enter a value: {"Name"="Zeal","userID"="123","email"="instructors@kplabs.in"}


Changes to Outputs:
  + variable_value = {
      + Name   = "Zeal"
      + userID = 123
    }
```

Relax and Have a Meme Before Proceeding



Input Variable Validation

Understanding the Challenge

Many times, the terraform plan would run properly without errors; however, as soon as you run “terraform apply”, you get an error.

```
C:\kplabs-terraform>terraform plan

Terraform used the selected providers to generate the following
following symbols:
  + create

Terraform will perform the following actions:

  # aws_s3_bucket.examplebucket will be created
  + resource "aws_s3_bucket" "examplebucket" {
      + acceleration_status         = (known after apply)
      + acl                         = (known after apply)
      + arn                         = (known after apply)
      + bucket                      = "hi"
      + bucket_domain_name          = (known after apply)
      + bucket_prefix               = (known after apply)
      + bucket_regional_domain_name = (known after apply)
      + force_destroy               = false
```

Plan Stage

```
aws_s3_bucket.this: Creating...

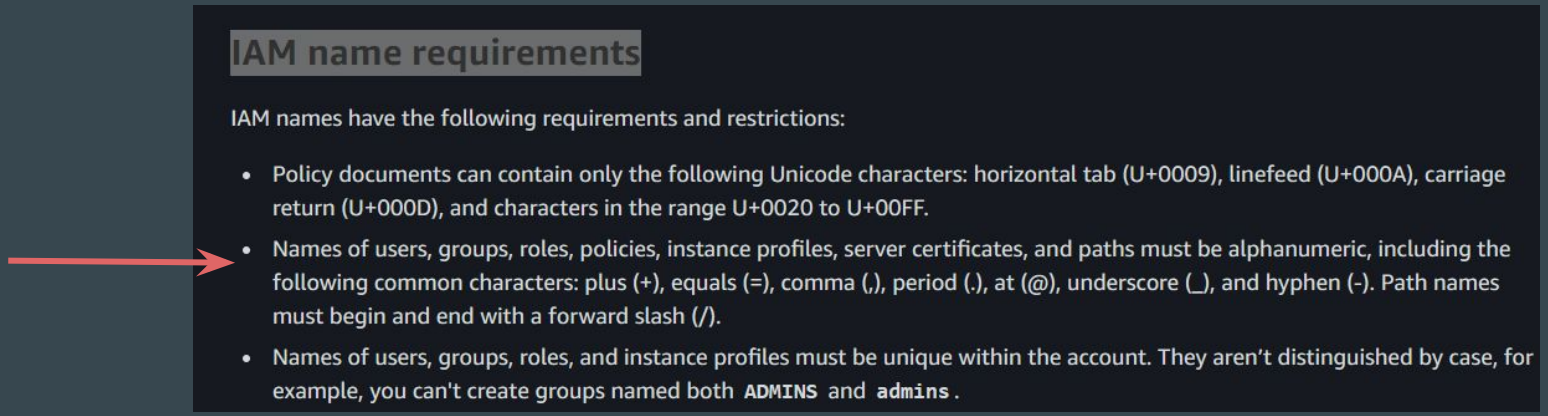
Error: creating S3 Bucket (hi): operation error S3: CreateBucket, https response error
T87GNG0E, HostID: W/rGOHVr4EyBDQhKsGWWrrY9HLx/NteANO+eOUn14g9G2r2HgyDJ3djNPaBk9bX8zYlvirI
The specified bucket is not valid.

   with aws_s3_bucket.this,
   on tf-logs.tf line 4, in resource "aws_s3_bucket" "this":
    4: resource "aws_s3_bucket" "this" {
```

Apply Stage

Reason is Due to Lack of Validation

Every service has some restrictions and limits on aspects like naming convention, capacity, etc.



IAM name requirements

IAM names have the following requirements and restrictions:

- Policy documents can contain only the following Unicode characters: horizontal tab (U+0009), linefeed (U+000A), carriage return (U+000D), and characters in the range U+0020 to U+00FF.
- Names of users, groups, roles, policies, instance profiles, server certificates, and paths must be alphanumeric, including the following common characters: plus (+), equals (=), comma (,), period (.), at (@), underscore (_), and hyphen (-). Path names must begin and end with a forward slash (/).
- Names of users, groups, roles, and instance profiles must be unique within the account. They aren't distinguished by case, for example, you can't create groups named both `ADMINS` and `admins`.

Terraform and Provider Validation

For many of the resources, Terraform and it's associated providers will validate the input so that any potential errors can be detected quickly.

```
resource "aws_iam_user" "dev" {
  name = "kplabs-user-01#"
}
```



```
C:\kplabs-terraform>terraform plan

 Error: invalid value for name (must only contain alphanumeric characters, hyphens, underscores, commas, periods, @ symbols,
plus and equals signs)

   with aws_iam_user.dev,
   on tf-logs.tf line 5, in resource "aws_iam_user" "dev":
    5:   name = "kplabs-user-01#"
```

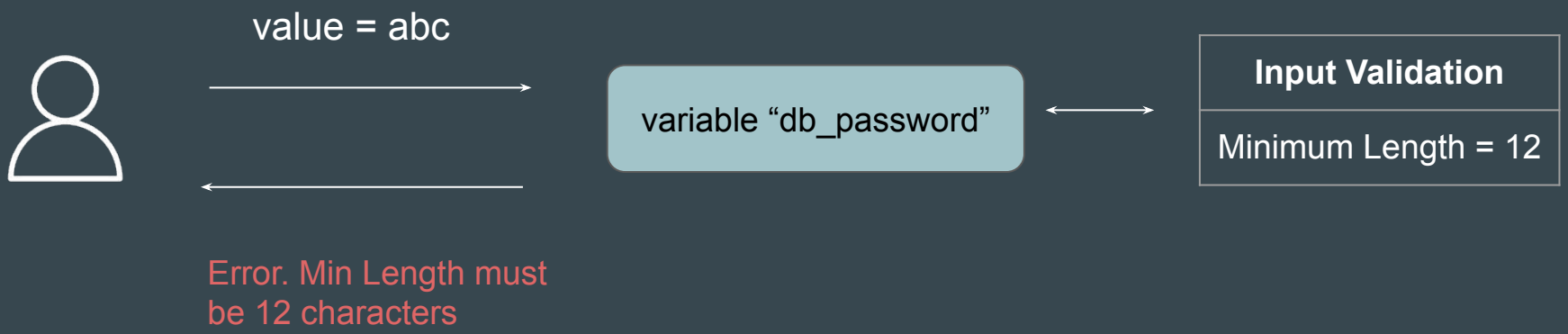
Terraform and Provider Validation

The provider-side validation works for many of the resources but NOT all of the resources.

Hence, it is essential to have user-side validation to ensure consistency and conformance to the standards.

Input Variable Validation

The input variable validation feature allows you to enforce certain rules or constraints on the values that can be assigned to input variables.



Reference Screenshot

```
C:\kplabs-terraform>terraform plan
var.db_password
  Password for the database

  Enter a value: abc


Planning failed. Terraform encountered an error while generating this plan.


  Error: Invalid value for variable

    on tf-logs.tf line 3:
     3: variable "db_password" {
       ├────────────────
       │ var.db_password is "abc"

  Database password must be at least 12 characters long.

  This was checked by the validation rule at tf-logs.tf:7,3-13.
```

Importance of Validation

It allows organizations to ensure consistency and adhere to organizational best practices.

Allows organizations to make make your Terraform code more predictable.

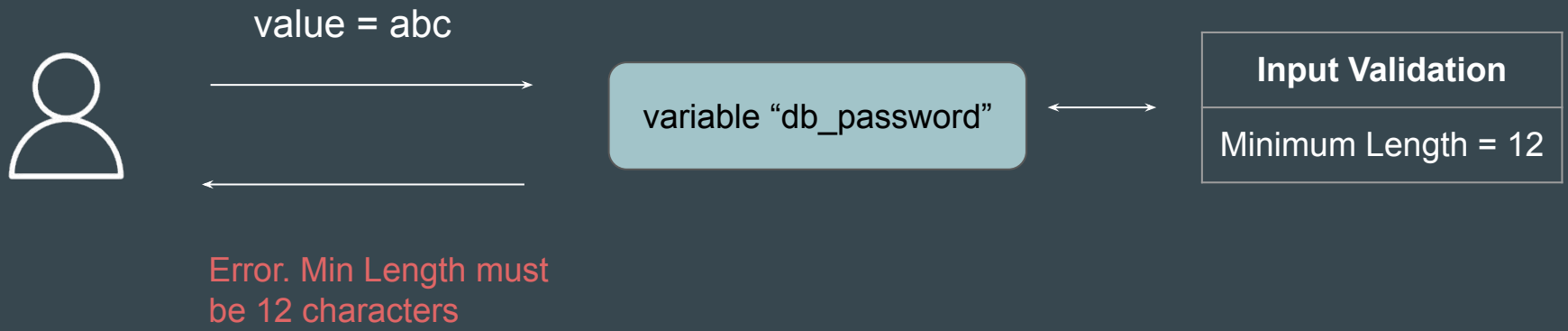
Allows organizations to catch misconfigurations at an early stage, saving time and potential infrastructure issues.

Input Variable Validation - Practical

Scenario To Consider for Practical

The value for “db_password” variable must be of minimum 12 characters length.

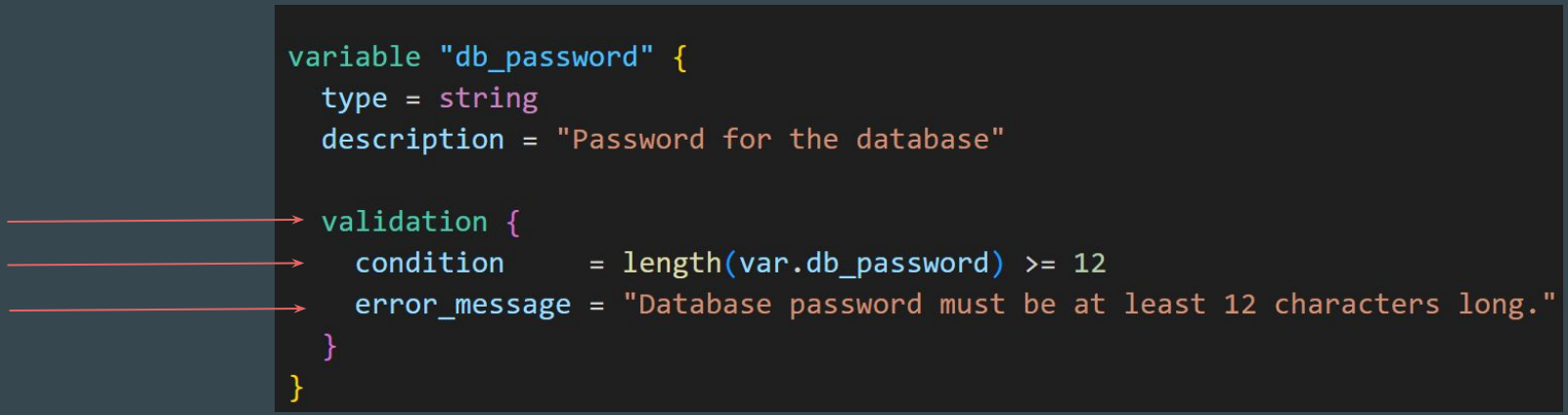
If user enters anything less than 12, Terraform should thrown an error.



Understanding the Structure

There are three important components of input variable validation feature.

Validation Block, Condition, and Error Message



Understanding Each Components

Validation block is used within a variable declaration to define one or more conditions that the variable's value must satisfy.

Condition is a boolean expression that must evaluate to true for the validation to pass.

The error message is a string that is displayed when the condition fails.

```
variable "db_password" {
  type = string
  description = "Password for the database"

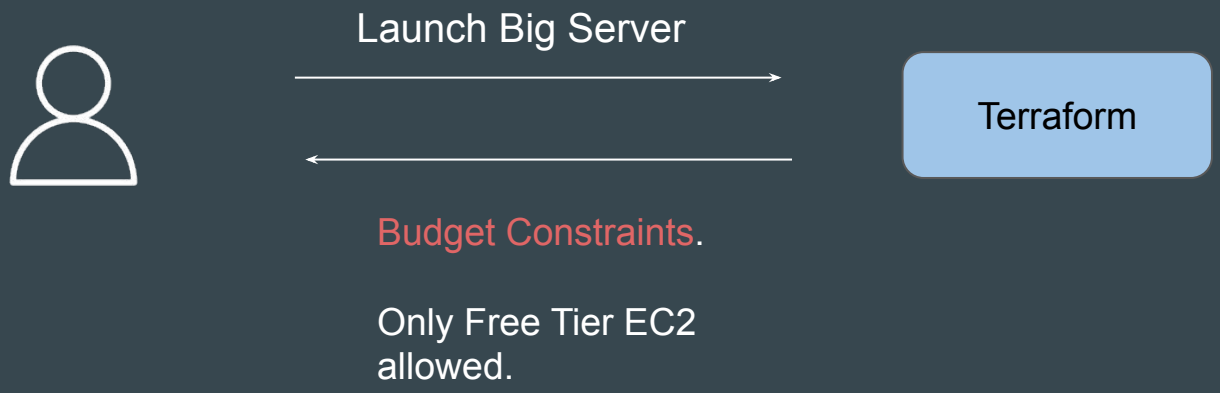
  validation {
    condition     = length(var.db_password) >= 12
    error_message = "Database password must be at least 12 characters long."
  }
}
```

Preconditions and Postconditions

Introducing Pre-Conditions

Terraform preconditions are custom conditions that are checked BEFORE evaluating the object they are associated with.

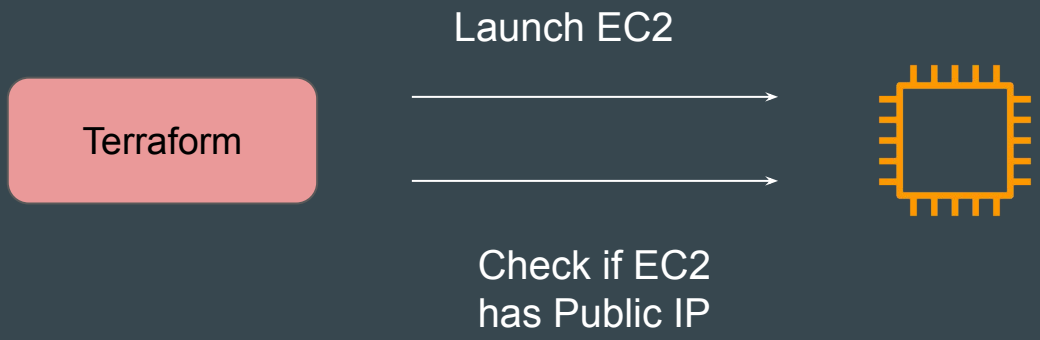
Example: Launch EC2 instance only if it is part of free tier



Introducing Post-Conditions

Terraform postconditions are custom conditions that are checked AFTER evaluating the object they are associated with.

Example: Verify if EC2 has Public IP address after it has been created.



Overall Syntax

The precondition and postcondition are defined inside the lifecycle block.

```
resource "aws_instance" "example" {
  instance_type = "t3.micro"
  ami           = "ami-123"

  lifecycle {
    precondition {
      condition     = logic-here
      error_message = "Error Message Here"
    }

    postcondition {
      condition     = logic-here
      error_message = "Error Message Here"
    }
  }
}
```

Reference Code

```
variable "instance_type" {}

data "aws_ec2_instance_type" "example" {
  instance_type = var.instance_type
}

resource "aws_instance" "example" {
  instance_type = var.instance_type
  ami           = "ami-066784287e358dad1"

  lifecycle {

    precondition {
      condition = data.aws_ec2_instance_type.example.free_tier_eligible
      error_message = "Instance Type is not part of free tier"
    }

    postcondition {
      condition = self.public_dns == ""
      error_message = "Public IPV4 or DNS is mandatory for this server"
    }
  }
  }
```

Point to Note

You can use the self object in postcondition blocks to refer to attributes of the instance under evaluation.

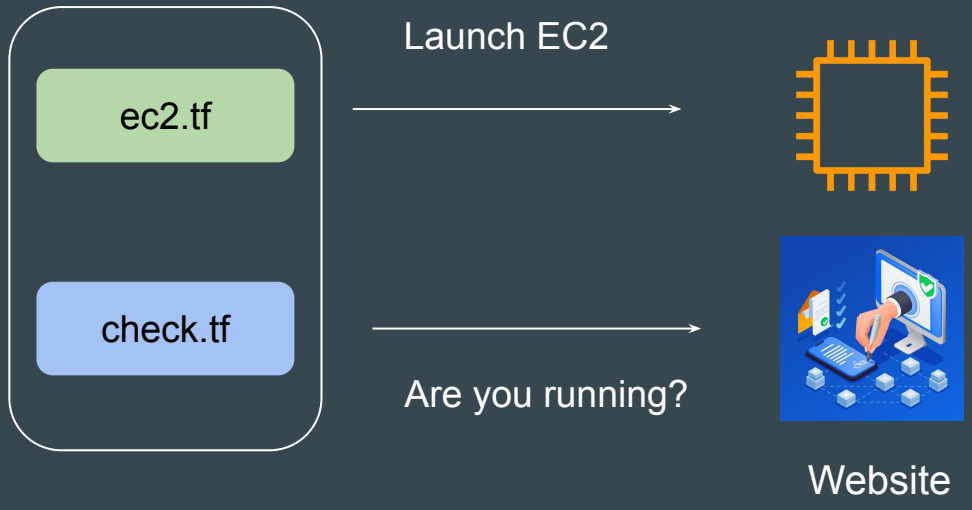
Preconditions and postconditions are available in Terraform v1.2.0 and later.

Preconditions and Postconditions are supported for resources, data sources, and outputs

Check Blocks

Basics of Check Blocks

The check block can validate your infrastructure outside the usual resource lifecycle.



Using Data Source + Instance Block

In default configuration, if you use data source block for checking website along with other resource block, if data block fails, Terraform will error out.

```
data "http" "example" {
  url = "https://google1212.com"
}

resource "local_file" "foo" {
  content  = "Hi"
  filename = "${path.module}/foo.txt"
}
```



```
Plan: 1 to add, 0 to change, 0 to destroy.

Error: Error making request

  with data.http.example,
  on check-block.tf line 14, in data "http" "example":
  14: data "http" "example" {

Error making request: GET https://google1212.com giving up after 1 attempt(s):
lookup google1212.com: no such host
```

Workflow of Check Block

Check block can include the nested data source block

If a scoped data source's provider raises any errors, they are masked as warnings and do not prevent Terraform from continuing operation execution.

```
check "website_check" {
  data "http" "terraform_io" {
    url = "https://google1212.com"
  }
```



```
Plan: 1 to add, 0 to change, 0 to destroy.
local_file.foo: Creating...
local_file.foo: Creation complete after 0s [id=94dd9e08c129c785f7f256e82fbe0a30e
data.http.terraform_io: Reading...

  Warning: Error making request

    with data.http.terraform_io,
    on check-block.tf line 25, in check "website_check":
    25:   data "http" "terraform_io" {

  Error making request: GET https://google1212.com giving up after 1 attempt(s):
  lookup google1212.com: no such host


Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Format of Check Block

Each check block must have at least one assert blocks.

Each assert block has a condition attribute and an error_message attribute.

```
check "website_check" {
  data "http" "terraform_io" {
    url = "https://google.com"
  }

  assert {
    condition = your-condition-here
    error_message = "your error message here."
  }
}
```

Point to Note - Check Blocks

Assertions do not affect Terraform's execution of an operation. A failed assertion reports a warning without halting the ongoing operation.

This contrasts with other custom conditions, such as a postcondition, where Terraform produces an error immediately, halting the operation and blocking the application or planning of future resources.

Sample Use-Case - AWS Budgets

Check the AWS Budget and warn if Budget has exceeded.

```
check "check_budget_exceeded" {
  data "aws_budgets_budget" "example" {
    name = aws_budgets_budget.example.name
  }

  assert {
    condition = !data.aws_budgets_budget.example.budget_exceeded
    error_message = "budet exceeded"
  }
}
```

```
Warning: Check block assertion failed

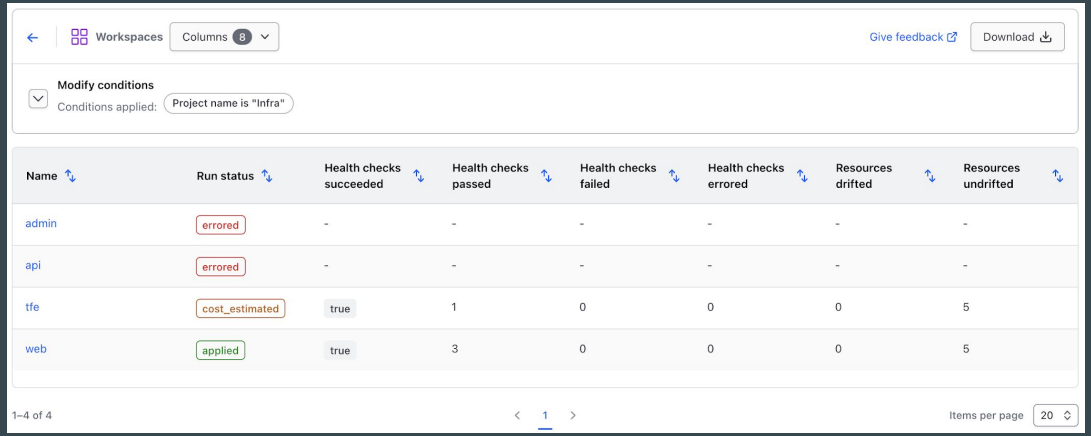
  on main.tf line 43, in check "check_budget_exceeded":
  43:     condition = !data.aws_budgets_budget.example.budget_exceeded
    ├────────────────
    │ data.aws_budgets_budget.example.budget_exceeded is true

AWS budget has been exceeded! Calculated spend: '1550.0' and budget limit: '1200.0'
```

Point to Note

Checks becomes more useful when they are regularly run in automation and can alert us if they fail.

HCP Terraform can use check blocks to continuously monitor health and provide notifications using continuous validation feature.



Moved Blocks

Setting the Base

In shared modules and long-lived configurations, you may eventually outgrow your initial module structure and resource names.

```
resource "aws_security_group" "database_firewall" {
  name        = "db_firewall"
}
```




```
resource "aws_security_group" "payments_database_firewall" {
  name        = "db_firewall"
}
```

Understanding the Challenge

Terraform understands moving or renaming an object as an intent to destroy the object at the old address and to create a new object at the new address.

```
resource "aws_security_group" "database_firewall" {
  name        = "db_firewall"
}

resource "aws_security_group" "payments_database_firewall" {
  name        = "db_firewall"
}
```

Rename

After rename

```
Terraform used the selected providers to generate the following execution plan.
Following symbols:
  + create
  - destroy

Terraform will perform the following actions:

  # aws_security_group.database_firewall will be destroyed
  # (because aws_security_group.database_firewall is not in configuration)
  - resource "aws_security_group" "database_firewall" {
      - arn                    = "arn:aws:ec2:us-east-1:042025557788:security-gr
      - description            = "Managed by Terraform" -> null
      - egress                 = [] -> null
      - id                     = "sg-02c7e7f0185c885f6" -> null
      - ingress                = [] -> null
      - name                   = "db_firewall" -> null
      - owner_id               = "042025557788" -> null
      - revoke_rules_on_delete = false -> null
      - tags                   = {} -> null
      - tags_all               = {} -> null
      - vpc_id                 = "vpc-069367557b523ef5a" -> null
        # (1 unchanged attribute hidden)
    }

  # aws_security_group.payments_database_firewall will be created
  + resource "aws_security_group" "payments_database_firewall" {
      + arn                    = (known after apply)
```

Introducing Moved Blocks

Using moved block, Terraform treats an existing object at the old address as if it now belongs to the new address.

```
resource "aws_security_group" "payments_database_firewall" {
  name         = "db_firewall"
}

moved {
    from = aws_security_group.database_firewall
    to   = aws_security_group.payments_database_firewall
}
```



```
C:\kplabs-terraform\1>terraform plan
aws_security_group.payments_database_firewall: Refreshing state... [id=sg-02c7e7f0185c885f6]

Terraform will perform the following actions:

  # aws_security_group.database_firewall has moved to aws_security_group.payments_database_firewall
    resource "aws_security_group" "payments_database_firewall" {
        id                     = "sg-02c7e7f0185c885f6"
        name                   = "db_firewall"
        tags                   = {}
        # (9 unchanged attributes hidden)
    }

Plan: 0 to add, 0 to change, 0 to destroy.
```

Moved Block Syntax

A moved block expects no labels and contains only from and to arguments:

```
moved {
  from = aws_instance.a
  to   = aws_instance.b
}
```

Point to Note

Both terraform state mv and moved block allow us to achieve a similar set of use cases.

One benefit of moved block is that the blocks are more visible for all team members to know of.

terraform state mv can be used in more complex scenarios as it can also support scripting for bulk operations.

Terraform Provisioners

Setting the Base

We have been using Terraform to create and manage resources for a specific provider.

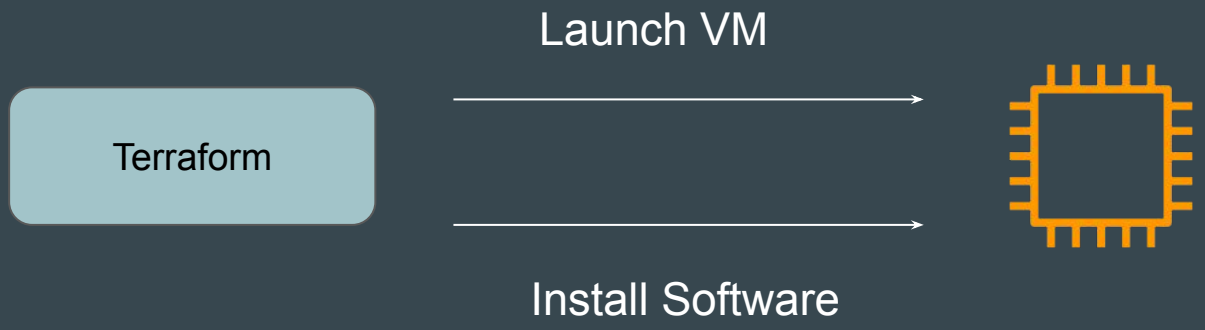
Organizations would want end-to-end solution for creation of infrastructure and configuring appropriate packages required for the application.



Introducing Provisioners

Provisioners are used to execute scripts on a local or remote machine as part of resource creation or destruction.

Example: After VM is launched, install software package required for application.

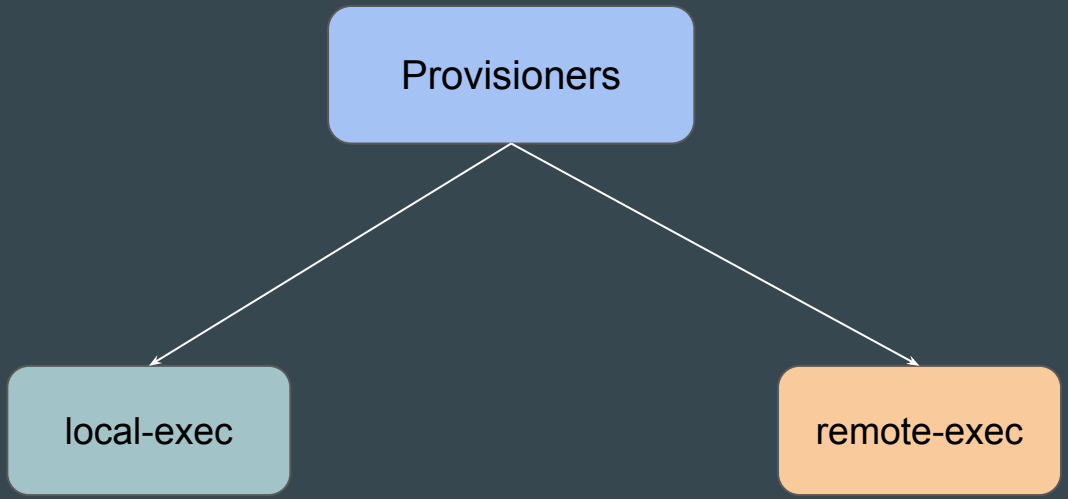


Types of Provisioners in Terraform

Setting the Base

Provisioners are used to execute scripts on a **local** or **remote** machine as part of resource creation or destruction.

There are 2 major types of provisioners available



Type 1 - local-exec provisioner

The local-exec provisioner invokes a local executable after a resource is created.

Example: After EC2 is launched, fetch the IP and store it in file server_ip.txt

